



DIPLOMA THESIS

Modular Smart Home System

free and open source

Performed in 2024/25 by:

Fabian Schätzschock
Richard Krammer

Supervisors:

Ing. Reinhard Rzepa
Ing. Rudolf Janeczek BEd MSc

St. Pölten, on 04.04.2025

Submission Statement:
Date:

Supervisors:

Affidavit

The undersigned candidates have chosen to prepare a diploma thesis with the following task description in accordance with the Schulunterrichtsgesetz (School Education Act) § 34 Abs. 3 Z 1 and § 37 Abs. 2 Z 2 [4], in conjunction with the provisions of the Prüfungsordnung BMHS (Examination Regulations for Vocational Middle and Higher Schools), Federal Law Gazette II No. 177/2012 [22], as amended

Modular Smart Home System free and open source

Individual tasks within the overall project:

- **Fabian Schätzschock:** Low-level software development, server-side front-end and back-end development
- **Richard Krammer:** Component selection, circuit design, and hardware development

The candidates acknowledge that the diploma thesis must be worked on and completed independently and outside of class time, although class results may be incorporated if appropriately cited as such.

The complete diploma thesis must be submitted digitally and in two printed copies to the supervising teacher no later than **04.04.2025**.

The candidates further acknowledge that cancellation of the diploma thesis is not possible.

Fabian Schätzschock

Richard Krammer

DIPLOMARBEIT

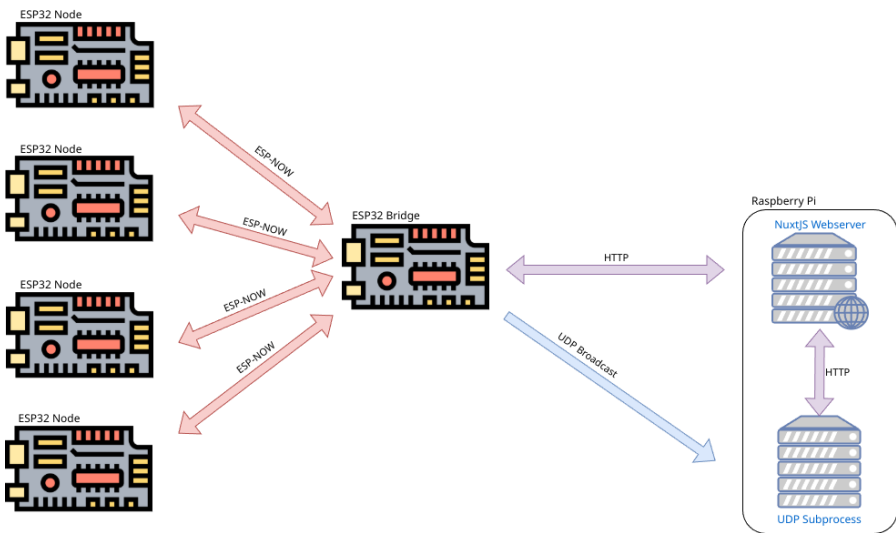
DOKUMENTATION

Namen der Verfasser/innen	Fabian Schätzschock / Richard Krammer
Jahrgang / Klasse Schuljahr	5BHELS / 2024/2025
Thema der Diplomarbeit	Modular Smart Home System
Kooperationspartner	-

Aufgabenstellung	Da viele Smart Home Lösungen auf dem Markt proprietär und wenig anpassungsfähig sind, soll ein Konzept eines Smart Homes gebaut werden, welches open source und einfach anpassbar ist. Das Hauptmerkmal des Systems ist die Modularität, dass bedeutet Module mit unterschiedlichen Funktionen sollen austauschbar sein. Die Tools und Technologien, die in diesem Projekt nötig sind, sollten für jeden Interessenten einfach und gratis verfügbar sein. Zum Erweitern des Systems soll eine Library zur einfachen Verwendung der relevanten Funktionalitäten verfügbar sein.
------------------	--

Realisierung	Server: Ein Raspberry Pi 4, der als Webserver und Userschnittstelle fungiert. Bridge Node: Ein ESP32-S3 Dev-Modul mit Ethernet Buchse und einem Anschluss für eine Antenne, welches als Schnittstelle aller Nodes und dem Webserver fungiert. Slave Nodes: Ein Teil des Smart Homes bestehend aus einem Basis-Modul und einem Modul mit beliebiger Funktionalität welches einfach angesteckt werden kann. Eine Slave-Node misst Daten oder kontrolliert Aktoren. Im Rahmen dieser Arbeit wurden dafür zwei aufsteckbare Zusatzmodule entwickelt.
--------------	--

Ergebnisse	Die Hardware, Kommunikationswege und Node-Funktionalitäten wurden erfolgreich entwickelt. Ein funktionsfähiges Beispiel eines Informationsaustausches wurde getestet. Ein Prototyp eines Webinterfaces welches Daten anfordert wurde realisiert inklusive des Networking- und ESP-NOW-Discovery Prozesses.
------------	---

Typische Grafik, Foto etc. (mit Erläuterung)	 Konzept des Smart Home Systems und dessen Kommunikation.	
Möglichkeiten der Einsichtnahme in die Arbeit	Die Diplomarbeit ist nicht gesperrt. Sie kann an der HTL St. Pölten, Waldstraße 3, 3100 St. Pölten Eingesehen werden	
Approbation (Datum / Unterschrift)	Prüfer:in	Dipl.-Ing. W. U. KURAN Abteilungsvorstand

DIPLOMA THESIS

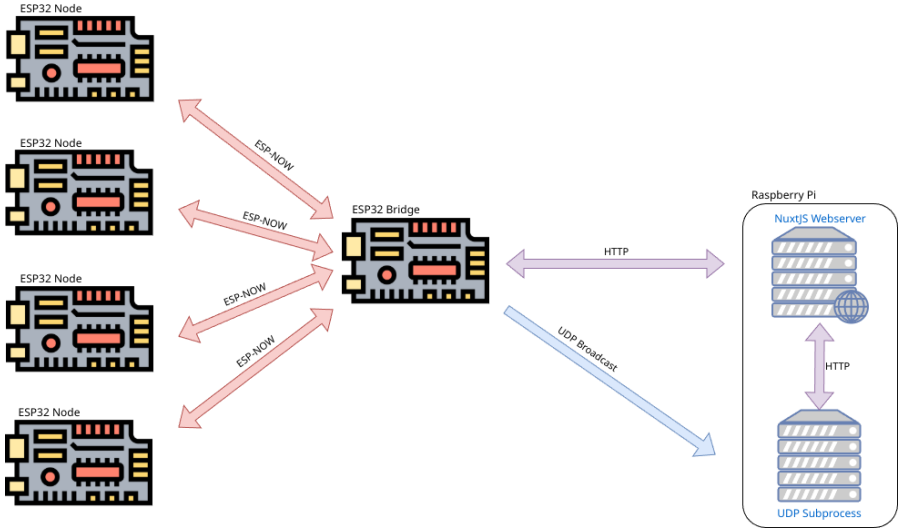
DOCUMENTATION

Author(s)	Fabian Schätzschock / Richard Krammer
Form Academic year	5BHELS / 2024/2025
Topic	Modular Smart Home System
Cooperation partners	-

Assignment of tasks (conceptual formulation / job definition)	Since many smart home solutions on the market are proprietary and lack adaptability, a concept for a smart home will be developed that is open-source and easily customizable. The main feature of the system is modularity, meaning that modules with different functions should be interchangeable. The tools and technologies required for this project should be easily accessible and free for anyone interested. To facilitate system expansion, a library will be available to simplify the use of relevant functionalities.
---	---

Realisation	Server: A Raspberry Pi 4 that functions as a web server and user interface. Bridge Node: An ESP32-S3 development module with an Ethernet port and an antenna connector, serving as the interface between all nodes and the web server. Slave Nodes: A part of the smart home system consisting of a base module and an attachable module with any desired functionality. A slave node collects data or controls actuators. As part of this work, two additional modules have been developed.
-------------	--

Results	The hardware, communication paths, and node functionalities have been successfully developed. A functional example of data exchange has been tested. A prototype of a web interface that requests data has been implemented, including the networking and ESP-NOW discovery process.
---------	---

Illustrative graph, photo (incl. explanation)	 Concept of the Smart Home Systems and its communication.
--	---

Accessibility of diploma thesis	The diploma thesis is not restricted. It can be viewed at: HTL St. Pölten, Waldstraße 3, 3100 St. Pölten.
---------------------------------	---

Approval (Date / Sign)	Examiner	Dipl.-Ing. W. U. KURAN Head of Department
---------------------------	----------	--

Contents

1	Introduction	2
1.1	Our Ambitions	2
1.2	Terminology	2
1.3	List of abbreviations	2
2	Executive Summary	4
2.1	Project Overview	4
2.2	Objectives	4
2.3	Key Features	4
2.4	Conclusion	4
3	Used technologies	5
3.1	Arduino	5
3.2	IDE	5
3.2.1	Visual Studio Code	6
3.2.2	PlatformIO	6
3.3	FreeRTOS	6
3.4	Communication	6
3.4.1	Note-to-Note	6
3.4.2	Note-to-Server	7
3.5	Nuxt.js	8
3.6	Flask	8
3.7	Docker	8
3.8	Altium Designer®24	9
3.9	Autodesk Fusion 360®	9
4	Networking	10
4.1	Overview	10
4.1.1	Web Server	10
4.1.2	Bridge-Node	11
4.1.3	Slave-Nodes	11
4.2	Device Discovery	11
4.2.1	Network Discovery	11
4.2.2	ESP-NOW Discovery	13
5	Nodes	15
5.1	Project Configuration	15
5.1.1	platformio.ini	15
5.1.2	Enabling the Serial Monitor	16
5.1.3	Including FaRiLib	16

5.1.4	Complete Example	16
5.2	FaRiLib	17
5.2.1	Library Structure	17
5.2.2	ESP NOW	18
5.2.3	HTTP	19
5.3	Project Structure	19
5.3.1	Slave Node	19
5.3.2	Bridge Node	20
6	Modules	21
6.1	Basic Module	21
6.1.1	Hardware - V2.2(Final)	21
6.1.2	Hardware - V2.1	31
6.1.3	Hardware - V2.0	32
6.1.4	Hardware - V1.1	35
6.1.5	Hardware - V1.0	39
6.1.6	Base Software	41
6.2	Bridge Node	42
6.2.1	Hardware	42
6.3	Motion Detector Module	43
6.3.1	Hardware - V1.1(Final)	44
6.3.2	Hardware - V1.0	46
6.4	Battery Module	49
6.4.1	Hardware - V2.2(Final)	49
6.4.2	Hardware - V2.1	54
6.4.3	Hardware - V1.0	55
6.5	Switch Module	58
6.5.1	Hardware - V1.1(Final)	59
6.5.2	Hardware - V1.0	66
6.6	Case	68
6.6.1	Basic	68
6.6.2	Battery	70
6.6.3	Motion	72
6.6.4	Switch	73
6.6.5	Combined Module	74
7	Server	76
7.1	Operating System	76
7.1.1	Raspberry Pi OS Lite	76
7.1.2	Other ARM-based OS	77
7.2	Web Server	77
7.2.1	Installing Nuxt.js	77

7.2.2	Requesting Data	78
7.3	UDP Process	79
7.3.1	Start on boot	79
7.4	Distribution	79
7.4.1	Image	80
7.4.2	Setup Script	80
7.4.3	Containerization	80
8	Results	83
8.1	Future Plans	83
8.1.1	Circumventing the ESP-NOW Peer Limit	83
8.1.2	Hot-swapping Modules	83
8.1.3	Database	83
8.2	Node Range Tests	83
8.2.1	Measuring Procedure	84
8.2.2	Open Field	84
8.2.3	Indoor Test	85
8.2.4	Conclusion	85
8.3	Costs	85
8.3.1	Base Module	86
8.3.2	Battery Module	86
8.3.3	Switch Module	86
8.3.4	Motion Module	86
9	Time Tracking	87
9.1	Fabian Schätzschock	87
9.2	Richard Krammer	88
10	Production Files	89
10.1	Basic Module	89
10.2	Motion Module	102
10.3	Battery Module	108
10.4	Switch Module	114
	Bibliography	124
	List of Figures	128

Acknowledgement

We would like to express our deepest gratitude to our supervisor, Professor Ing. Reinhard Rzepa, for his invaluable guidance and support throughout the course of this project. His expertise and insights have been instrumental and have greatly contributed to the success of this work.

We also extend my sincere thanks to the faculty and staff of the HTBLuVA St. Pölten for providing the necessary resources and environment for our studies.

Finally, we would like to acknowledge the support of our families and friends, whose encouragement has been a source of great strength and motivation.

1 Introduction

1.1 Our Ambitions

Most free smart home solutions are either too limited in functionality or dependent on proprietary software. The goal of this project is to provide an open source smart home solution that enables the user to easily modify and extend the system with their own design. Another goal of this project is to provide a smart home solution that does not require every node to be reachable via WLAN. Finally, the smart home devices functionalities should be interchangeable in a live environment. This means that the user can change the functionality of a node without having to reprogram it.

1.2 Terminology

- **Node:** A whole smart home device that consists of a base module and optional extension modules.
- **Module:** A part of a Node that provides a specific functionality. An extension module can be attached to a Node to extend its functionality.

1.3 List of abbreviations

- **ESP:** Espressif Systems
- **HTTP:** Hypertext Transfer Protocol
- **UDP:** User Datagram Protocol
- **GPIO:** General Purpose Input/Output
- **IDE:** Integrated Development Environment
- **CMake:** Cross-platform Make
- **PIR:** Passive Infrared Sensor
- **PCB:** Printed Circuit Board
- **USB:** Universal Serial Bus
- **UART:** Universal Asynchronous Receiver-Transmitter
- **ADC:** Analog-to-Digital Converter

- **ARGB:** Addressable RGB
- **SoC:** System on Chip
- **MCU:** Microcontroller Unit
- **PLA:** Polylactic Acid

2 Executive Summary

This page serves as a brief overview of the Modular Smart Home System project. It provides a summary of the project's goals, objectives, and key features.

2.1 Project Overview

The Modular Smart Home System project aims to develop an open-source, all-in-one smart home solution with a modular design. This system allows users to easily modify and extend its functionalities via extension slots to suit their specific needs.

2.2 Objectives

The primary objectives of this project are:

- To create a user-friendly smart home system that integrates self-made home automation devices.
- To ensure the system is modular, allowing for easy customization and expansion.
- Modules can be added or removed on the fly without having to adjust code.
- To provide comprehensive documentation and support for users and developers.

2.3 Key Features

The key features of the Modular Smart Home System include:

- A frontend interface for monitoring and controlling the smart home system.
- Easily changeable modules that changes the Nodes functionality.
- An easily understandable and extendable codebase.

2.4 Conclusion

The Modular Smart Home System project aims to deliver a versatile and customizable smart home solution that meets the needs of a wide range of users. By focusing on modularity and ease of use, the project seeks to empower users to create their ideal smart home environment.

3 Used technologies

The following chapter will outline the requirements for the project. It describes what concepts and technologies are needed to implement the project.

3.1 Arduino



The Arduino framework [29] is an open source platform that provides a subset of C++ tailored for embedded systems with a vast ecosystem of libraries. It is widely used and beginner-friendly, making it the perfect choice for this project, due to the fact that the smart home should be easy to extend and modify.

3.2 IDE

For developing purposes an Environment fulfilling the following criteria is needed:

- **Integrates the Arduino framework:** Needs to integrate Arduino as most of the code will be written using it. (see section 3.1)
- **Support for different microcontrollers:** Supporting different microcontrollers, to allow for flexibility in developing new Nodes.
- **Manages Library integration:** Streamlines installing and managing libraries used in the project.
- **Free:** To allow anyone to easily modify or create their own smart home using this project as a base.
- **Compiling and uploading code:** Simplifying the process of uploading code to the microcontroller.
- **Edit Code through SSH:** Streamline the process of editing and executing code remotely

3.2.1 Visual Studio Code

Visual Studio Code [27] is a text editor developed by Microsoft that offers a wide range of features, including a VSCode server that allows for remote development. It also offers a wide range of extensions, including PlatformIO, which transforms it into a powerful IDE for developing microcontroller applications.



3.2.2 PlatformIO

PlatformIO [21] is an open-source ecosystem for microcontroller development that supports a wide range of microcontrollers including the ESP32 and ESP8266. It supports multiple frameworks like Arduino or ESP-IDF and can be integrated into Visual Studio Code, CLion or used through the command line. It also eases library inclusion and streamlines the compiling and uploading process.



3.3 FreeRTOS

FreeRTOS [12] is a market-leading open source embedded system RTOS supporting over 40 processor architectures including the ESP32 with a small memory footprint, fast execution times, and cutting-edge RTOS features.

A RTOS is necessary for this project as there are a lot of asynchronous tasks that need to be handled, like reading sensor data, controlling actuators, and handling communication between the nodes. Using interrupts would be too complex and error-prone, so the integrated RTOS is the preferred solution.



3.4 Communication

The following section will outline the communication protocols considered and used for the various communication channels.

3.4.1 Note-to-Note

Communication between nodes should be possible between the bridge and any node, all traffic should be routed through the bridge. The following protocols are considered:

MQTT



MQTT [17] is an OASIS standard messaging protocol for IoT-Applications that builds on top of TCP/IP and works on a publishing/subscribe model, which makes it easy to set up communication between devices without knowing their IP-Addresses. It requires a dedicated broker to function, that can be set up as a service on the Raspberry Pi. One mayor upside of using MQTT is that it is compatible with every, in this smart home involved, device.

ESP NOW



ESP-NOW [8] is a wireless communication protocol capable of working with Wi-Fi and Bluetooth. It supports a multitude of Espressif microcontrollers and is widely used in IoT-Applications. The main advantage of using ESP-NOW is that it requires no router or existing network to function meaning it's not dependent on a good Wi-Fi coverage. While indoor use drastically reduces the range of 200m per unit depending on various factors [5] the integrated multihop feature that allows the devices to relay messages to each other can be used to extend the range to hard-to-reach locations. One mayor downside of the protocol is that one peer can only communicate with up to 20 other peers.

Due to the listed advantages, ESP-NOW is the **preferred communication protocol** for this project.

3.4.2 Note-to-Server

Communication between the nodes and the web server, as well as the database, should be possible both ways.

MQTT

MQTT can be used to bridge the gap between the nodes and the server. The server can subscribe to the same topics as the nodes, and the nodes can publish to the same topics as the server.

Disadvantages: Unfortunately, MQTT integration for the Web server as well as the database is not as straightforward as the alternative and would require a lot more overhead.

HTTP

HTTP can be used to bridge the gap between the nodes and the server. This way a dedicated Node can be set up to handle the communication with both the Web server and the database and distribute it to the rest of the nodes.

For this project, HTTP is the preferred protocol for node-to-server communication.

3.5 Nuxt.js

The frontend should be able to visualize the data stored in the database, control the actuators and sensors, and display the current state of the system. To implement this, a high-level web framework is preferred to simplify the development process.

For this project, Nuxt.js [18] is used as the web framework. Nuxt.js is a framework for creating universal Vue.js applications. It is a high-level framework that simplifies the development of Vue.js applications by providing a set of conventions and best practices. It is built on top of Vue.js and provides a set of features that make it easy to create a web application.



3.6 Flask

Flask is a lightweight WSGI web application framework. It is designed to make getting started quick and easy. [28] In this thesis, Flask is not used in the traditional way, as it is not used to serve the web application, but rather to serve as a receiver of UDP packets from the bridge node (see section 4.2.1). This is done because Python offers a lot of libraries for listening to UDP packets and parsing the data. This data is then accessed through an HTTP-GET request from Nuxt.js when needed.



3.7 Docker

Docker is an open platform for developing, shipping, and running applications. Docker enables you to separate your applications from your infrastructure, so you can deliver software quickly. With Docker, you can manage your infrastructure in the same ways you manage your applications. By taking advantage of Docker's methodologies for shipping, testing, and deploying code, you can significantly reduce the delay between writing code and running it in production. [30]



3.8 Altium Designer®²⁴

Altium is a PCB design software that is used professionally in the industry. It offers a wide range of features such as simulating circuits, drawing schematics, and laying out PCBs. [19] All of the hardware is designed using Altium Designer. The software is used to create the schematic and the PCB layout.

3.9 Autodesk Fusion 360®

Fusion 360 is an all-in-one CAD, CAM, CAE, and PCB platform. [2] It is used in this project to design the enclosure for the PCBs.

4 Networking

The following chapter will outline the networking structure of the project. It describes how the different components of the smart home will communicate with each other and the user.

4.1 Overview

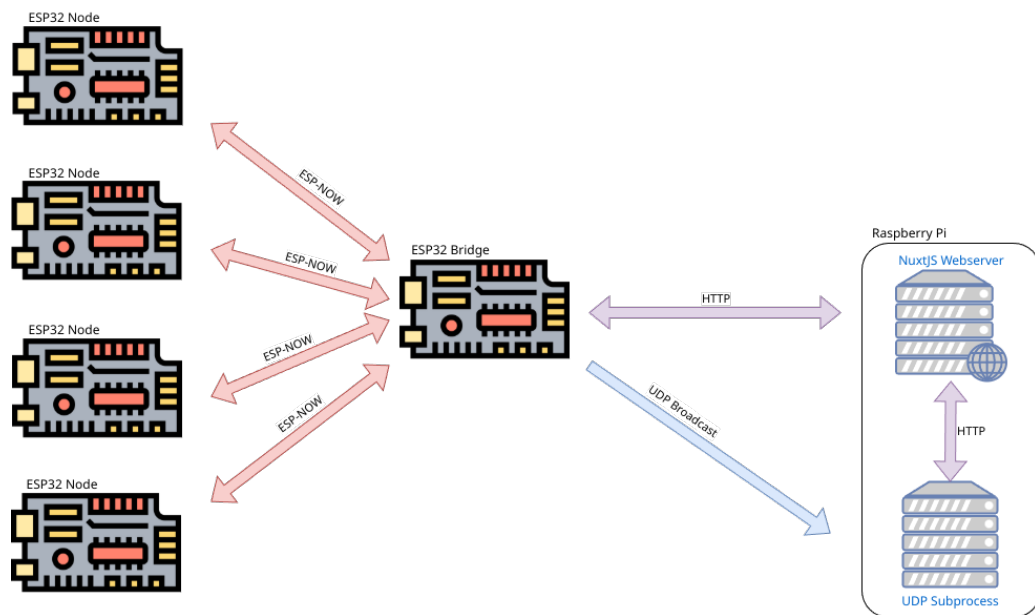


Figure 4.1: Networking Structure

4.1.1 Web Server

A Nuxt.js Web server will function as the main interface for the user. It will communicate with the database and the dedicated Bridge-Node via HTTP. It should be capable of displaying and changing the state of the devices in the smart home.

4.1.2 Bridge-Node

The Bridge-Node will act as a bridge between everything running on the Server and the ESP-NOW devices. It will be capable of receiving and sending data via both HTTP and ESP-NOW. It will also be responsible for discovering new ESP-NOW devices.

4.1.3 Slave-Nodes

The Slave-Nodes will be the ESP32 devices. They will be capable of sending and receiving data via ESP-NOW to and from the Bridge-Node. This only happens when a request is made by the Bridge-Node. Due to the fact that the Bridge only sends a request when the user interacts with the Web server, all Slave traffic is triggered by the Webserver. Note that this excludes the discovery process (see section 4.2.2).

4.2 Device Discovery

This section discusses important aspects or problems of the device discovery process. This refers to both the ESP-NOW discovery process and the network discovery process.

4.2.1 Network Discovery

For discovering the Raspberry Pi on the ESP and vice versa, IP's need to be exchanged. This was tried in two ways:

By Hostname

The first approach was to set a hostname for the ESP32 and fetch data through it. While this worked in most networks, it isn't possible or needs to be configured in some, which would make the average consumer unable to make use of the system. Therefore, this approach was discarded.

Via UDP Broadcast

The second approach was to broadcast the IP of the ESP32 via UDP-Broadcast. This would send a message into the network that the Raspberry Pi would catch and save, this could then be used to send a direct HTTP-Post to the bridge to share the servers IP. While sending a broadcast is a lot of network traffic, it is the most reliable way to discover the Raspberry Pi in any network. To minimize unintended traffic, the broadcast are only sent for a set period of time after boot.

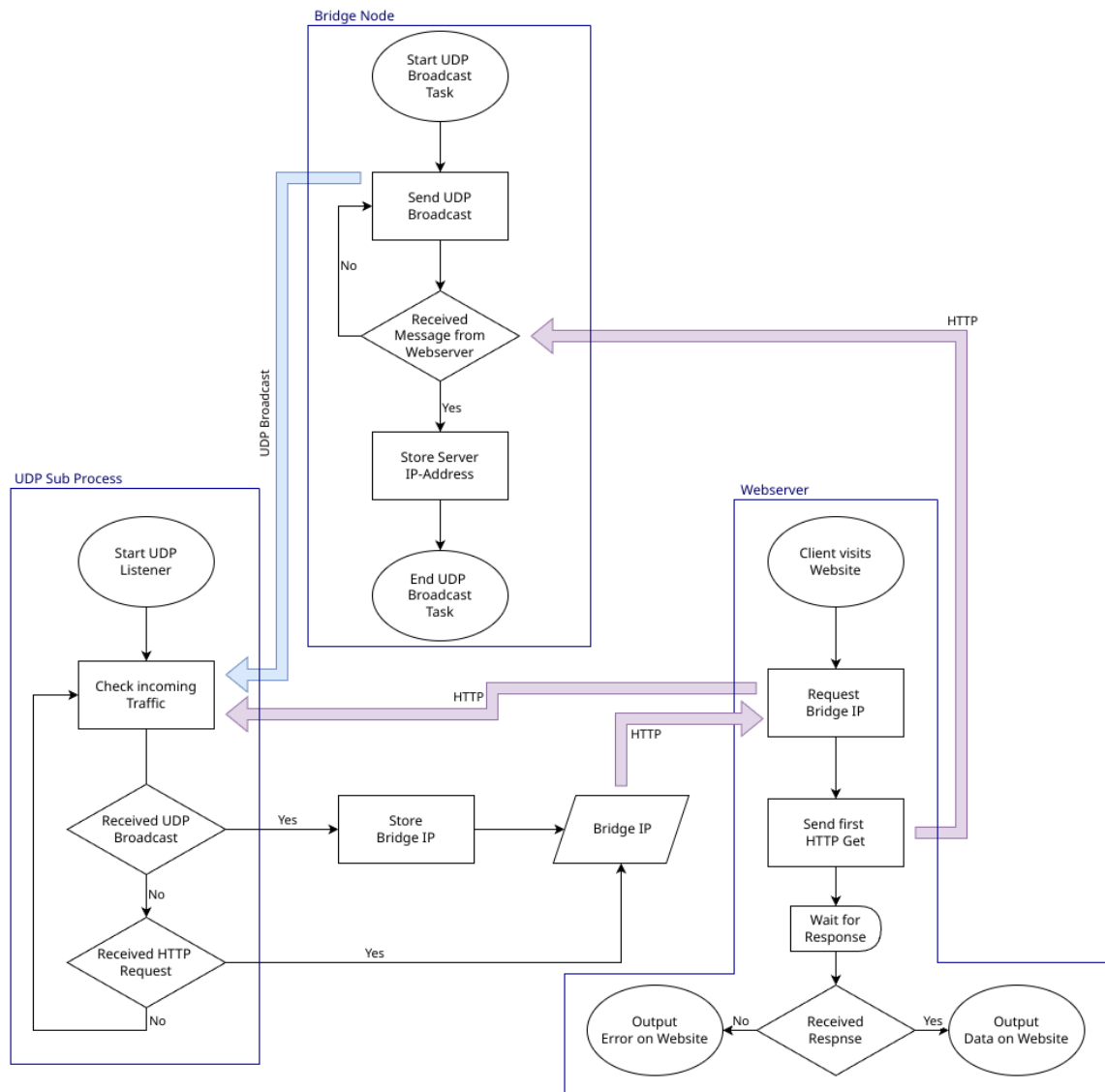


Figure 4.2: UDP Discovery Process

4.2.2 ESP-NOW Discovery

The ESP-NOW discovery process is a crucial part of the system. It allows the bridge node to discover new nodes and allows the slave nodes to find the bridge node. The process is as seen in the following flowchart:

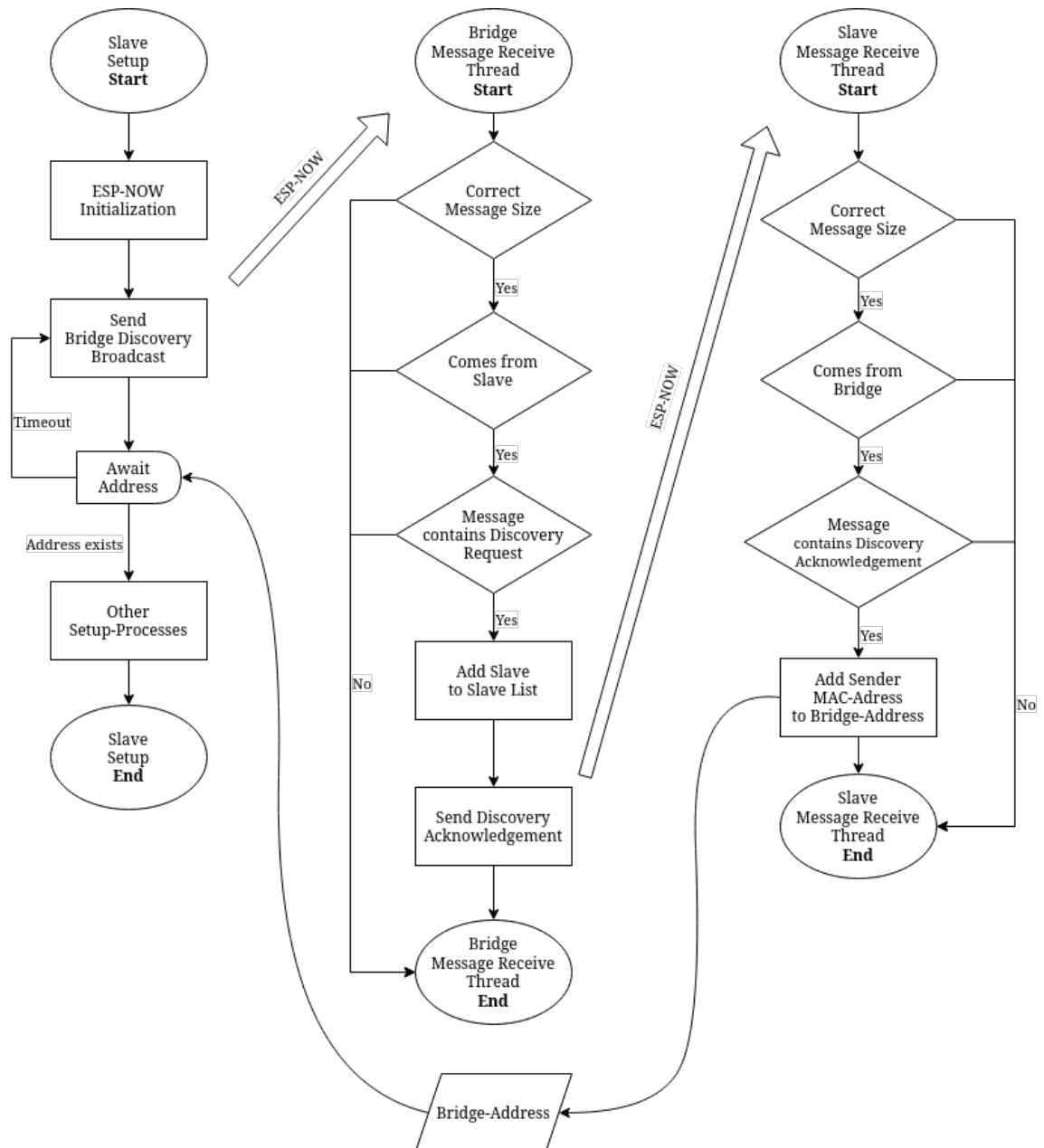


Figure 4.3: ESP-NOW Discovery Process

5 Nodes

The following chapter describes the general implementation of the nodes in the Smart Home System. This includes the project configuration and an overview of the FaR-iLib library.

5.1 Project Configuration

5.1.1 platformio.ini

Every project in PlatformIO requires a `platformio.ini` file. This file contains all the necessary information for the build process. Its content may vary depending on the project, but for this Smart Home System, the following configuration stays consistent:

```
[env:<BOARDNAME>]
  platform = espressif32
  board = <BOARDNAME>
  framework = arduino
```

The `<BOARDNAME>` should be replaced with the name of the board used for the Node, for example, `esp32-s3-devkitm-1` for the ESP32-S3 or `esp32c3-devkitm-1` for the ESP32-C3.

platform

The `platform` key specifies the platform to build for. Due to the fact that the networking is dependent on ESP-NOW, this will most likely be `espressif32` or `espressif8266` depending on the microcontroller.

framework

The `framework` key specifies the framework to use. All, for this thesis developed nodes, are based on the Arduino framework, however, other frameworks for the ESP32 are available, such as ESP-IoT-Development-Framework (ESP-IDF for short) maintained by Espressif and designed exclusively for the ESP32.

Other **more specific configurations** that are necessary will be discussed in the following sections.

5.1.2 Enabling the Serial Monitor

While reading the serial monitor is not necessary for functionality, it is a useful tool for debugging. For older ESP32 boards no additional configuration should be necessary, newer boards however, such as the ESP32-S3 and the ESP32-C3, require the following `build_flags`:

```
build_flags =  
    -DARDUINO_USB_MODE=1  
    -DARDUINO_USB_CDC_ON_BOOT=1
```

After that, the monitor speed can be set as well:

```
monitor_speed = 9600
```

The baud rate can be adjusted to any desired value as long as the clock rate of the board supports it, but note that it should match the baud rate set in the code. Common values are 115200 or 9600 on older boards.

5.1.3 Including FaRiLib

Due to the fact that the FaRiLib library should be used across all nodes, it is placed inside the `SW-ESP` directory and included in all node-projects. To add the directory to the include path, the following lines should be added to the `platformio.ini` file:

```
lib_extra_dirs =  
    ../globalLibs
```

Note that the path is relative, meaning it needs to be adjusted should the project be placed outside the `SW-ESP` directory.

5.1.4 Complete Example

A complete example of a `platformio.ini` file for a slave project using the ESP32-C3 would look like this:

```
[env:esp32-c3-devkitm-1]  
platform = espressif32  
board = esp32-c3-devkitm-1  
framework = arduino  
  
build_flags =  
    -DARDUINO_USB_MODE=1  
    -DARDUINO_USB_CDC_ON_BOOT=1
```



```
lib_deps =
  fastled/FastLED@^3.9.5
  milesburton/DallasTemperature@^3.11.0

lib_extra_dirs =
  ../globalLibs
```

5.2 FaRiLib

To share common functionality across all nodes, a library called FaRiLib was created. Its goal is to provide a common interface for all nodes to use, such as a common way to handle incoming messages and to send messages to the bridge node.

It's placed in the SW-ESP/globalLibs directory and should be included in every node project placed in the SW-ESP directory.

5.2.1 Library Structure

```
FaRiLib
├── CMakeList.txt
├── include
└── src
```

- **CMakeLists.txt**: The CMake configuration file for the library.
- **src/**: The source files of the library.
- **include/**: The header files of the library.

CMakeLists.txt

For the inclusion to work properly, a **CMakeLists.txt** file is necessary. It should contain the following lines:

```
project(FaRiLib)

include_directories(include)

set(SOURCE_FILES
    src/*
)
```

In combination with the configuration done in section 5.1.3, this enables an include statement as simple as `#include <FaRiLib.h>` in all node projects.

5.2.2 ESP NOW

The library provides a wrapper around the necessary ESP-NOW functionality. Functionally it is split into two header files, `espnowMaster.h` and `espnowSlave.h`. The master node is the bridge node, while the slave nodes are the nodes that send data to the bridge node.

ESP-NOW Objects

The aforementioned header files contain the `EspNowMaster` and the `EspNowSlave` classes. They both inherit from the abstract `EspNow` class, which contains common functionality of both.

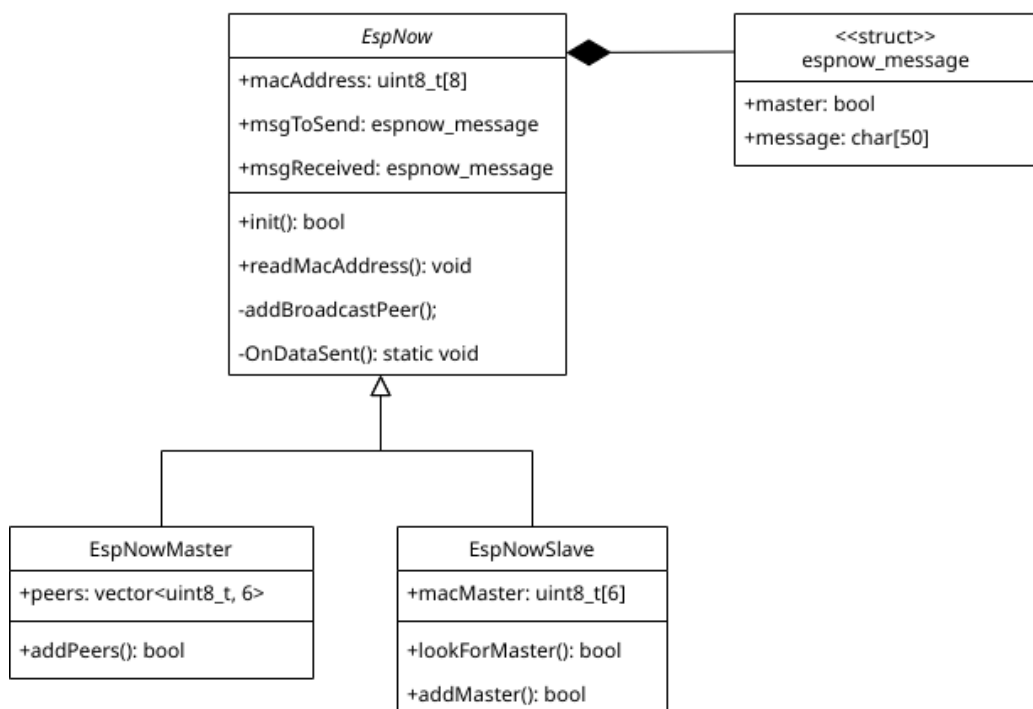


Figure 5.1: ESP-NOW Object Inheritance

5.2.3 HTTP

There is also a template for the HTTP communication and UDP-Discovery Process. To initialize the full HTTP functionality, the following functions need to be called inside the `Setup()` function of the bridge-node:

```
#include "udpBroadcast.h"
#include "WebServer.h"

void setup() {

    ethernetInit();
    initWebServer(&server);
    createTasks();
    //...
}
```

This initializes the Ethernet connection, the Webserver, and starts a task that periodically sends UDP broadcasts until the task is suspended.

5.3 Project Structure

This section describes the project structure of the nodes and how functionality is meant to be extended. Every Project should make use of the FaRiLib library (see section 5.2) and follow a similar structure.

5.3.1 Slave Node

Ideally there should only be one slave binary that is able to run on all slave Nodes. This means, unless there is a good reason, new functionality should be implemented into the preexisting slave project.

Adding new Functionality

To add new functionality to the slave node, the following steps should be taken:

1. Create a new task that handles the new functionality
2. Add the new case to the switch statement in the `Setup` function choosing what tasks to activate
3. Adjust the Web server to display the data

5.3.2 Bridge Node

While it is possible to have more devices communicating with the web server, the recommended course of action is to have one bridge node that handles all communication. This means only one binary is needed for the bridge node.

Adding new Functionality

For adding new Nodes communicating via ESP-NOW to the smart home system, there is only minimal restructuring of the source code necessary.

If the new Node is not a ESP32, a new communication protocol needs to be implemented. While this is not a trivial task, it is possible to extend the FaRiLib library without complete refactoring.

Basic functionality can be added in the following files:

- `main.cpp`: Add a task or process an incoming message
- `WebServer.cpp`: Add a new HTTP endpoint

6 Modules

This chapter describes how the hardware is constructed and changed over time. The revisions of the modules are presented backwards in this document, from the latest to the oldest design.

6.1 Basic Module

Every Node (except the bridge node) consists of the same base module. It's capable of measuring the ambient temperature and controlling four RGB-LEDs. The functionality of the basic module can be extended by attaching another module to the base module.

There were multiple revisions of the basic module. Revision 1.0 was designed to be a very rough prototype to test the functionality and concept of an attachable battery. Revision 2.x was designed to be a more polished version of the basic node, all the other modules were designed and developed alongside this version.

6.1.1 Hardware - V2.2(Final)

This section describes the latest hardware of the basic module. The basic module is a simple node that can measure the ambient temperature and control four RGB-LEDs. The hardware consists of the following components:

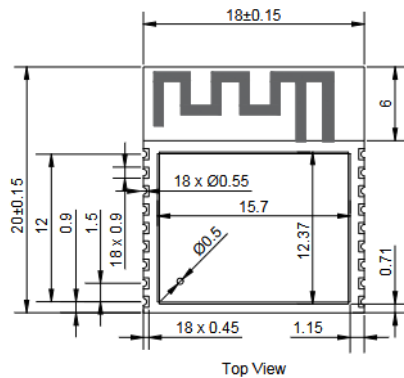
- Microcontroller
- Temperature Sensor
- LED Indicator
- USB-C Connector
- Serial Port
- DC-DC Converter
- Module Connector
- Battery Monitoring
- Module Detection
- Spare Header

Microcontroller

The basic module uses an ESP32-C3-wroom-02 [9] microcontroller. It is a low-power, highly integrated Wi-Fi and Bluetooth SoC. In the data sheet it was stated that the MCU is recommended for a wide range of applications, including smart home devices and other IoT products. Before the ESP32-C3 was chosen, it was compared with the ESP32-S3-wroom-2[11] to see which one would be the better fit for the basic module.

ESP32-C3-WROOM-02 :

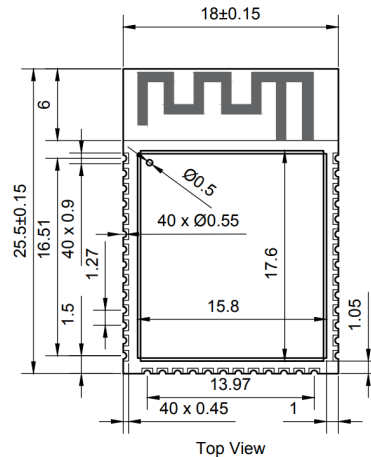
- 15 GPIOs
- Single-core 32-bit RISC-V CPU up to 160Mhz
- 4 or 8 MB flash
- PCB antenna external antenna version



(a) ESP-C3

ESP32-S3-WROOM-2 :

- 33 GPIOs
- Dual-core 32-bit RISC-V CPU up to 240Mhz
- 16 or 32 MB flash
- PCB antenna external antenna version



(b) ESP-S3

Figure 6.1: Physical comparison of the ESP32-C3 and ESP32-S3, Unit in mm.

The ESP32-C3 does not have as many GPIO pins compared to the ESP32-S3, however the abundance of GPIO pins is not necessary for the basic module. The ESP32 C3 is also cheaper and allocates less space on the PCB.

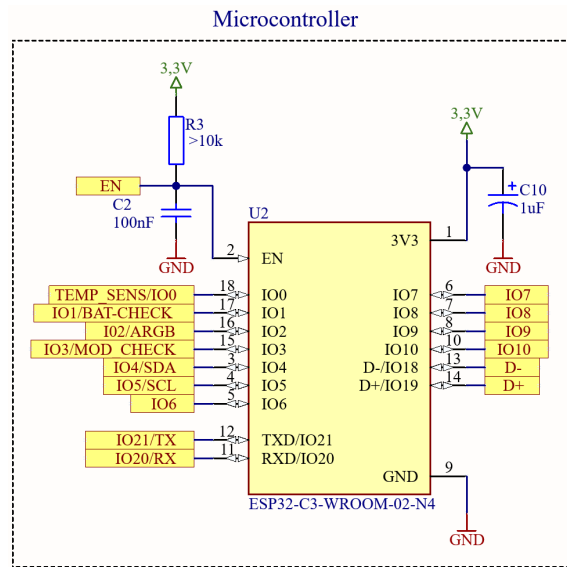


Figure 6.2: ESP32-C3 implemented in the schematic.

Temperature Sensor

The DS18B20^[7] was chosen as the temperature sensor. It is a digital thermometer with an operating temperature range of -55°C to $+125^{\circ}\text{C}$. The sensor is connected to the microcontroller via GPIO0 and communicates over a 1-Wire bus.

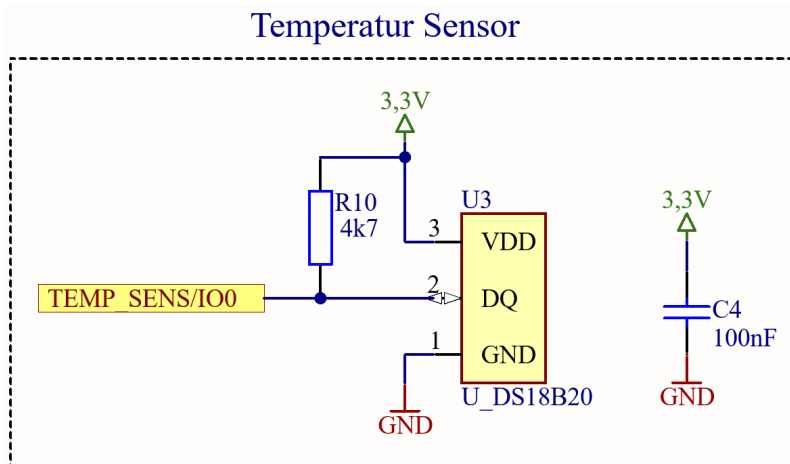


Figure 6.3: DS18B20 implemented in the schematic.

As seen on the PCB in figure 6.12 the DS18B20 is mounted near the ESP-32-C3. While testing, it was noticed that the temperature sensor was affected by the heat of the ESP32-C3. This issue will be fixed in the next version of the basic node.

LED Indicator

The basic module has four addressable RGB-LEDs. Those LEDs have an integrated controller and can be controlled individually. The WS2812b[31] needs 5V, GND and a DATA-IN. DATA-IN is connected to the microcontroller via GPIO 2. The LEDs can be cascaded by connecting the DATA-OUT of the last LED to the DATA-IN of the next LED. The LEDs are used to indicate the status of the node. For example, to indicate if it detected motion or to display the charge-level of the battery, when the corresponding module is attached.

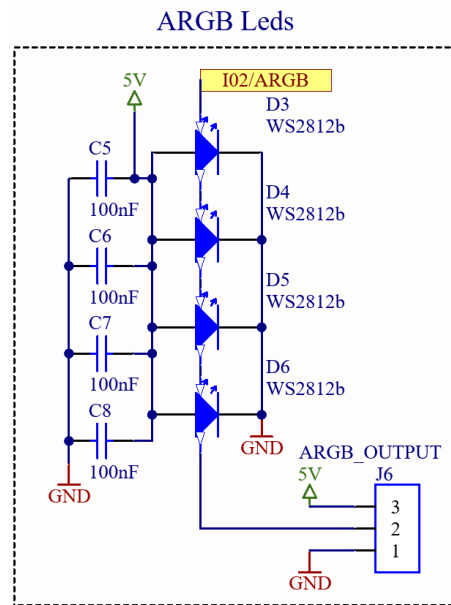


Figure 6.4: RGB-LEDs implemented in the schematic.

USB-C Connector

The USB-C connector is used to power and program the basic node. The connector can be plugged in both ways, which makes it more user-friendly. This particular connector was chosen, due to being the new standard for USB connectors, therefore being widely spread.

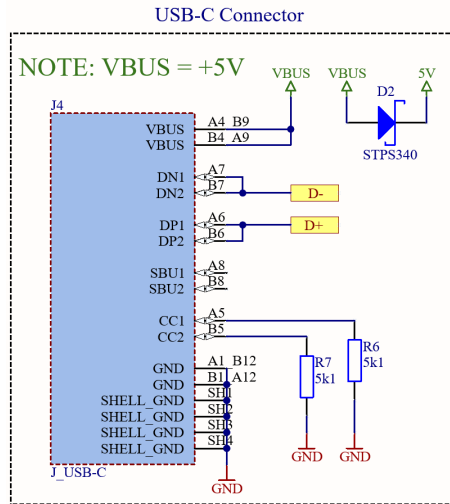


Figure 6.5: USB-C Connector implemented in the schematic.

In order to power the basic node with 5V, a 5.1k Ω resistor is connected to the CC1 and CC2 each. The resistor tells the charger that the basic node is a default USB device and can draw up to 3A of current. [23] A schottky diode is used to avoid backflow of current.

Serial Port

This interface is used to program the basic node. The interface is connected directly with the UART of the ESP32-C3.

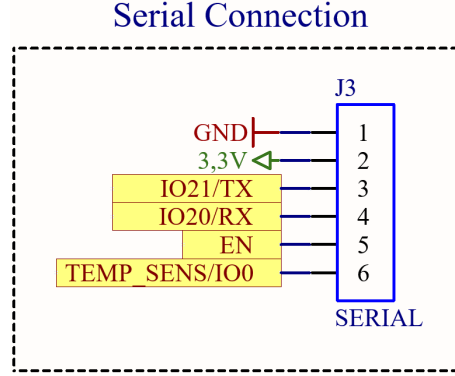


Figure 6.6: Serial-Port implemented in the schematic.

It must be noted that pin 0 is not the right one to bring the ESP32-C3 into programming mode. It was designed out of habit, since previous ESP32 always used pin 0 to enter programming mode. The correct pin is GPIO 9 like stated in the documentation[3].

DC-DC Converter

The step-down converter TLV62569 [25] from Texas Instruments is used to convert the 5V from the USB-C connector to 3.3V. The converter has a high efficiency of up to 95%, which is significant for battery operation. It powers the microcontroller, temperature sensor and a module, if attached to the module connector.

To set the output voltage, a voltage divider is used to feed the FB pin of the IC. With following formula the output voltage can be calculated:

$$V_{OUT} = V_{FB} \cdot \left(1 + \frac{R_1}{R_2}\right) \quad (6.1)$$

$$R_1 = \left(\frac{V_{OUT}}{V_{FB}} - 1\right) \cdot R_2 \quad (6.2)$$

The output voltage V_{OUT} was set to 3.3V. The feedback voltage V_{FB} of the converter is 0.6V. The resistor R_2 was set to 200kΩ.

$$R_1 = \left(\frac{3.3V}{0.6V} - 1\right) \cdot 200k\Omega = 900k\Omega \approx \mathbf{910k\Omega(E24)} \quad (6.3)$$

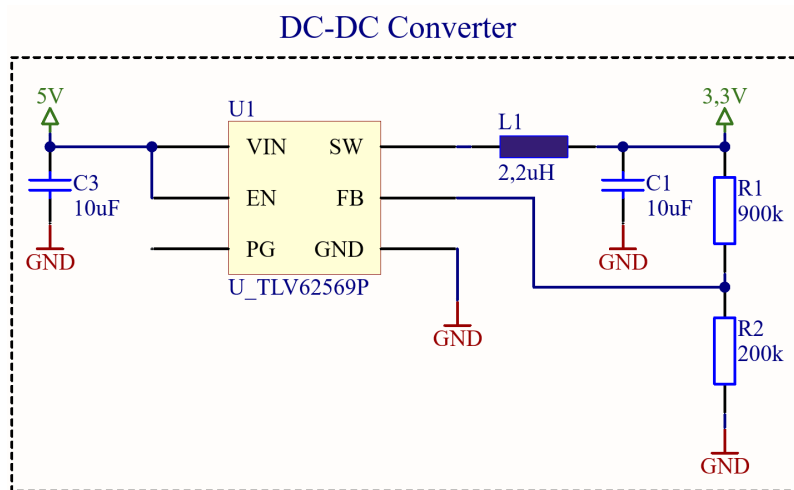


Figure 6.7: TLV62569 Converter implemented in the schematic.

The Power-Good feature is not used in this application. The EN pin is connected to 5V to enable the converter.

Module Connector

The module connector is used to join additional modules to the basic module. The connector has 15 pins, 8 for power and power management and 7 for communication.

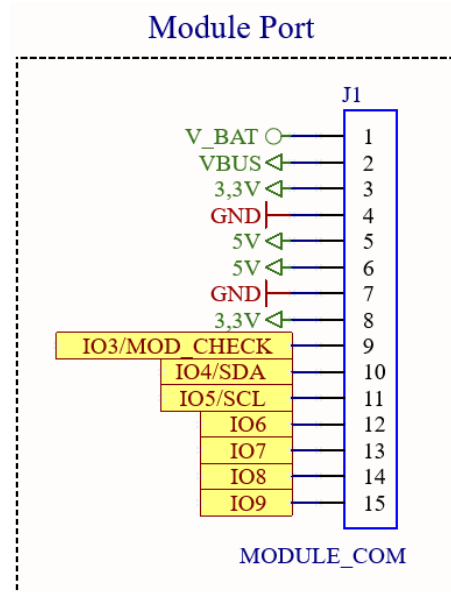


Figure 6.8: Module-Connector implemented in the schematic.

Pin 3-5 and 6-8 are symmetrical to each other. It was designed this way to allow the

battery module and motion module(or for future modules) to be attached simultaneously. The battery module is plugged into pin 1-5 and is connected to following voltages:

- 1. VBAT - used to monitor the battery voltage
- 2. VBUS - used to charge and disable the battery module when the USB-C is connected
- 3. 3V3 - not used, but may be relevant for future modules/revisions
- 4. GND
- 5. 5V - The battery module powers the basic node over this pin

Pin 6-15 are used for communication between the modules.

Spare Header

The basic node has a spare header, which can be used to attach a button or magnetic switch. The button can be used to interact with the node or trigger a specific action. The header has 2 pins GND and GPIO 10 which is pulled to 3.3V.

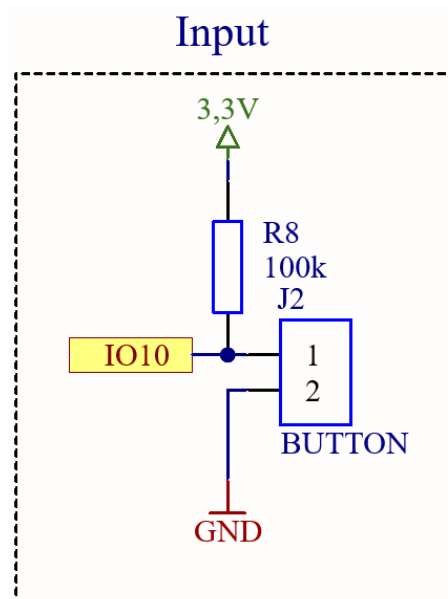


Figure 6.9: Spare header implemented in the schematic.

Battery Monitoring

The battery module has a voltage divider to monitor the battery voltage, to reduce the voltage to a safe level for the MCU to read. The voltage is measured by the ADC of the microcontroller and can be used to determine the charge level of the battery. The voltage divider is connected to pin 1 of the module connector. When the battery reaches a certain voltage, the microcontroller can go into deep-sleep mode to save power. The 100nF capacitor is used to filter out noise from the battery.

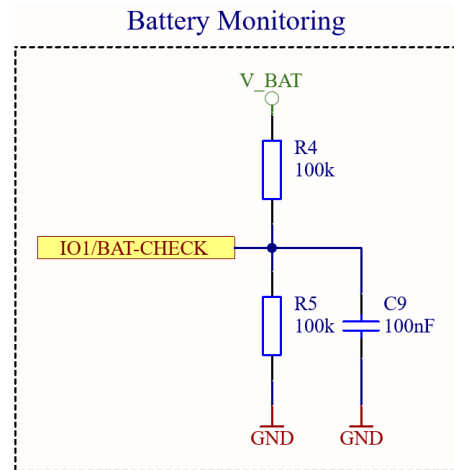


Figure 6.10: Battery Monitoring implemented in the schematic.

Module Detection

The basic node recognizes the attached module by measuring the voltage on pin 3. A 100kOhm resistor is installed on the base module pulling pin 3 to GND. When a module is attached, a voltage divider is created. The input voltage is determined by the second resistor on the module.

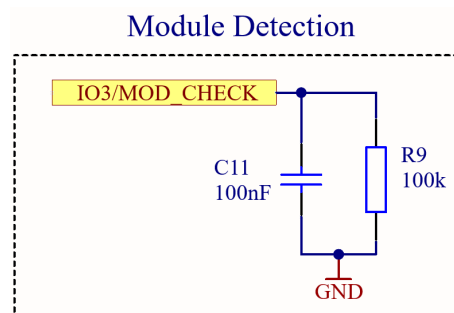


Figure 6.11: Module-Detection implemented on the base module.

PCB - V2.2

The PCB has a size of 75x30mm. The components used were mainly SMD components, to keep the PCB small. The PCB is a 2-layer board with a ground plane on both sides.

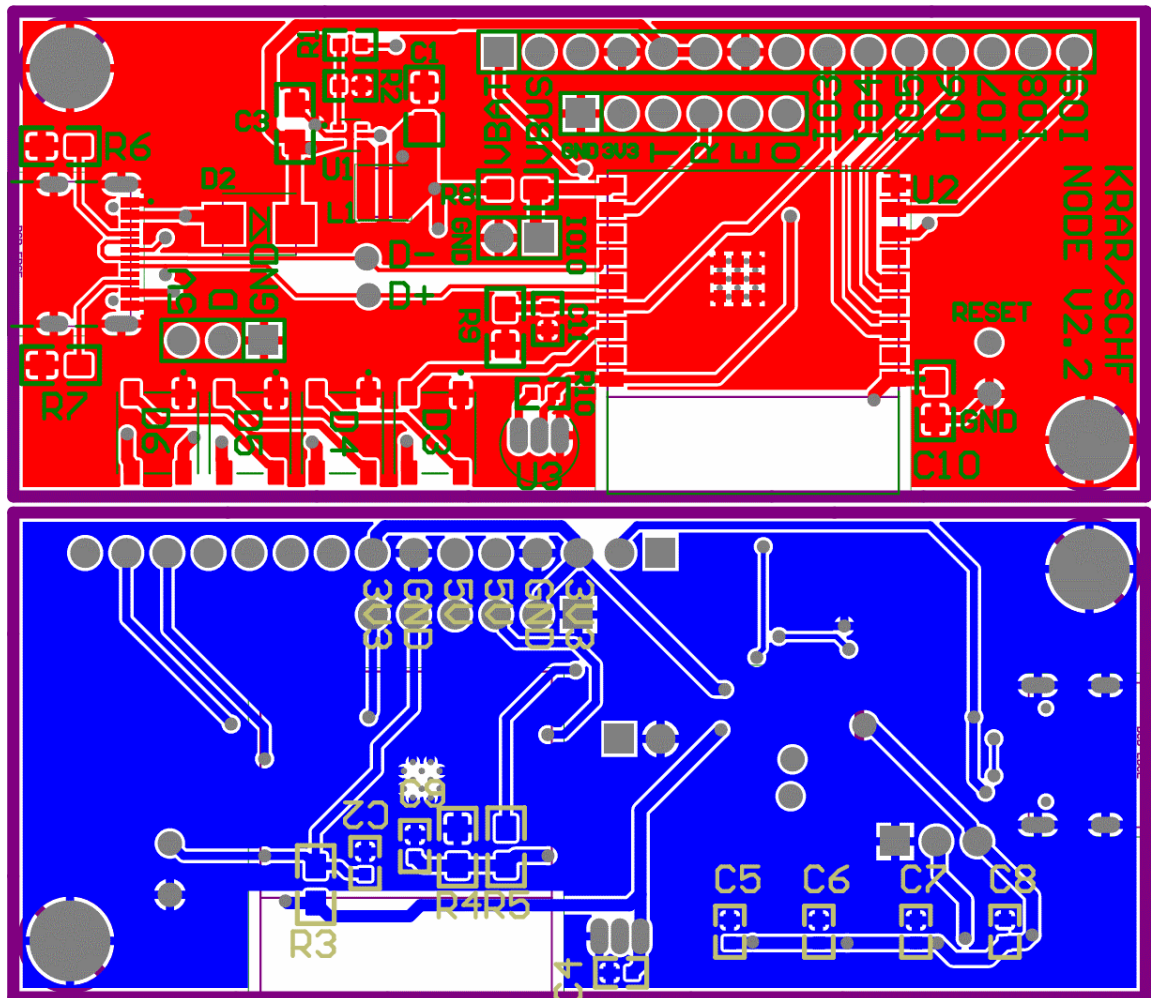


Figure 6.12: PCB of the basic node. [Red - Frontside, Blue - Backside]

6.1.2 Hardware - V2.1

This version of the basic module had minor changes to the hardware compared to the version after. It was produced and tested, resulting in V2.2.

Temperature Sensor

This version of the base module had a missing pull-up resistor on the temperature sensor. The DS18B20 was not able to work without it. It needs a pull-up resistor to work properly as stated in the datasheet[7]. This was fixed in V2.2.

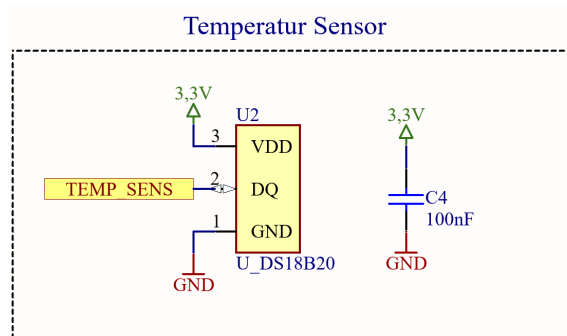


Figure 6.13: MCU implemented in the schematic, note the missing pull-up resistor.

Serial Port

It lacked an laid out Serial Interface, which was added in V2.2 as seen in Figure 6.6 above. During the software development of the basic node, it happened that the programming over the native USB of the ESP32 C3 was not possible. Duo to setting build flags, that would disable USB programming. To make programming easier and fix this issue, the serial interface was laid out on the board.

Module-Detection

While testing this version, it was noticed that pin 3 was floating when no module is connected, potentially causing the base module to recognize a module without one being attached. To fix this issue in V2.2, the voltage divider on the modules were split up and a 100kOhm resistor was added to pull the pin to GND on the base module.

PCB - V2.1

The size of the PCB is identical to V2.2 with a footprint of 75x30mm.

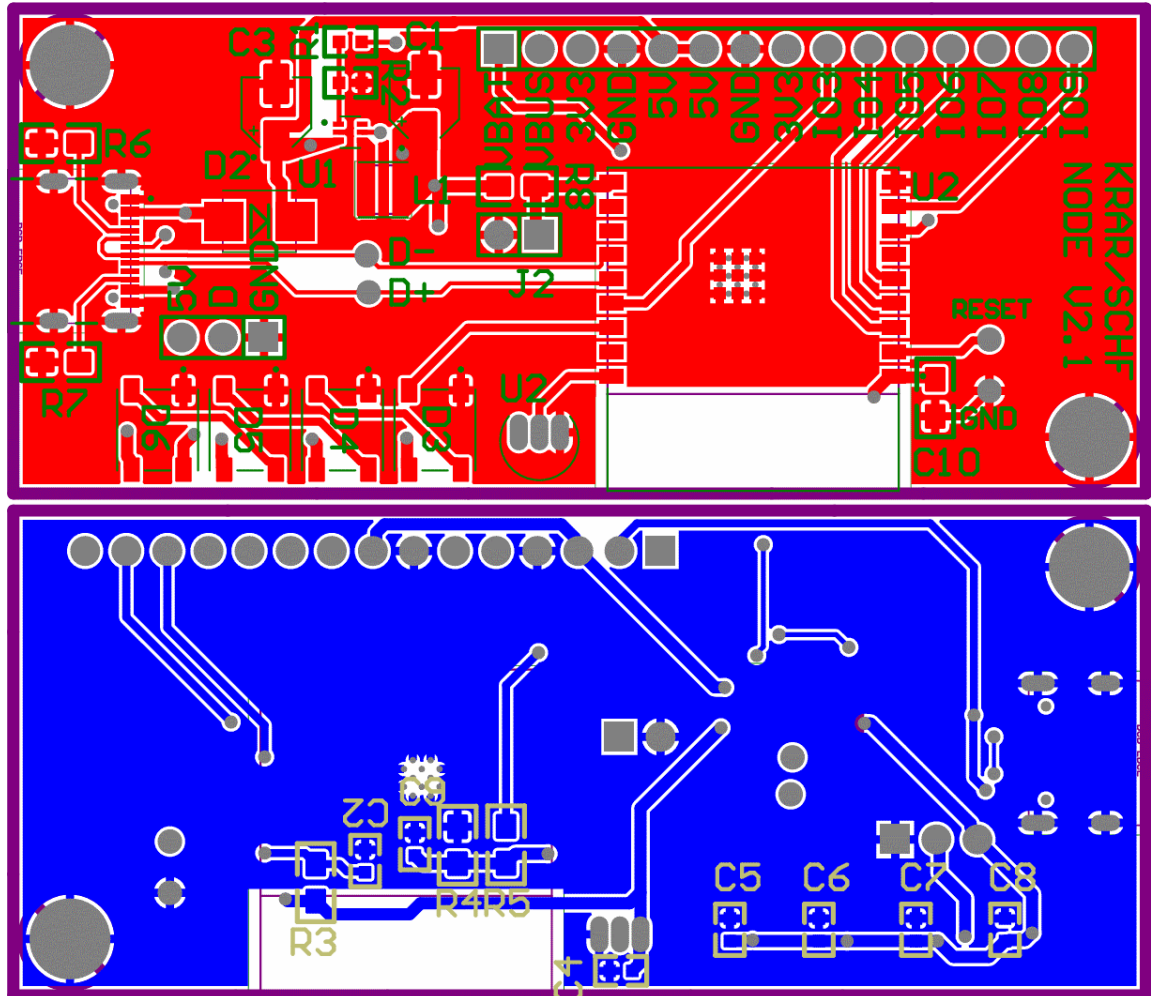


Figure 6.14: PCB of the basic node V2.1. [Red - Frontside, Blue - Backside]

6.1.3 Hardware - V2.0

This version of the basic module had minor changes compared to V2.1. The PCB was produced, however it was never populated or tested, as the shipment took so long, that version 2.1 was finished within that time and ordered as well. Following flaws were discovered in V2.0:

Ground Plane

The ground plane was removed underneath the ESP32-C3 to improve the signal quality.

Spare Header

GPIO 10 of the MCU was not used, so it was decided to add a pin header to later versions of the basic module. Where a button or magnetic switch could be attached to. When pressed or triggered, the button could be used to interact with the node or trigger a specific action.

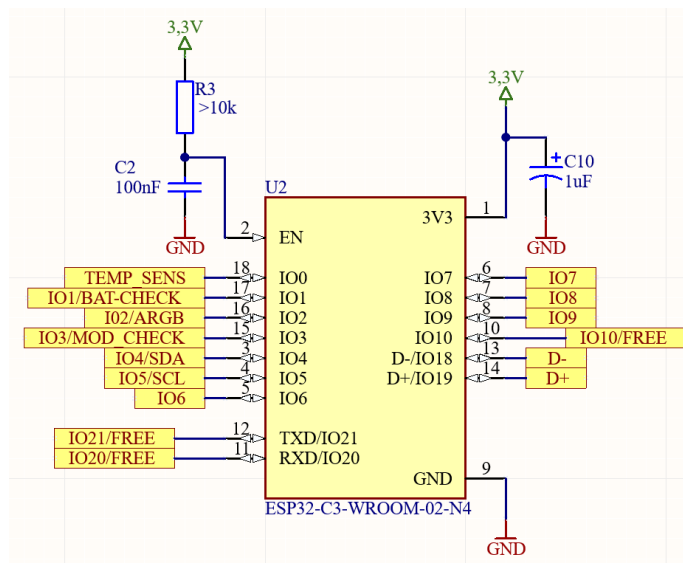


Figure 6.15: MCU implemented in the schematic version 2.0 with pin 10 not used.

PCB - V2.0

The size of the PCB is identical to V2.1 with a footprint of 75x30mm. In figure 6.16 the ground plane under the ESP32-C3 pcb-antenna is clearly visible.

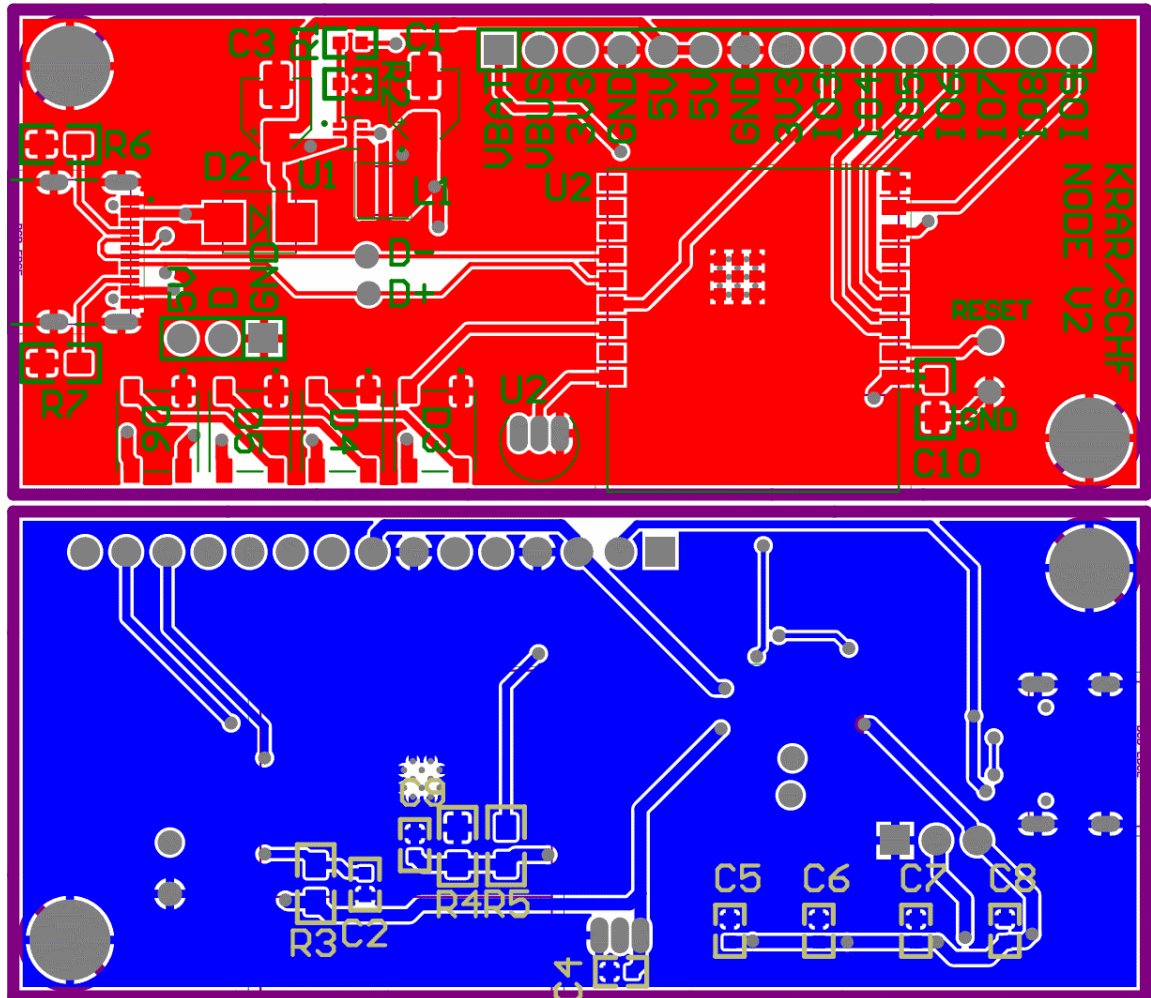


Figure 6.16: PCB of the basic node V2.0. [Red - Frontside, Blue - Backside]

6.1.4 Hardware - V1.1

Version 1.x of the basic module was intended to be a ruff prototype to test the concept of a plugable battery powered module. For this reason only the battery module was designed for this version. Version 1.0 was never manufactured, but the design was used as a base for the version 1.1. The version 1.1 was manufactured and tested.

Module Connector

The module connector has four connectors, one for the battery and three for the "modules".

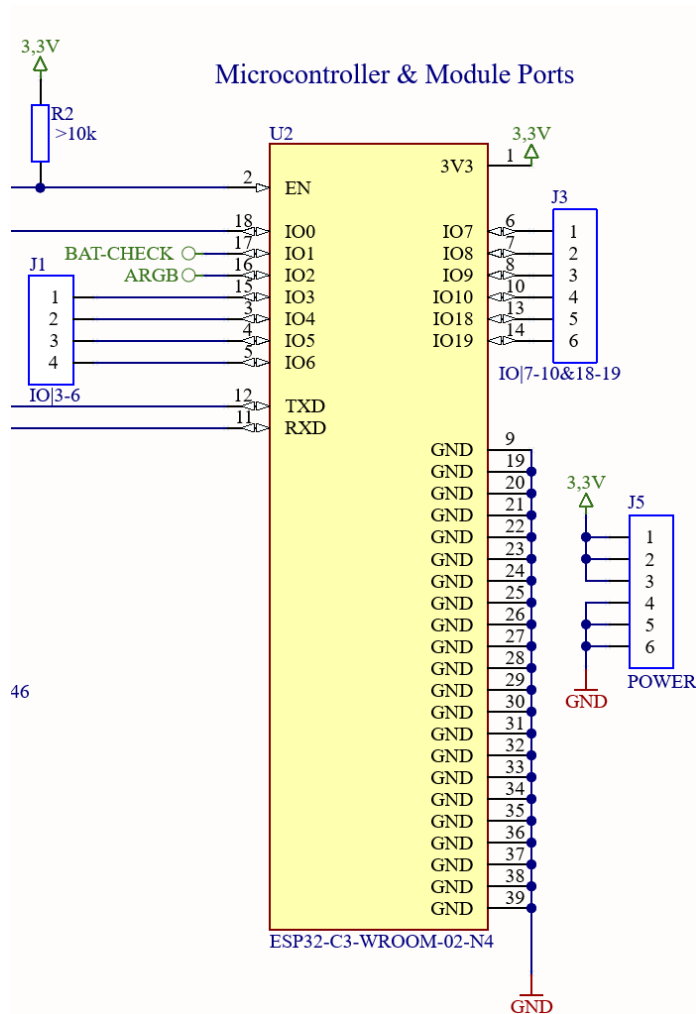


Figure 6.17: Microcontroller and module connector implemented in the schematic.

There are a 4 and 6 pin header with IO ports and a 6 pin header for supplying power. It was not intended to design a module, besides the battery module, for this version.

It was planned to be used as a development board by connecting the sensors over jumper wires.

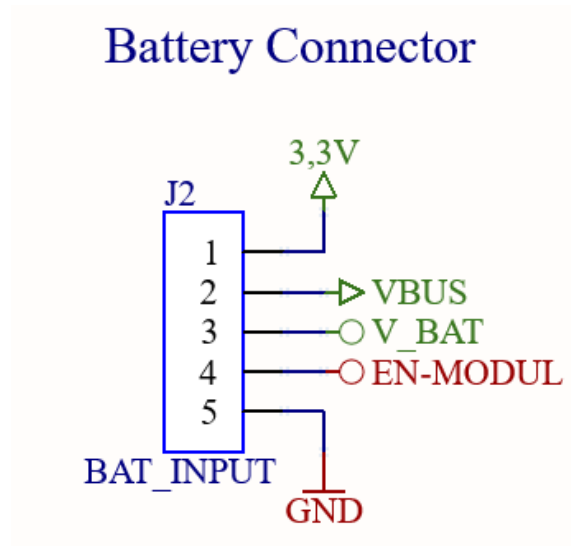


Figure 6.18: Battery connector implemented in the schematic.

When the Battery module is connected, VBUS and EN-MODUL are shorted together by the battery module. This is used to disable the linear voltage regulator on the basic module with the MOSFET Q1. The battery module would then supply the basic module with 3.3V.

In version 2.0 a "real" module connector was thought out and implemented see figure 6.8.

DC-DC Converter

To convert the 5V from the USB port to 3.3V a LD33V linear regulator was used. This regulator would then be bypassed by the battery module when connected. Then the 3.3V would be supplied by the battery module. The 5V was also used to charge the battery.

The LD33V is turned off by a P-channel MOSFET when the battery module was connected.

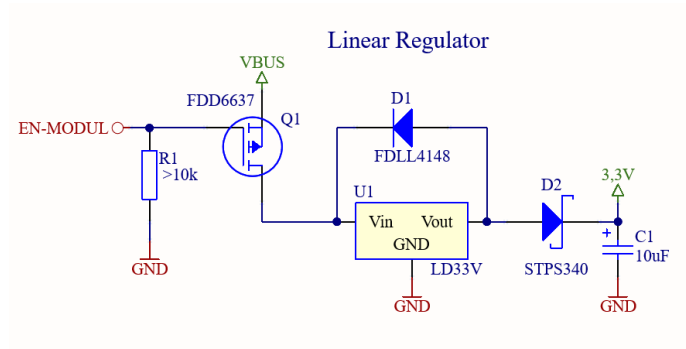


Figure 6.19: DC-DC Converter implemented in the schematic.

Since there is no 5V on the board when the battery module is connected and this voltage may be needed for future modules, it was decided to always supply the base module with 5V and to replace the LD33V with a step-down converter in later version.

USB-to-UART Converter

The USB-to-UART converter was deemed to be unnecessary since the IC was hard to solder on and the MCU could also be programmed over native USB, it was removed from later versions.

It must be noted that IO 0 is not the right pin to put the ESP32-C3 into programming mode. The correct pin is IO 9 as stated in the documentaion of Espressif.[\[3\]](#) This flaw was not discovered until V2.2 since it was not possible to solder the CP2102 on the board without destroying it.

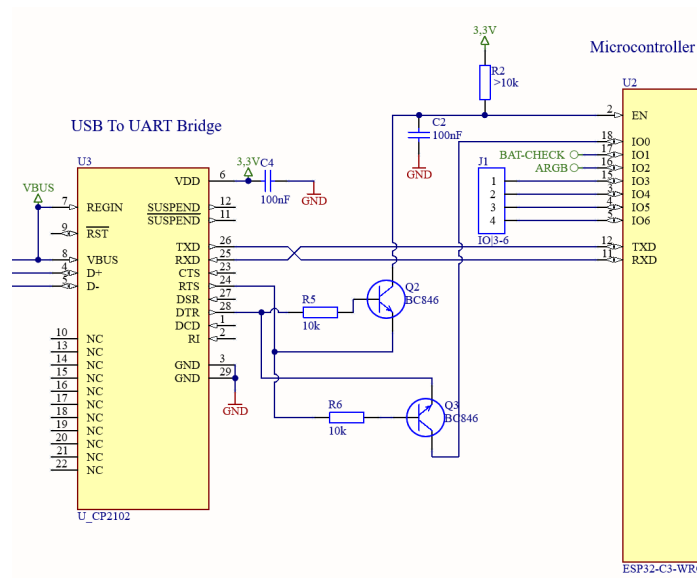


Figure 6.20: Serial-to-USB Converter implemented in the schematic.

PCB

The PCB of the basic module is a 2-layer board with a size of 75x30mm. The PCB is designed to be mounted over the battery module.

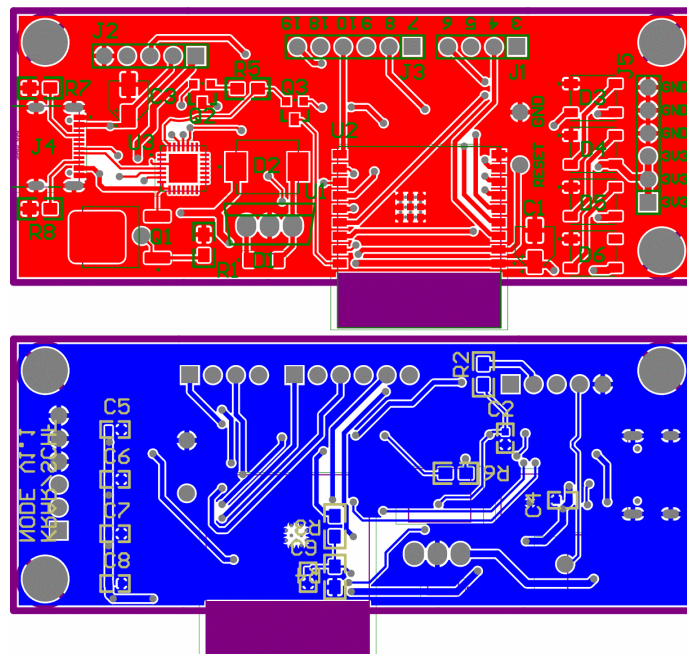


Figure 6.21: PCB of the basic module V1.1. [Red - Frontside, Blue - Backside]

The most noticeable change to the board is the ESP32-C3's antenna, which extends beyond the board's edge. In later versions the antenna stayed within the PCB to keep the size small and to make the designing of the case easier.

6.1.5 Hardware - V1.0

This version had a finished schematic, but the PCB was never finished and further development into version 1.1. The schematic is very similar to the version 1.1. Following components were different in this version:

- 1. Micro USB connector was used.
- 2. No power header.
- 3. Different component values used for battery monitoring.

Micro USB Connector

The first version of the basic module was designed to be powered over a micro USB connector.

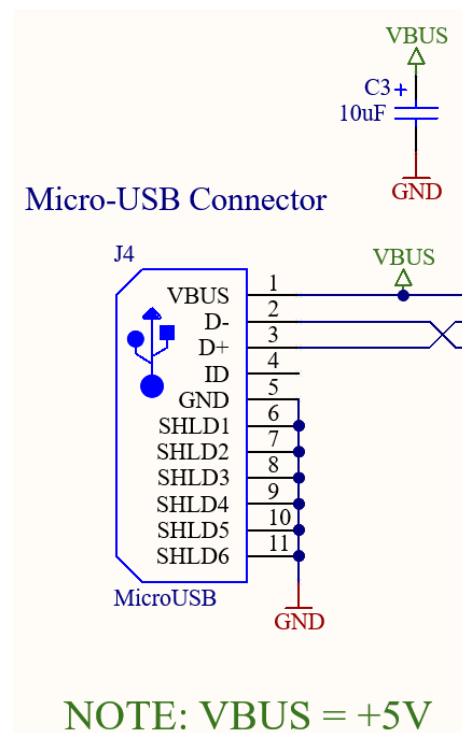


Figure 6.22: USB-C Connector implemented in the schematic.

Power Headers

The power header J5 as seen in figure 6.17 is not implemented in the first version of the basic module.

Battery Monitoring

The battery monitoring was done with a voltage divider with two 22kOhm resistors.

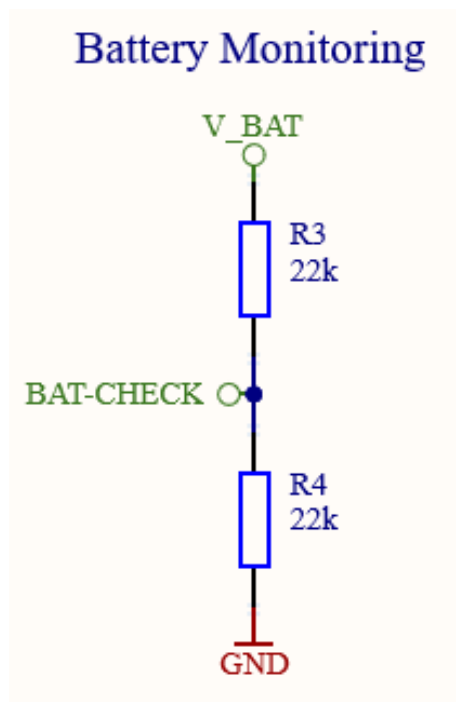


Figure 6.23: Battery implemented in the schematic.

In version 1.1V the resistor R3 and R4 were increased to 100kOhm to reduce the current consumption of the voltage divider and a 100nF capacitor was added to filter the voltage.

6.1.6 Base Software

The base software is the foundation of every node.

All slave nodes are customizable and modules can be swapped without flashing the firmware. To implement this, module specific source code is executed in its own thread. In the `Setup()` function, the module is determined, and the corresponding thread is started.

Determining the Module

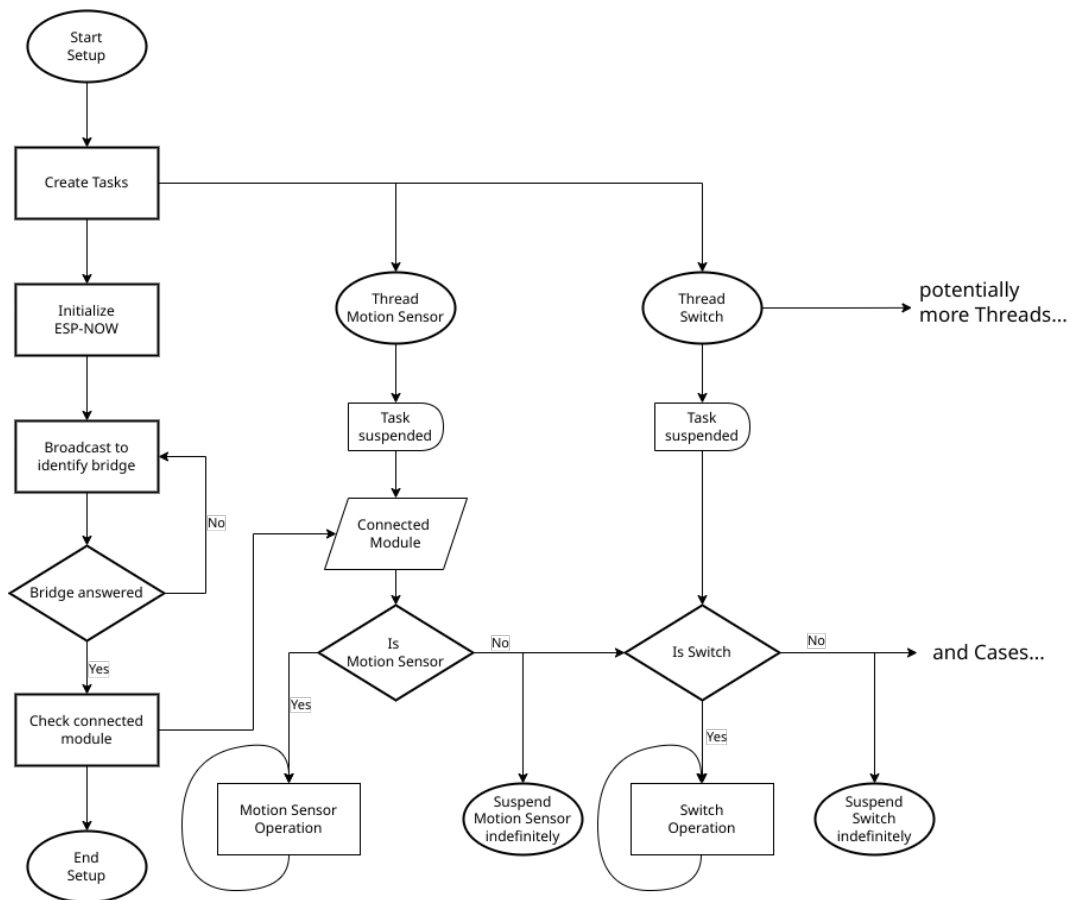


Figure 6.24: Node Module Selection

6.2 Bridge Node

The bridge node is the central node in the network. It is responsible for handling all incoming messages from the nodes and distributing them to the correct destination.

6.2.1 Hardware

The bridge node is an ESP32-S3 development board with a USB-C connector and an W5500 from Waveshare. This board uses an external antenna, which is connected to the ESP32-S3.

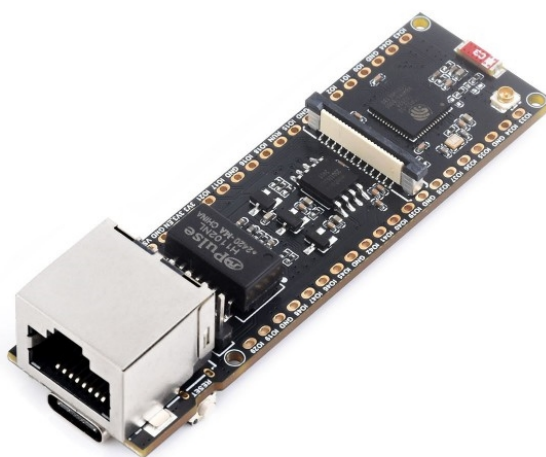


Figure 6.25: Picture of the Waveshare ESP32-S3 ETH Development Board[10].

The bridge node is powered over the USB-C connector. The W5500 is connected to the ESP32-S3 over SPI.

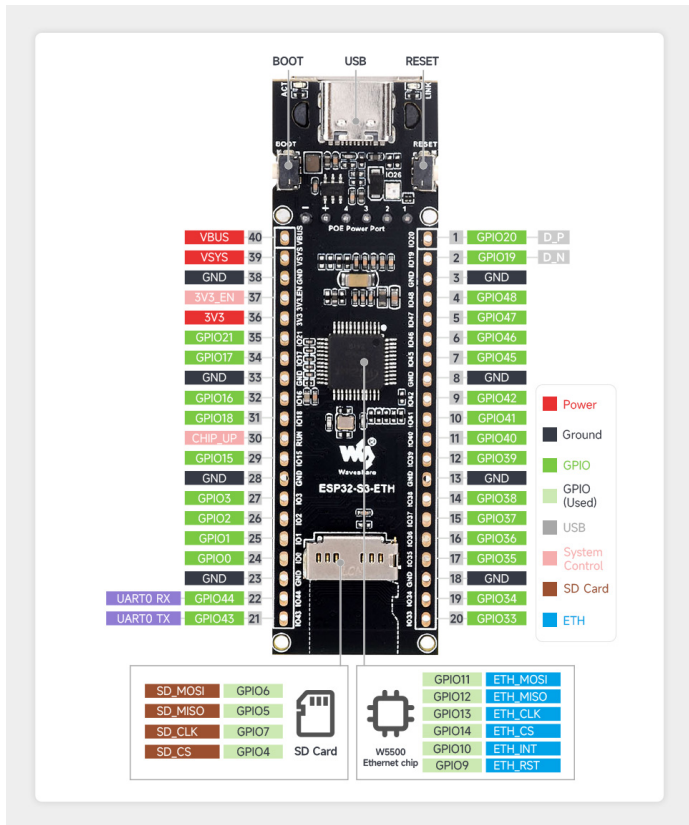


Figure 6.26: Pinning of the Waveshare ESP32-S3 ETH Development Board[10].

6.3 Motion Detector Module

This module allows the node to detect motion in its environment. This module can be attached simultaneously with the battery module to the base module. The motion module has two revisions, V1.0 and V1.1. The only difference is that a resistor was removed from the V1.1 version. The schematic and layout did not change. The resistor was removed from this print and one was added to the Base Module in order to prevent the GPIO pin from floating.

6.3.1 Hardware - V1.1(Final)

Following components are required to fulfill the requirements:

- 1. Sensor
- 2. Module Connector

Sensor

The PIR sensor EKMB1305111K[20] is used to detect motion. This particular sensor was chosen for its ability to oversee a wide angel of about 150° with a range of 5m, therefor it is a good choice to observe corridors or rooms in a house.

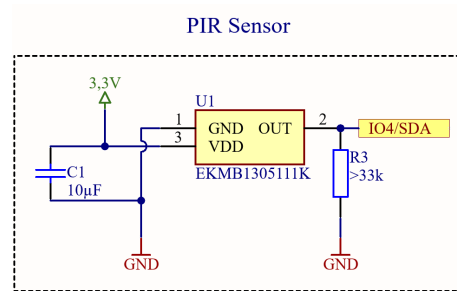


Figure 6.27: PIR Sensor implemented in the schematic.

The sensor operates with a voltage of 3.3V. The sensor has a digital output, which is connected to the GPIO pin 4 of the ESP32-C3. The pull down resistor R3 is required since the sensor output is a drain.

Module Connector

The Motion Module connects with an 10-pin connector to the Base Module. It is supplied with 3.3V and GND. And pin 4 is used to read the sensor.

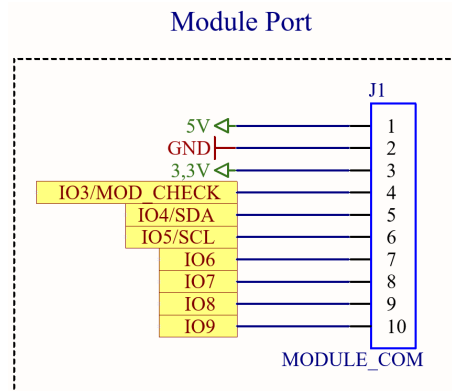


Figure 6.28: Module-Connector implemented in the schematic.

Module Detection

A 22kOhm resistor is connected to pin 3 to set the voltage divider.

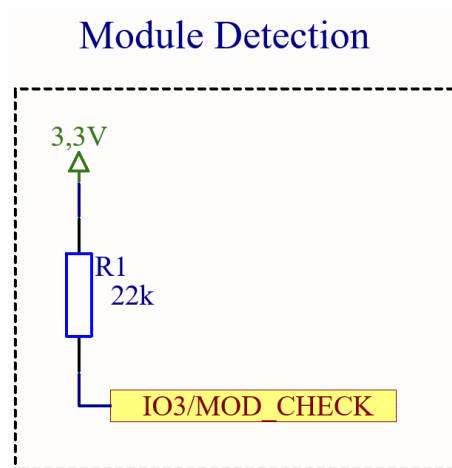


Figure 6.29: Module detection implemented in the schematic.

PCB - V1.1

The PCB of the motion module is a 2-layer board with a size of 47,1x30mm. The PCB is designed to be mounted on the base module. The PCB layout of the motion module is shown in figure 6.30.

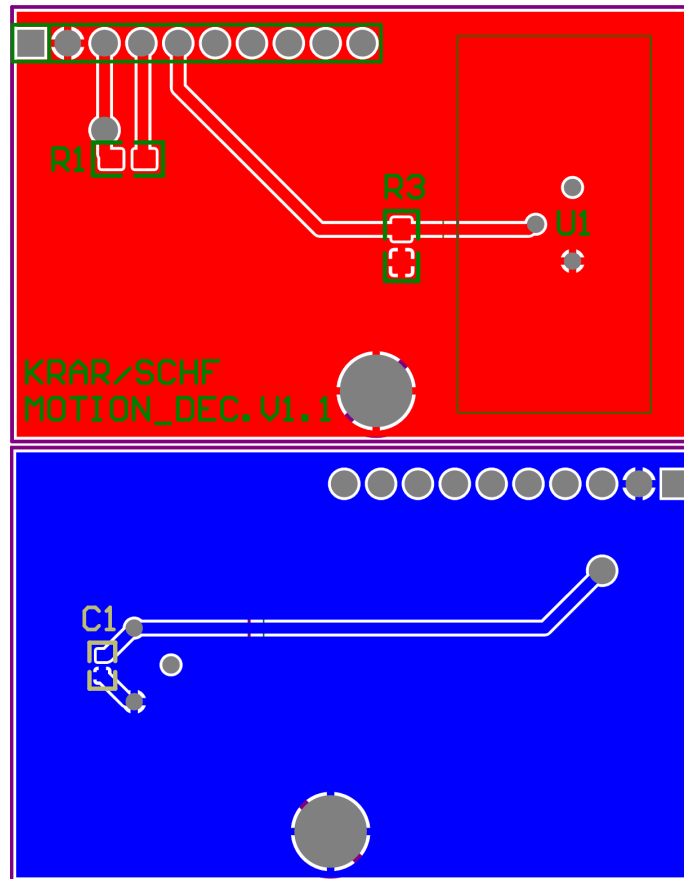


Figure 6.30: PCB of the motion module V1.1. [Red - Frontside, Blue - Backside]

6.3.2 Hardware - V1.0

Version 1.0 of the motion module is identical to version 1.1, except for the resistor R2.

Module Detection

This version had a complete voltage divider with a $330\text{k}\Omega$ and $680\text{k}\Omega$ setting the voltage between the resistors to about 2.2V .

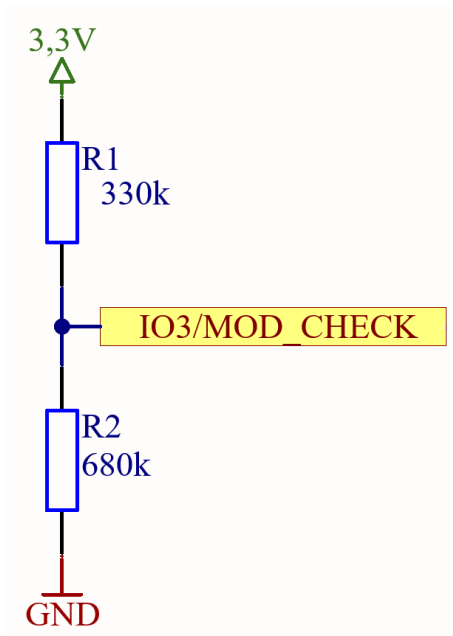


Figure 6.31: Module detection implemented in the schematic.

In V1.1 the resistor R2 was removed from the motion module and one was added to the base module. The reason for this change is to prevent the GPIO pin on the base module from floating. The PCB layout of the motion module is shown in the following page.

PCB - V1.0

The PCB of this version is identical with V1.1. The only difference is, that there is a footprint for the resistor R2 on the top side of the PCB.

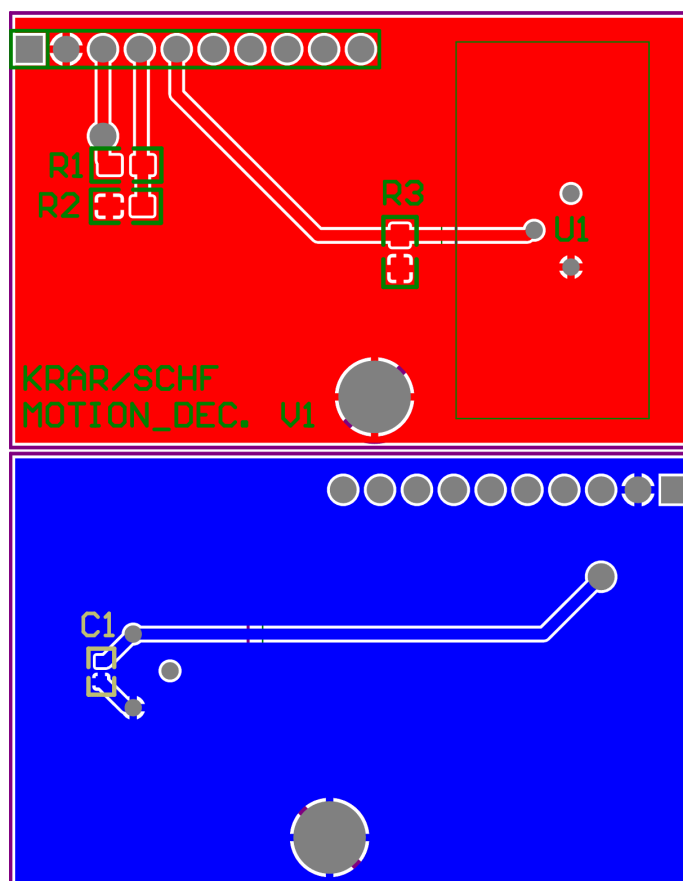


Figure 6.32: PCB of the motion module V1.0. [Red - Frontside, Blue - Backside]

6.4 Battery Module

The battery module is used to power the basic node, without the need to have a power outlet nearby. This module can be attached to the base module alongside the motion detector module.

Following components are required to fulfill the requirements:

- 1. Battery
- 2. Power-Management-System
- 3. DC-DC Converter
- 4. ON/OFF switch

6.4.1 Hardware - V2.2(Final)

This section describes the hardware of the final version of the battery module.

Module-Connector

The basic module is plugged into the battery module with the pins 1-5. The battery module is connected to the following voltages:

- 1. VBAT - used to detect battery and monitor the battery voltage
- 2. VBUS - used to charge and disable the battery module when the USB-C is connected
- 3. 3V3 - not used, but may be relevant for future modules/revisions
- 4. GND
- 5. 5V - The battery module powers the basic node over this pin

With the voltages the basic node is powered, and the battery is charged.

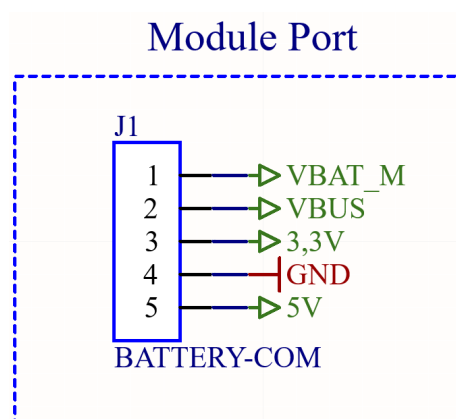


Figure 6.33: Module-Connector implemented in the schematic.

Battery

The battery module uses a 3.7V Li-Ion battery to power the basic node. The battery can be any Li-Ion battery. The battery is secured with a 1A fuse.

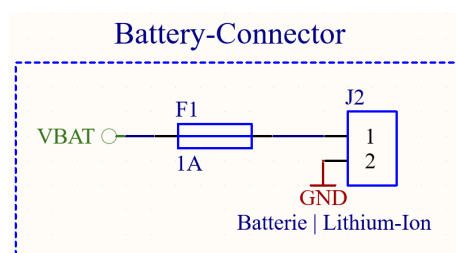


Figure 6.34: Battery implemented in the schematic.

Battery Management System

The BMS handles the charging of the battery and assures that the battery is not being overcharged. The used IC is the MCP73811[16] from Microchip, it is a linear charge management controller for both Li-Ion and Li-Polymer batteries. To prevent over-discharging, the microcontroller measures the battery voltage and can go into deep sleep mode when the battery voltage is too low.

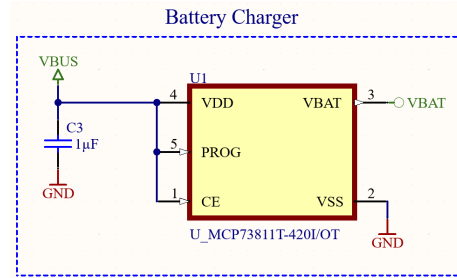


Figure 6.35: Battery-Management-System implemented in the schematic.

Boost Converter

The boost converter is used to convert the 3.7V from the battery to 5V to power the basic node. The used IC is the TLV61070 from Texas Instruments, it is a high efficiency boost converter with a 95% efficiency.

The inductor and the output voltage were calculated like stated in the datasheet[24]. The inductor was calculated with the formula:

$$I_L(DC) = \frac{V_{OUT} \cdot I_{OUT}}{V_{IN} \cdot \eta} \quad (6.4)$$

Where I_{OUT} is the output current, V_{IN} is the input voltage (worst case) and η is the efficiency (typical 90%) of the boost converter. The output current was set to 1A, the input voltage to 3V and the output voltage to 5V.

$$I_{L(DC)} = \frac{5V \cdot 1A}{3V \cdot 0.9} = \mathbf{1.85A} \quad (6.5)$$

The output voltage was set with a resistor divider to 5V:

$$R1 = \left(\frac{V_{OUT}}{V_{REF}} - 1 \right) \cdot R2 \quad (6.6)$$

Where V_{REF} [24] is 500mV and V_{OUT} is 5V. R2 was set to 100kOhm.

$$R1 = \left(\frac{5V}{500mV} - 1 \right) \cdot 100k\Omega = 900k\Omega \approx \mathbf{910M\Omega(E24)} \quad (6.7)$$

The closest value to $900k\Omega$ was $910k\Omega$ in the E24 series.

To prevent the boost converter from operating when the USB-C port is connected, the VBUS voltage is used to disable the boost converter. When a charger is connected the VBUS signal(5V) it switches the transistor Q1 and pulls the enable pin to ground and disabling it.

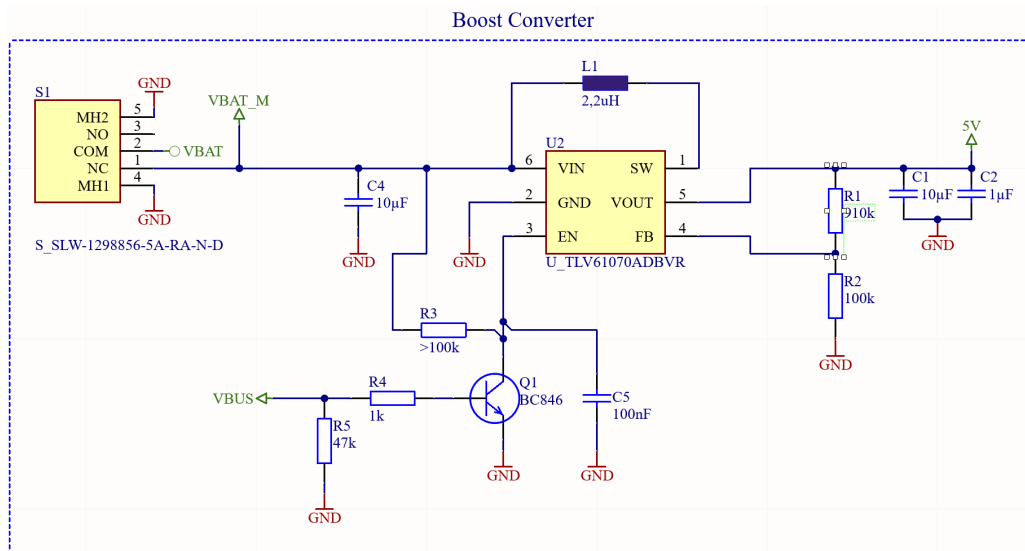


Figure 6.36: Boost-Converter implemented in the schematic.

ON/OFF Switch

The switch disconnects the battery from the boost converter. Note that the battery charging circuit is not influenced since VBUS is directly connected to the BMS as seen in figure 6.35.

PCB - V2.2

The PCB has a size of 30x41.5mm, it fits directly under the basic module. The components used were mainly SMD components, in order to keep the PCB small. It is a 2-layer PCB.

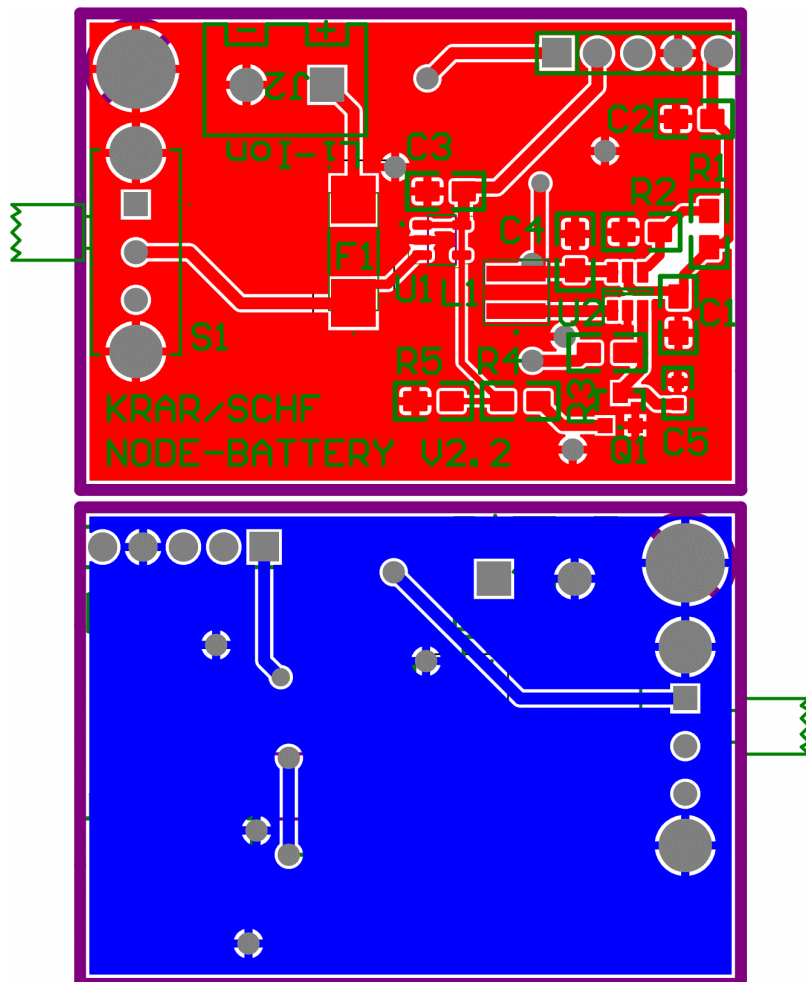


Figure 6.37: PCB of the battery module V2.2. [Red - Frontside, Blue - Backside]

6.4.2 Hardware - V2.1

There is no version 2.0 of the battery module. During the design process, complications arose that directly led to the designation V2.1. Version 2.1 of the battery module is identical to version 2.2, except for the following components:

- 1. A resistor was added to the boost converter circuit.
- 2. Some footprints were different in size.

Boost Converter

A resistor was added to the boost converter to prevent it from turning off when the battery module is connected to the base module. The boost converter was not able to turn on, when the battery module was connected to the base module. The boost converter turned off, even though the battery USB-C was not connected. The reason for this was, that the transistor was switched on due to a small voltage that was present on the base of the transistor when it was connected to the base module.

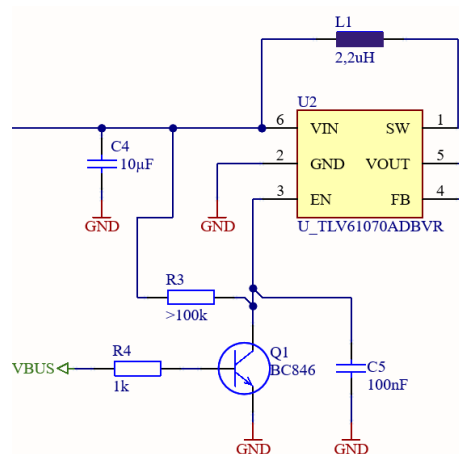


Figure 6.38: Boost-Converter implemented in version 2.1.

To fix this issue, a $47\text{k}\Omega$ resistor was added to the base of the transistor pulling it to ground.

PCB - V2.1

The PCB is identical in size with V2.2 with a footprint of 30x41.5mm.

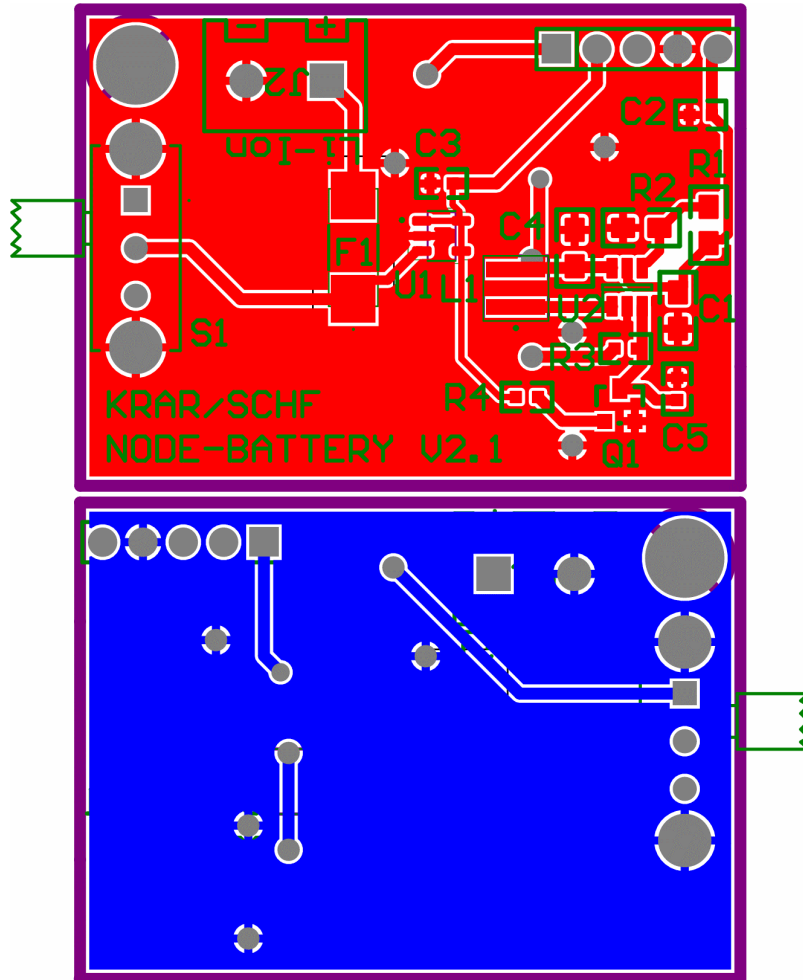


Figure 6.39: PCB of the battery module V2.1. [Red - Frontside, Blue - Backside]

6.4.3 Hardware - V1.0

This version of the battery module is working completely different from the later versions. It was intended to be a rough prototype for the base module V1.0 & V1.1 (see section 6.1) to test the concept of a pluggable battery module. The battery module is used to power the basic node, without the need to connect a USB-C cable.

Module-Connector

The battery module is connected to the following voltages:

- 1. 3V3 - The battery module powers the basic node over this pin
- 2. VBUS - used to charge and disable the battery module when USB is connected
- 3. V_BAT - used to monitor the battery voltage
- 4. EN-MODUL - Used to disable the linear voltage regulator on the basic node
- 5. GND - Ground

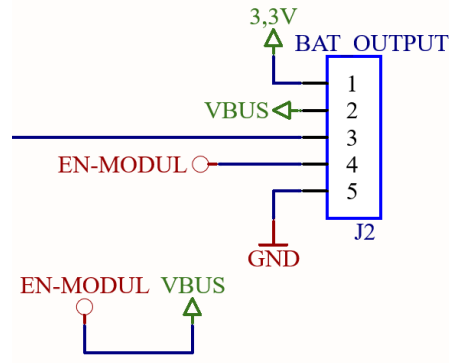


Figure 6.40: Battery implemented in the schematic.

With the voltages the basic node is powered, and the battery is charged.

DC-DC-Converter

A buck-boost converter is used to convert the 3-4.2V from the battery to 3.3V to power the basic module. The used IC is the TPS63000 from Texas Instruments it is a high efficiency buck-boost converter with a efficiency of up to 96%. The inductor and the output voltage were calculated like stated in the datasheet[26]. The inductor was selected after calculating the maximum inductor current:

$$D = \frac{V_{OUT} - V_{IN}}{V_{VOUT}} = \frac{3.3V - 3.0V}{3.3V} = 0.0909 \quad (6.8)$$

$$I_{PEAK} = \frac{I_{OUT}}{\eta \cdot (1 - D)} + \frac{V_{IN} \cdot D}{2 \cdot f \cdot L} \quad (6.9)$$

Where D is the duty cycle, I_{OUT} is the output current set to $0.8A$, η is the efficiency (0.9 as assumption), V_{IN} is the lowest input voltage set to $3V$, f is the switching frequency set to $2.5MHz$ and L is the inductor value set to $2.2\mu H$.

$$I_{PEAK} = \frac{0.8A}{0.9 \cdot (1 - 0.0909)} + \frac{3V \cdot 0.0909}{2 \cdot 2.5MHz \cdot 2.2\mu H} = 1.0025A \approx 1A \quad (6.10)$$

The output voltage was set with a resistor divider to $3.3V$:

$$R1 = R_2 \cdot \left(\frac{V_{OUT}}{V_{FB}} - 1 \right) \quad (6.11)$$

Where V_{FB} is $500mV$, R_2 is $200k\Omega$ and V_{OUT} is $3.3V$.

$$R1 = 200k\Omega \cdot \left(\frac{3.3V}{0.5V} - 1 \right) = 1120k\Omega \approx 1M\Omega \quad (6.12)$$

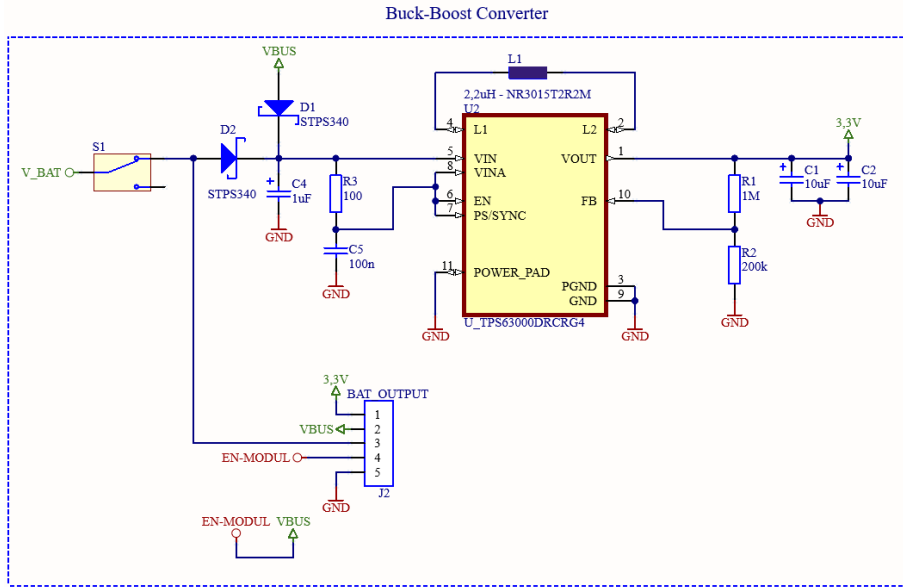


Figure 6.41: DC-DC-Converter implemented in the schematic.

There were problems with getting the TPS63000 to work properly. The output was not at the desired voltage of $3.3V$. This issue was not investigated further, because it was decided to boost the voltage to $5V$ on the battery module and replace the linear regulator on the basic module with a buck converter to improve efficiency. Therefore, a buck-boost converter was not needed anymore in later versions of the battery module.

PCB - V1.0

The PCB of the battery module is a 2-layer board with the same size as the basic module with 75x30mm. The print would be plugged into the basic module V1.1 from underneath.

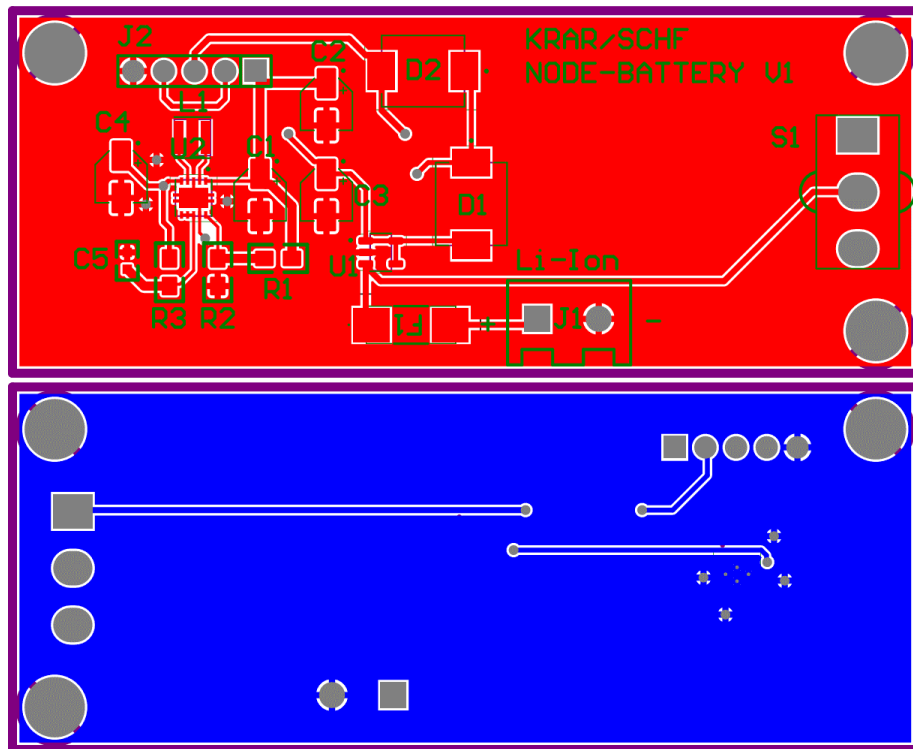


Figure 6.42: PCB of the battery module V1.0. [Red - Frontside, Blue - Backside]

6.5 Switch Module

The switch module allows to switch 230V devices on and off and measure the current draw. This module can not be combined with any other module. The switch module has two revisions, V1.0 and V1.1.

Following components are required to fulfill the requirements:

- 1. Power-Supply
- 2. Switch
- 3. Current-Sensor

6.5.1 Hardware - V1.1(Final)

Power-Supply

Because the switch module is switching 230V, it will also supply itself with this voltage. A transformer was chosen because of the simplicity.

The used print transformer has the type FL4/9[15] it has two 115V primary windings and two 9V secondary windings. The primary windings are connected in serial to the 230V mains and the secondary windings are connected in serial to a full-bridge rectifier.

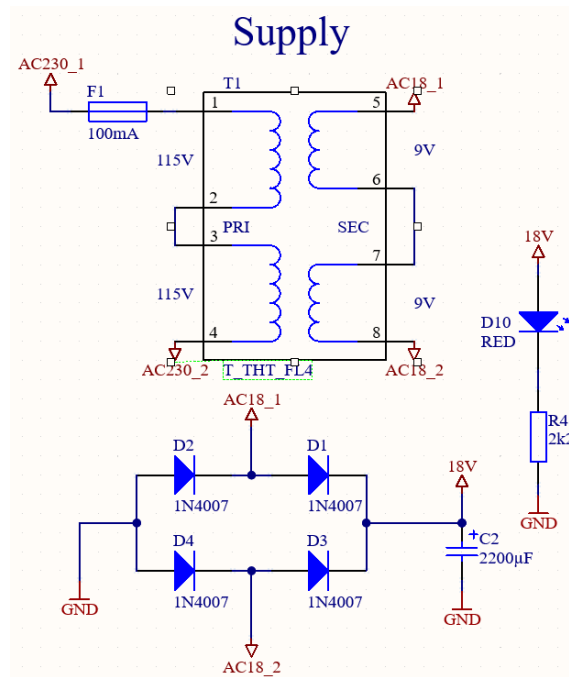


Figure 6.43: Power-Supply implemented in the schematic.

The output capacitor is calculated with the following formula:

- I ... Current draw
- ΔT ... Half the period of the frequency
- ΔU ... Ripple voltage

$$C = \frac{I \cdot \Delta T}{\Delta U} = \frac{100mA \cdot 10ms}{0.5V} = 2mF \approx \mathbf{2.2mF} \quad (6.13)$$

The output voltage of 18V is stepped down to 12V with the LM2575[14] buck converter. The output voltage is set with a resistor divider to 12V. There are fixed and

adjustable versions of the LM2575. If the 12V output version is chosen the Resistor R6 is not needed and R5 is bridged.

The 12V are used to drive the relay and to generate 5V with the LM7805 linear regulator. The base module and current sensor are powered with the 5V.

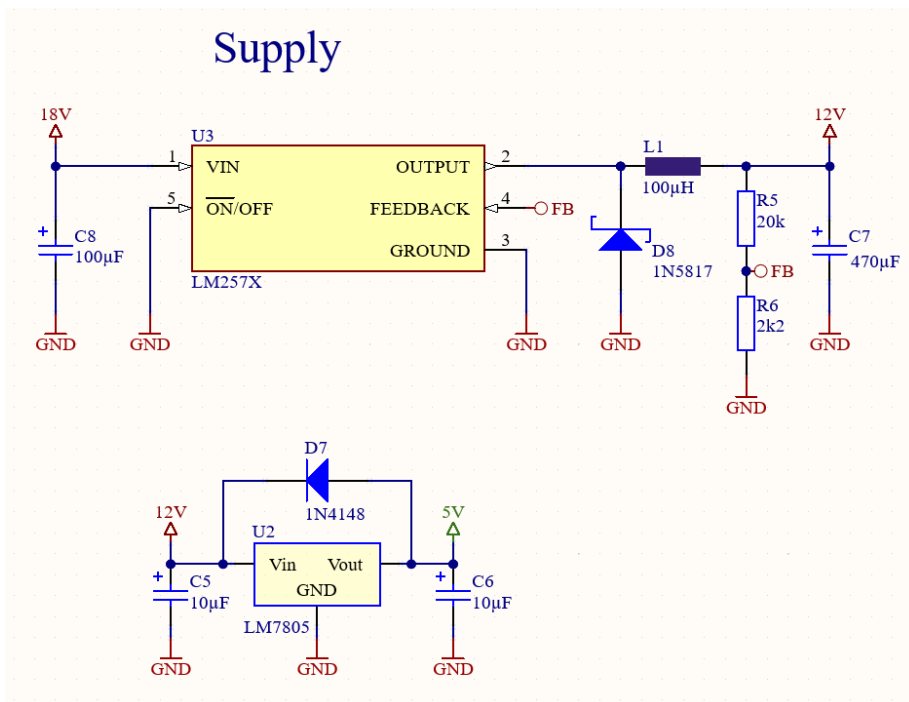


Figure 6.44: Power-Supply implemented in the schematic.

Relay

The switch module uses a relay to switch the 230V devices on and off. The relay used is the JW1aFSN from Panasonic[13]. The relay is controlled by a transistor and a flyback diode to protect the transistor from the voltage spike when the relay is switched off. The relay is rated for 10A at 250VAC. A LED was added to the relay to indicate the state of the relay. The LED is connected to the port of the microcontroller and a resistor to limit the current.

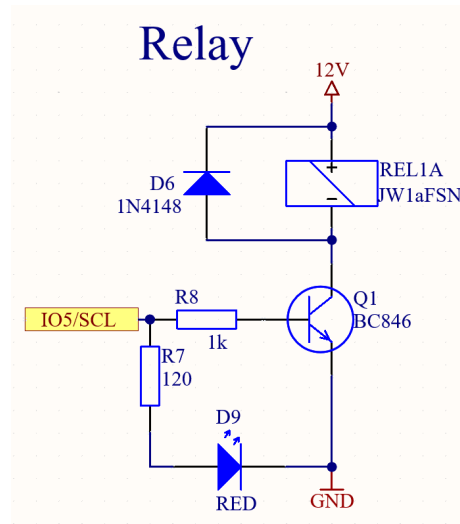


Figure 6.45: Switch implemented in the schematic.

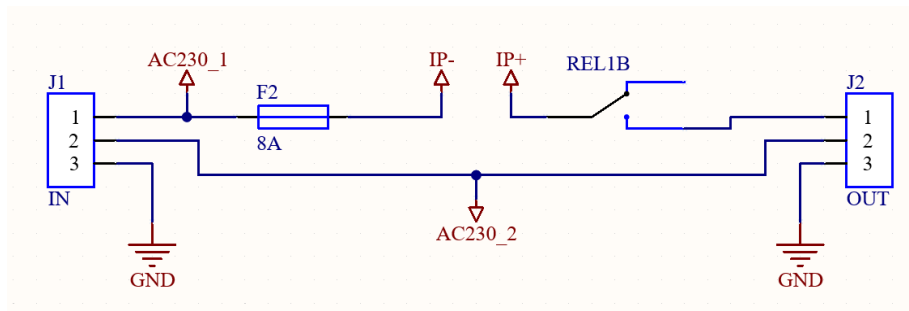


Figure 6.46: Switch implemented in the schematic.

IP- and IP+ are the terminals for the current sensor.

Current-Sensor

The current sensor is used to measure the current draw of the switched device. For this purpose the ACS712ELCTR-20A-S[1] from Allegro was chosen. It is connected to the microcontroller over an analog pin. The sensor is rated for 20A.

The available versions of the sensor were capable of measuring $\pm 5A$, $\pm 20A$ and $\pm 30A$. The 20A version was chosen because it covers the 10A current the relay is capable of. The sensor is a Hall effect sensor, which measures the magnetic field of the current flowing through the wire. The sensor is placed in the current path of the relay. The sensor has a sensitivity of 100mV/A, which means that the output voltage is 100mV for every ampere flowing through the wire.

The IC is powered with 5V and the output on VIOUT is 2.5V when no current and 4.5V when 20A is flowing. In Order to adapt the output to the 3.3V ADC of the ESP32, a voltage divider is used like shown in the datasheet.

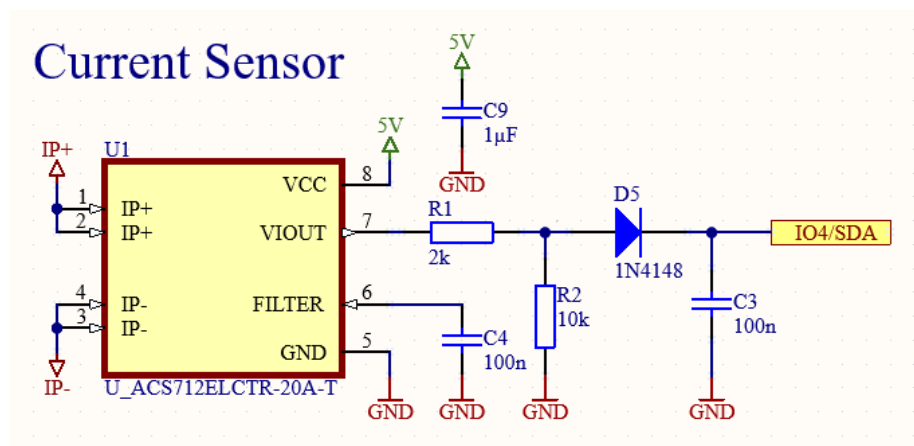


Figure 6.47: Current-Sensor implemented in the schematic as shown in the datasheet.

The sensor is embedded in the current path of the relay. The terminals are electrically isolated from the signal pins like described in the datasheet [1].

The output of this circuit was tested and measured. A power supply in series with an amperemeter was connected to the terminals of the current sensor and the amount of current was set in the power supply. The IC was supplied with 5V from a second power supply. The voltage was measured as the current was altered though the IC.

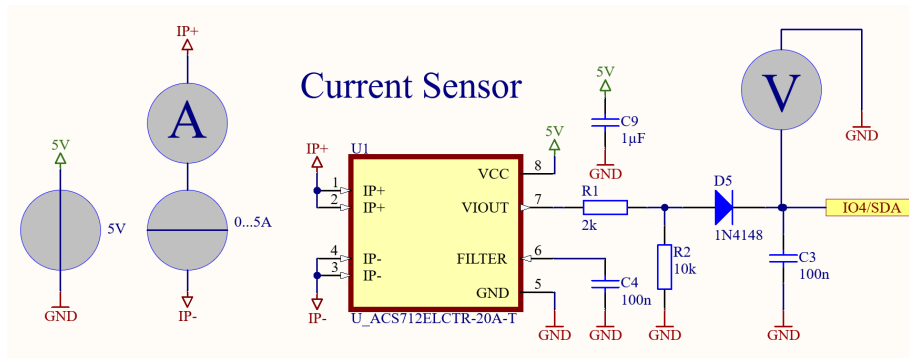
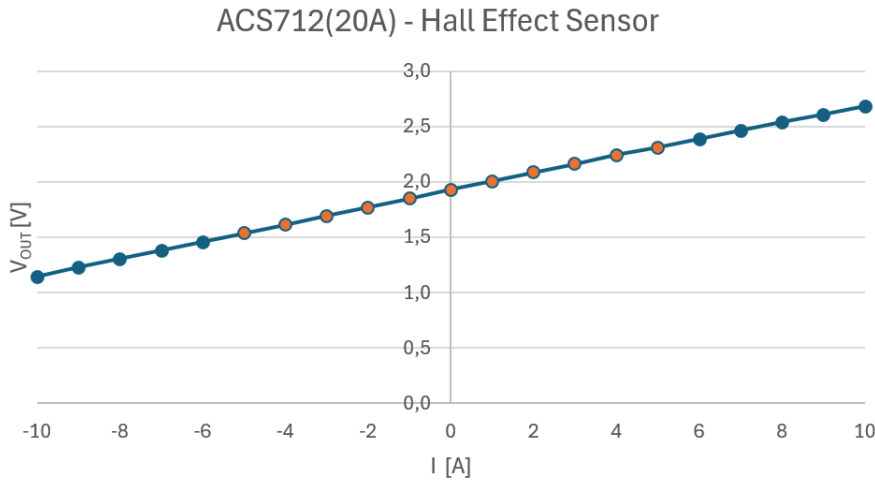


Figure 6.48: Measurement circuit

The orange points are measured. All the other points were generated since the power supply is only capable of 5A. The negative current was achieved by swapping the terminals at the ACS712.



I[A]	V_{OUT} [V]
5	2.310
4	2.240
3	2.160
2	2.087
1	2.006
0	1.928
-1	1.849
-2	1.769
-3	1.691
-4	1.613
-5	1.537

Figure 6.49: Output Voltage versus Sensed Current of the ACS712.

Table 6.1: Measurements

The output voltage stays linear with an average change of 77.3mV/A.

Device	Model	Purpose
Powersupply	EMG 18135	Supply voltage for ACS712
Powersupply	Korad KA3005D DC	Simulating current draw
Multimeter	Voltcraft VC271	Measuring voltage
Multimeter	Fluke 175 true rms	Measuring current

Table 6.2: Used devices for measuring the current sensor.

PCB

The 230V aspect was taken into account when designing the circuit board. The PCB is divided into two sections, the 230V and the low voltage section, which is marked by the ground plane. The relay, current sensor and the transformer are placed in the 230V section. Underneath the current sensor is a milled slot to ensure the isolation of the terminals.

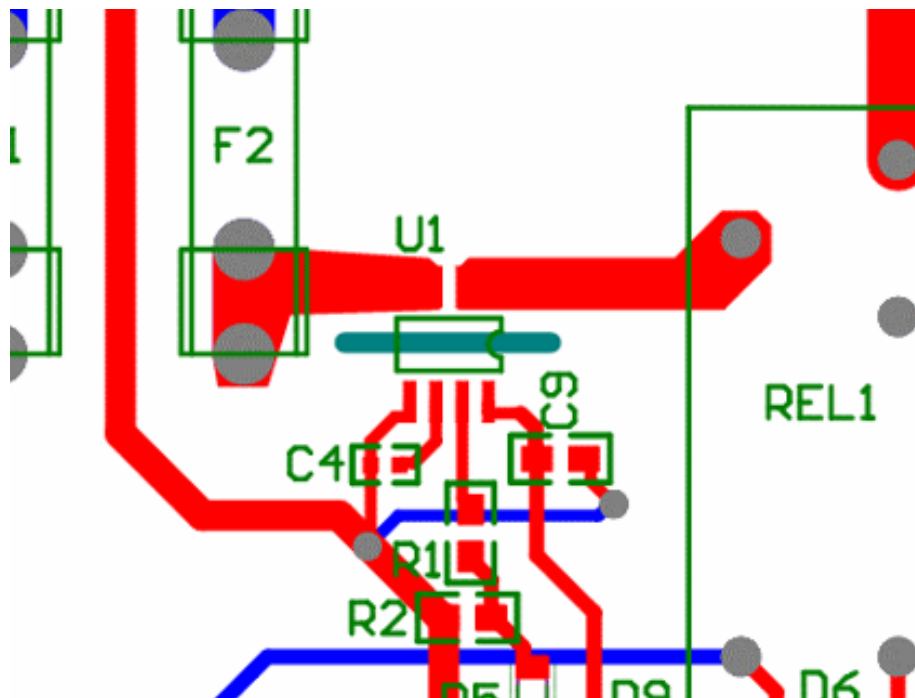


Figure 6.50: Cutout to prevent leakage current.

The PCB has a size of 100x92mm. The switch module uses both SMD and THT components. It was decided to use more THT components on this module, because most parts were already available as THT.

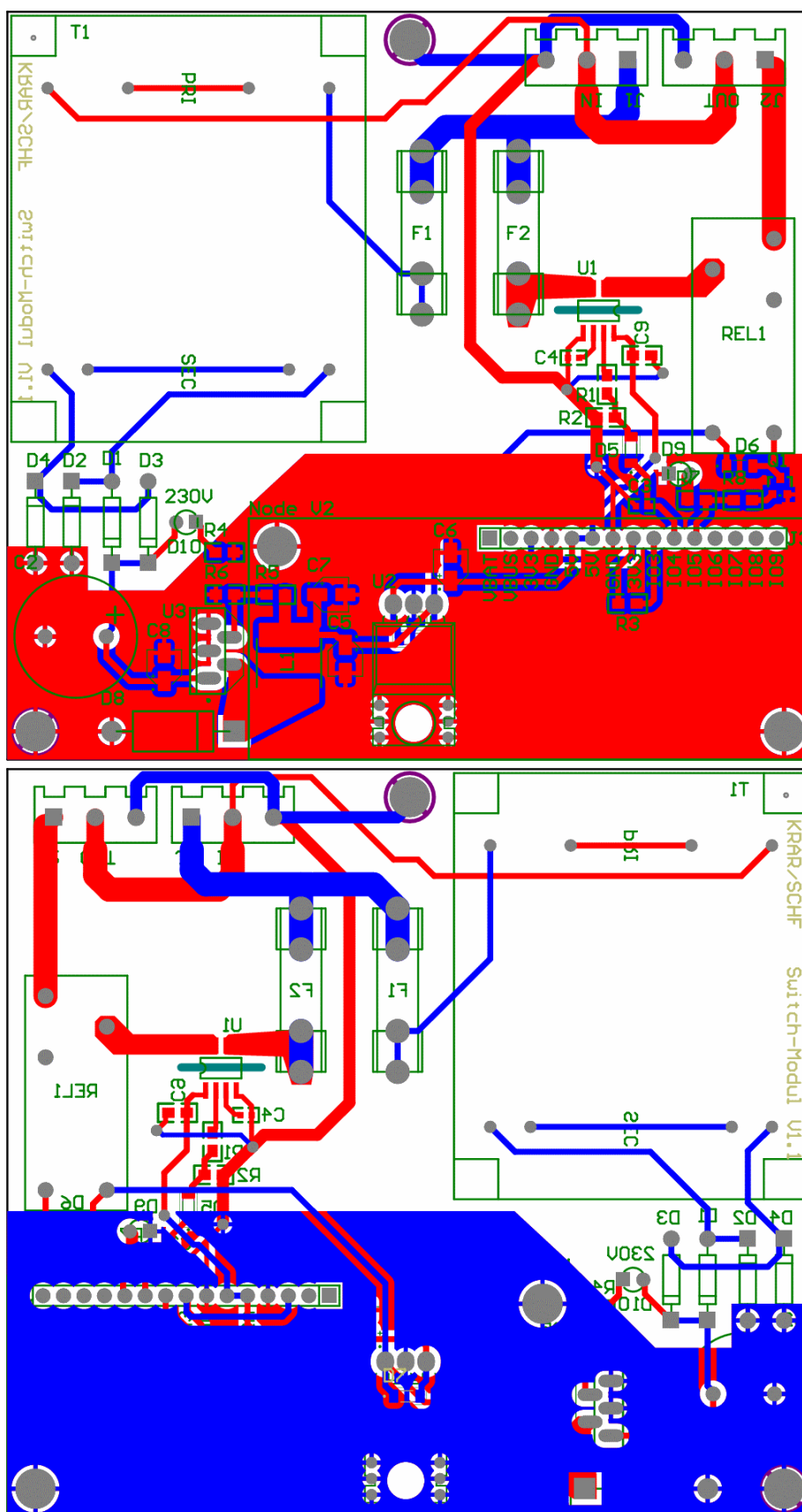


Figure 6.51: PCB of the switch module V1.1 [Blue - Backside, Red - Frontside].

6.5.2 Hardware - V1.0

This version of the switch module is almost identical to the V1.1. Only minor changes were made to ensure the functionality of the module:

Relay

The base resistor of the transistor was forgotten in the schematic. The transistor was destroyed when the MCU pulled the port high. To prevent this in V1.1, a base resistor was added to limit the current.

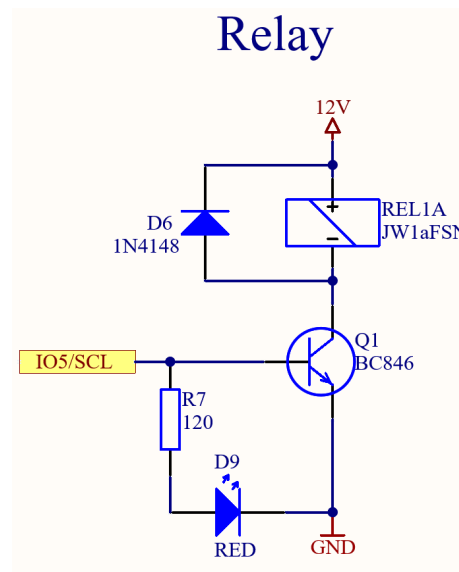


Figure 6.52: Missing base resistor before transistor Q1.

Power-Supply

While testing the module it was found that it is nowhere shown when 230V is present. Therefore, a LED was added after the rectifier to indicate the presence of 230V.

Footprint

Footprints were changed in size to ensure the correct fit of the components.

PCB

The PCB is identical in size with 100x92mm.

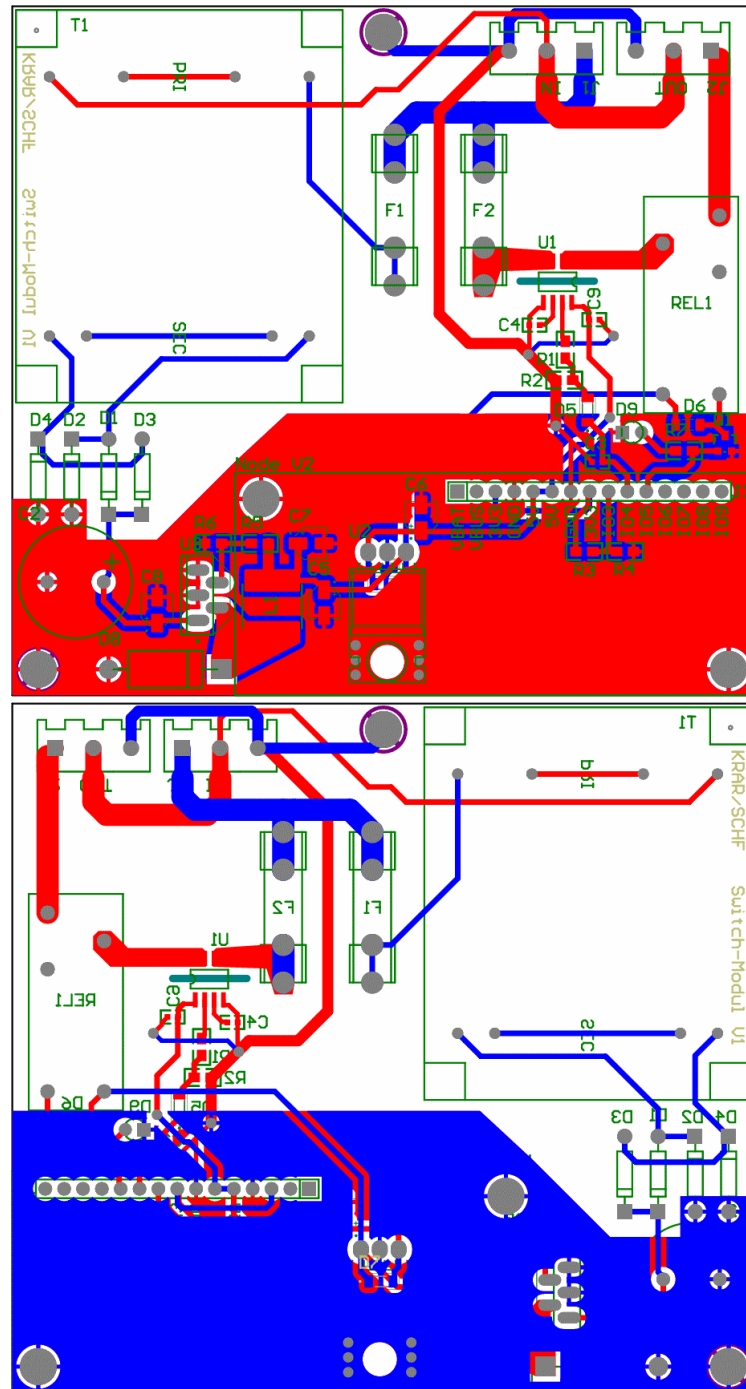


Figure 6.53: PCB of the switch module V1.0 [Blue - Backside, Red - Frontside].

6.6 Case

There were cases designed for every module to protect the electronics inside and give the modules a finished look. The cases were only designed for the final versions of the modules. It was designed in Fusion 360 and manufactured with an FDM 3D printer out of PLA.

6.6.1 Basic

This case consists of two parts, the top and the bottom. The top part has a cutout for the temperature sensor, LEDs the spare header and the serial interface.



Figure 6.54: Top view

The bottom part has a cutout for the module connector.



Figure 6.55: Bottom view

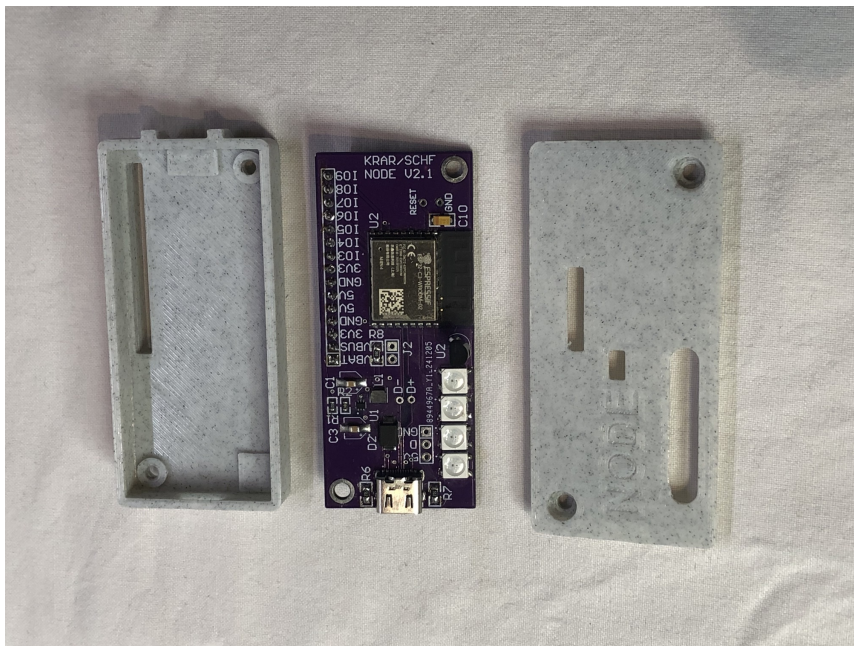


Figure 6.56: Partly disassembled

6.6.2 Battery

This case consists of three pieces, the main part and the two covers for the PCB and the battery.

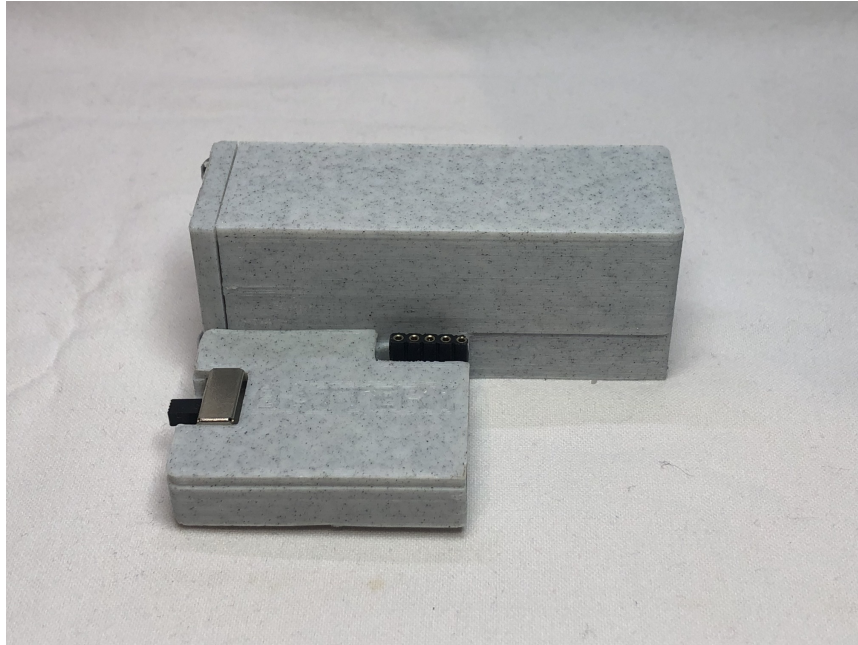


Figure 6.57: Front view

The battery case is designed to hold a 22700 battery.



Figure 6.58: Back view

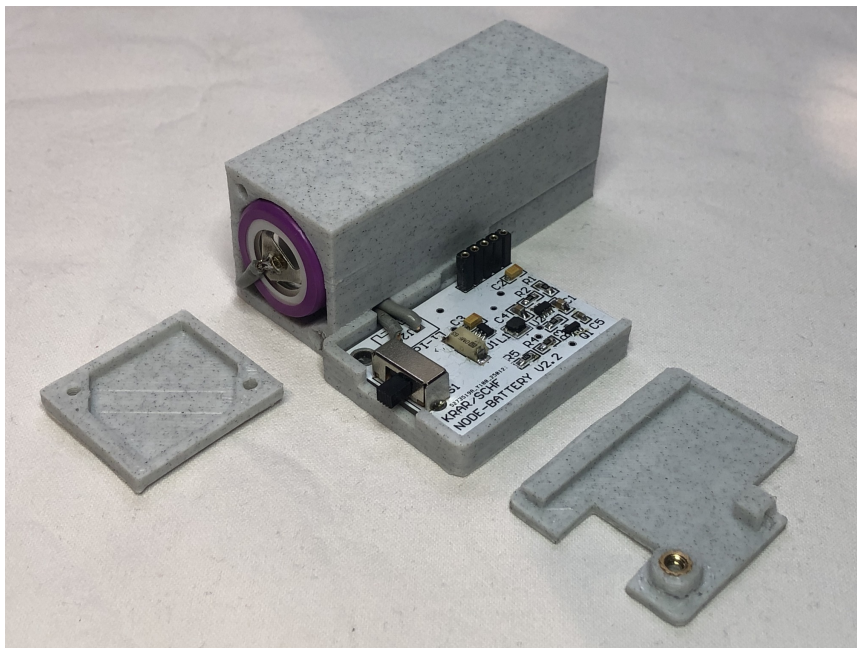


Figure 6.59: Partly disassembled

6.6.3 Motion

This case consists of two parts, the bottom and the cover for the PCB.

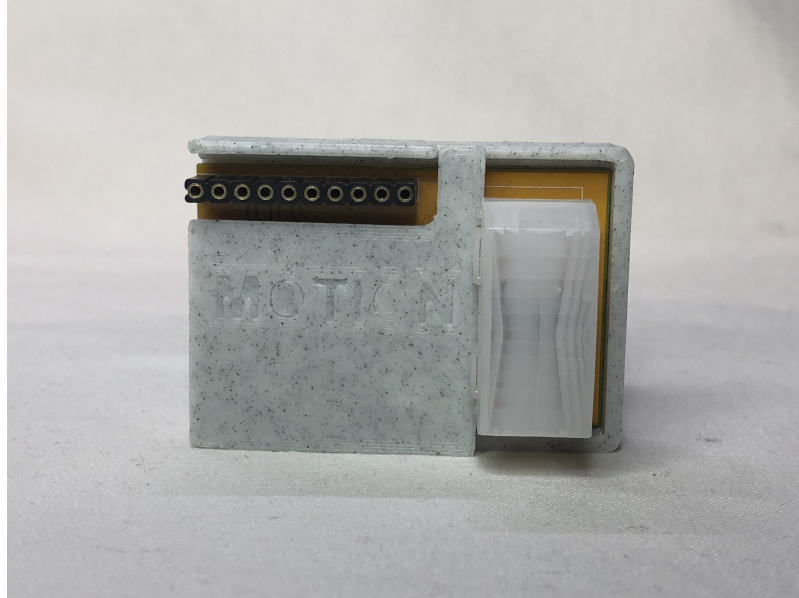


Figure 6.60: Top view

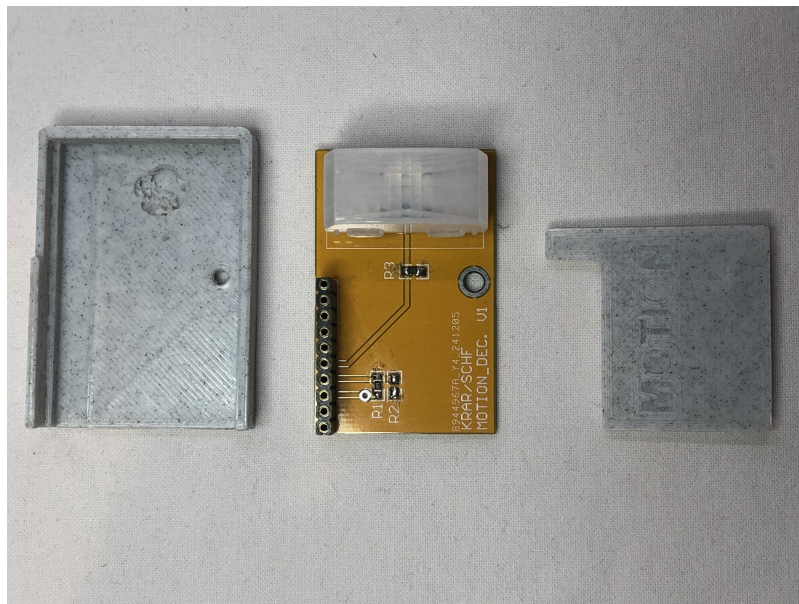


Figure 6.61: Disassembled

6.6.4 Switch

This case consists of three part, the base and the two covers for the PCB.

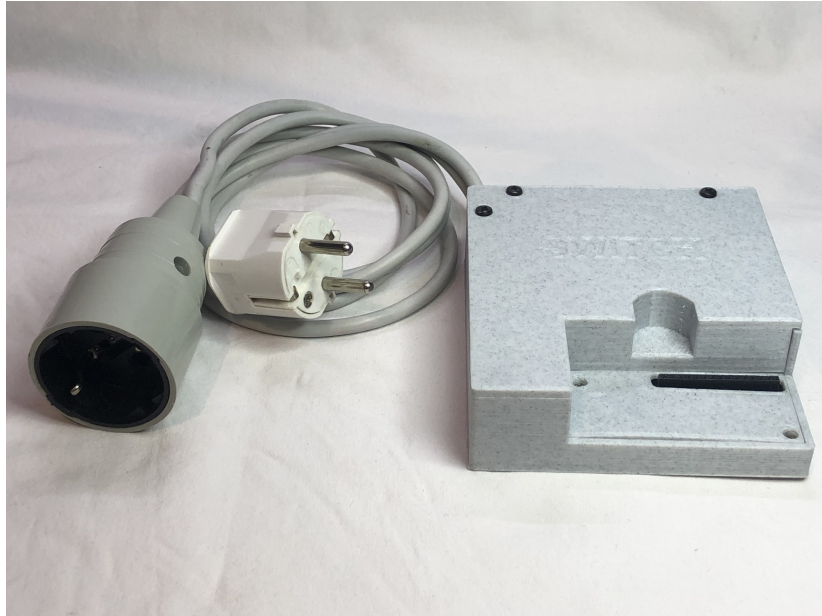


Figure 6.62: Switch fully assembled with input and output cables.

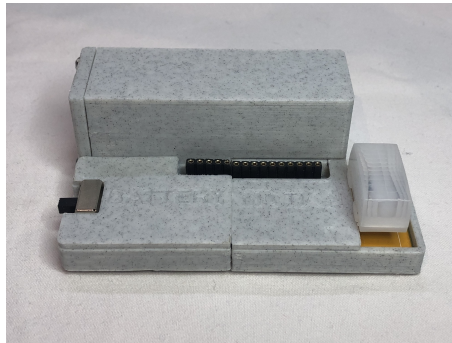


Figure 6.63: Partly disassembled

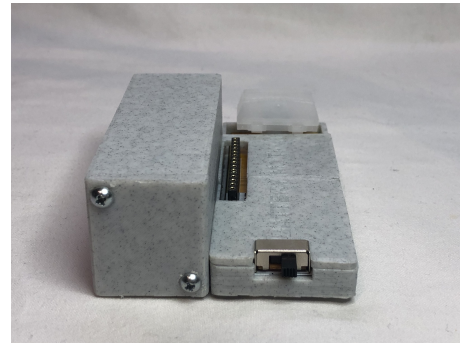
A cable strain relief was integrated into the case.

6.6.5 Combined Module

Base, Battery and Motion Module



(a) Front view



(b) Side view

Figure 6.64: Battery and motion module next to each other.

The base module fits on top of both modules.

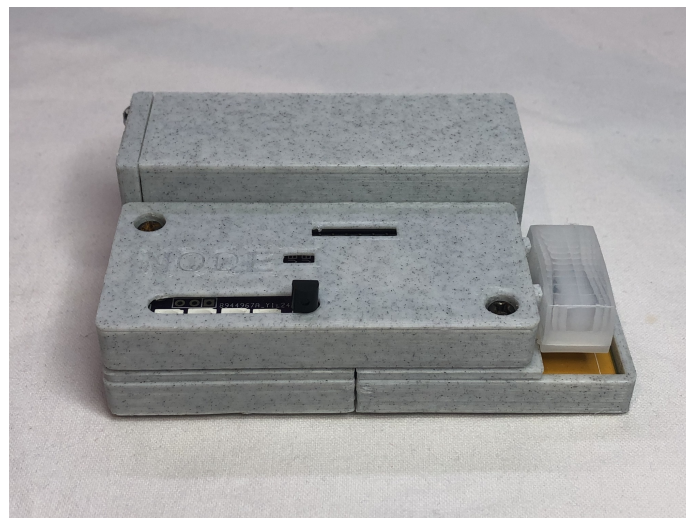


Figure 6.65: Basic, battery and motion module combined.

Base and Motion Module



Figure 6.66: Base module combined with switch module.

A cutout was added to make pulling the base module off the switch module easier.

7 Server

This chapter discusses the software side of the Raspberry Pi Server, including the chosen software and operating system, as well as the implementation of the web server and the option of running the services containerized on different operating systems.

7.1 Operating System

In order to find a suitable operating system for the Raspberry Pi, a few options were considered. The requirements for the operating system are as follows:

- **Lightweight:** Should require as little resources as possible and not come with unnecessary software preinstalled. This includes a Desktop Environment.
- **Software Support:** Has to support the software needed for the project, such as a `Nuxt.js` web server (section 7.2).
- **ARM Based:** As the Raspberry Pi uses an ARM processor, the operating system needs to be compatible with it.
- **Reproducibility:** Should offer a way to replicate the operating system on other Raspberry Pi's.

7.1.1 Raspberry Pi OS Lite

Raspberry Pi OS Lite is a lightweight version of the Raspberry Pi OS. It removes some of the unnecessary software that comes with the full version, such as a Desktop Environment. This makes it ideal for running a server, as it requires less resources and is less prone to security vulnerabilities. It supports all the software needed for the project and offers extensive documentation.

7.1.2 Other ARM-based OS

Many other Linux distros offer a version for ARM processors. For the purpose of this project, there was no need to look further than the Raspberry Pi OS Lite.

7.2 Web Server

This section discusses the implementation of the web server on the Raspberry Pi. The web server is responsible for serving the web application to the user and for handling HTTP requests from the user.

7.2.1 Installing Nuxt.js

To execute the web application, we need to install the npm package manager. This can be done by running the following command:

```
sudo apt install npm
```

Note that this is only necessary if the server is set up manually as the docker compose file already includes the installation of npm. After the installation of npm the application can just be started as npm will automatically install all dependencies.

7.2.2 Requesting Data

This thesis includes a prototype of a web page that fetches temperature from a Slave node, this example exists on <http://localhost:3000/getExample> and can be accessed after the server has been set up.

The rendering process of the page looks like this:

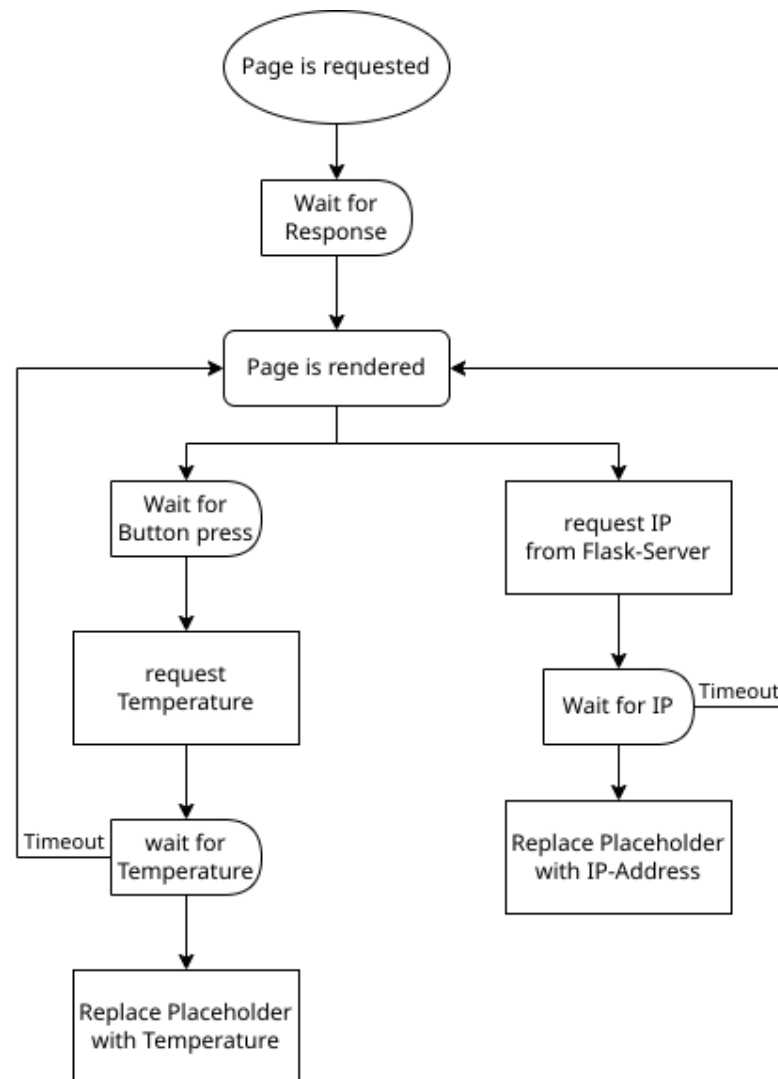


Figure 7.1: Rendering process of the web page

7.3 UDP Process

The Raspberry Pi runs a python process starting at boot with the purpose of listening to a UDP broadcast sent by the bridge node. It then communicates the IP address of the bride to the web server when requested.

7.3.1 Start on boot

The UDP process is started on boot by a systemd service. This section describes how to add a systemd service to the Raspberry Pi.

1. To add a systemd entry first create a systemd service file:
`$ sudo vi /etc/systemd/system/udp_discovery.service`

2. Then add the following in the file:

```
[Unit]
Description=udp discovery
After=network.target

[Service]
ExecStart=/usr/bin/python3 /home/fabian/farismart/Main.py
WorkingDirectory=/home/fabian
StandardOutput=inherit
StandardError=inherit
Restart=always
User=fabian

[Install]
WantedBy=multi-user.target
```

3. Enable the service:
`$ sudo systemctl enable udp_discovery.service`

7.4 Distibution

This section discusses the process of replicating and distributing the operating system of the Raspberry Pi. There are three options to consider.

7.4.1 Image

Making an image of the operating system is the most straightforward method. The image can be created using the following command:

```
sudo dd if=/dev/sdX of=/image.img bs=4M
```

While this method is simple, it has some drawbacks. The image will be the same size as the SD card, even if the card is mostly empty. This can lead to large image files that take up unnecessary space. Additionally, you are locked into the same Raspberry Pi model and partition size.

7.4.2 Setup Script

A setup script can be used to replicate the operating system. This script can be written in any language, but bash is the most common. The script will install all the necessary packages and configurations. This method is more flexible than creating an image, but it requires more work.

7.4.3 Containerization

The best solution is to use containerization. Docker is the most popular containerization tool, it allows you to create a container that contains all the necessary packages and configurations. This container can then be run on any Computer that has Docker installed.

This allows for easy replication of the operating system in addition to the flexibility of not having to use specific hardware.

Dockerfile

A Dockerfile is a text document that contains all the commands a user could call on the command line to assemble an image. This section shows the most important aspects of the Dockerfile. [6]

The Dockerfile starts with a base image. In this case, the base image is the latest version of Debian, then the required packages are installed:

```
FROM debian:latest

# Install required packages
RUN apt-get update && \
    apt-get install -y python3 python3-pip nodejs npm \
    python3-flask python3-flask-cors && \
    rm -rf /var/lib/apt/lists/
```


The next step is to copy the files from the host system to the container. This is done using the `COPY` command. This copies the content of the `SW-Raspberry` directory to the `/app` directory in the container.

```
# Set the working directory
WORKDIR /app

# Copy project files into the container
COPY ./ /app
```

The contents of this directory includes a bash script that will be executed once the updates are finished. This script will start the Flask and Nuxt.js servers:

```
RUN chmod +x /app/start.sh

# Expose ports (adjust as needed)
EXPOSE 3000 5000

# Start command
ENTRYPOINT ["/app/start.sh"]
```

Docker Compose

In Docker, a Compose file (`docker-compose.yml`) is used to define and manage multi-container applications. It allows users to specify services, networks, and volumes in a structured YAML format. While this thesis only uses a single container, the file is still needed to expose the ports and is generally recommended for maintainability.

```
version: '3'
services:
  app:
    build: .
    ports:
      - "3000:3000" # Nuxt.js default port
      - "5000:5000" # Flask default port
```

Network Communication

The approach described in the sections above does host the web server, but it does not allow for communication with the nodes as the UDP discovery process (see section 4.2.1) then saves the IP that bridges the communication between the container and the rest of the network. To fix this, the container needs to be run in host mode. This can be done by adding the following line to the `docker-compose.yml` file:

```
network_mode: "host"
```

This allows the container to use the host's network stack, enabling direct communication with the nodes. This then makes the `ports` section obsolete, as the container will use the host's ports directly. Therefore, it can be removed. The final `docker-compose.yml` file looks like this:

```
version: '3'
services:
  app:
    build: .
    network_mode: "host"
    environment:
      - NUXT_HOST=0.0.0.0
      - NUXT_PORT=3000
```

8 Results

In the scope of this diploma thesis, a concept for a modular smart home system was developed into a working prototype featuring working modules and a complete example of a complete communication chain, meaning a measured value can be requested from a Node, sent to the bridge and then displayed on a web interface. The following chapter will outline the results of the project. It will describe future plans, the results of the range tests conducted with the Nodes and price calculations for the components used in the project.

8.1 Future Plans

This section discusses future plans outside the scope of this diploma thesis, meaning possible improvements or extensions to the system.

8.1.1 Circumventing the ESP-NOW Peer Limit

To circumvent the 20 peer limit per ESP, the bridge can dynamically dedicate another peer to act as a second bridge should the need arise. This would theoretically allow for infinite scalability in terms of Nodes. Implementing this feature would require major refactoring to the bridge's codebase.

8.1.2 Hot-swapping Modules

Currently, the modules are determined at boot time. It is possible to run a thread that periodically checks the connected module and activates/deactivates the corresponding thread. This would allow for true hot-swapping of modules.

8.1.3 Database

Currently, data can only be fetched from the bridge node. A future improvement would be to implement a database on the server that stores all data. This would allow for more complex queries and data analysis.

8.2 Node Range Tests

The range of the nodes was tested in a real-world scenario. To quantify the results, the RSSI value was used. The RSSI value is a measure of the received signal strength. The closer the value is to 0, the stronger the signal. The RSSI value is measured in dBm.

8.2.1 Measuring Procedure

The range of the nodes was tested by placing a Node at a fixed location and walking away from it. The test was conducted in two different environments:

1. An open field
2. A 3-story apartment building

The stationary Node was programmed to periodically send a message to the other one. The other Node was programmed to respond to the message. After the message was received, the time it took to receive the message and the signal strength was recorded. This process was repeated for averaging ten times.

8.2.2 Open Field

The open field test was conducted in a field with no obstacles and up to a distance of 160m.

Distance (m)	RSSI (dBm)	Latency (ms)
10	-65	4
20	-80	5
30	-77	4
40	-88	5
60	-90	6
80	-83	6
100	-89	4
160	-89	4

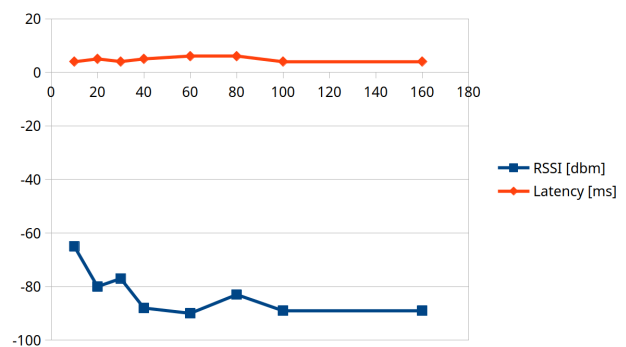


Table 8.1: Node Range Test Results in an Open Field

8.2.3 Indoor Test

The indoor test was conducted in a three-storey apartment building. The Node was placed on the top floor and the second Node was moved further down the building. Every floor is approximately 4m high. The test was conducted up to a distance of 12m, meaning 3 stories down and three ceilings in between Nodes.

Distance (m)	RSSI (dBm)	Latency (ms)
4	-74	11
8	-84	22
12	-89	21

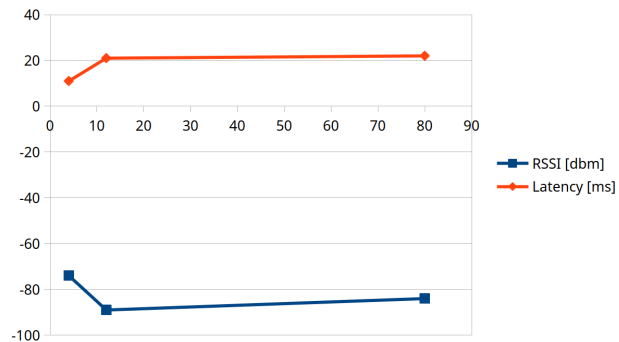


Table 8.2: Node Range Test Results indoors

8.2.4 Conclusion

With the help of the tests above, it can be concluded that the range of the Nodes suffers greatly from obstacles, however for most homes the range should be sufficient. If this is not the case, the peer hopping feature of the protocol can extend the range as needed.

8.3 Costs

This section lists the cost of manufacturing each module per unit. Only the final versions of the modules are listed. Licenses and development costs are not included in this section as the project is open source and not for commercial use. Economic calculations were not included for the same reason.

8.3.1 Base Module

Component	Price
PCB	€5,24
Components	€13,56
Total	€18.80

Table 8.3: Costs of the base module

8.3.2 Battery Module

Component	Price
PCB	€5,24
Components	€6,70
Total	€11.94

Table 8.4: Costs of the battery module

8.3.3 Switch Module

Component	Price
PCB	€5,24
Components	€41,37
Total	€46.61

Table 8.5: Costs of the switch module

8.3.4 Motion Module

Component	Price
PCB	€5,24
Components	€14,39
Total	€19.85

Table 8.6: Costs of the motion module

9 Time Tracking

9.1 Fabian Schätzschock

2024		
Week	Task Description	Hours
36	Preparation	12
47	ESP-NOW	8

2025		
Week	Task Description	Hours
2	FaRiLib Development	20
4	Started on Thesis	20
5	Expanding Thesis	5
5	ESP-NOW	8
8	UDP Broadcast	18
9	ESP-NOW Discovery	21
10	Node-Implementation and Documentation	16
11	Restructured Codebase	19
12	Server-Implementation	21
13	Server-Implementation	20
14	Wrote Thesis	14

Total	202
--------------	------------

9.2 Richard Krammer

2024		
Week	Task Description	Hours
36	Designed schematic NodeV1	4
37	Designed PCB NodeV1 & schematic BatteryV1	8
38	Finished schematic and PCB BatteryV1	3
40	Assembled & tested NodeV1 & BatteryV1	4
41	Parts selection & drawing schematic Node	4
43	Designed NodeV2 PCB	4
44	Designed schematic BatteryV2/SwitchV1; revised NodeV2 to V2.1	11
46	Designed schematic & PCB of SwitchV1; revised BatteryV2 to V2.1	6
47	Assembled & tested NodeV2.1; designed schematic & PCB Motion V1	8
50	SwitchV1 assembled & tested	4
51	BatteryV2.1 & MotionV1 assembled and tested	8

2025		
Week	Task Description	Hours
1	Fixed issues on Node, Battery, Switch, Motion	6
4	Designed case for NodeV2.x	4
5	Assembled & tested NodeV2.2 & BatteryV2.2	5
6	Redesigned case for Node, tested module compatibility	5
7	Wrote Thesis	5
8	Assembled & tested SwitchV1.1, wrote Thesis	5
9	Wrote Thesis	3
10	Designed case for Motion & Battery; assembled BatteryV2.2	8
11	Designed case for switch; wrote thesis	8
12	Wrote Thesis	9
13	Modules installed in the housing; wrote Thesis	19
14	Wrote Thesis	15

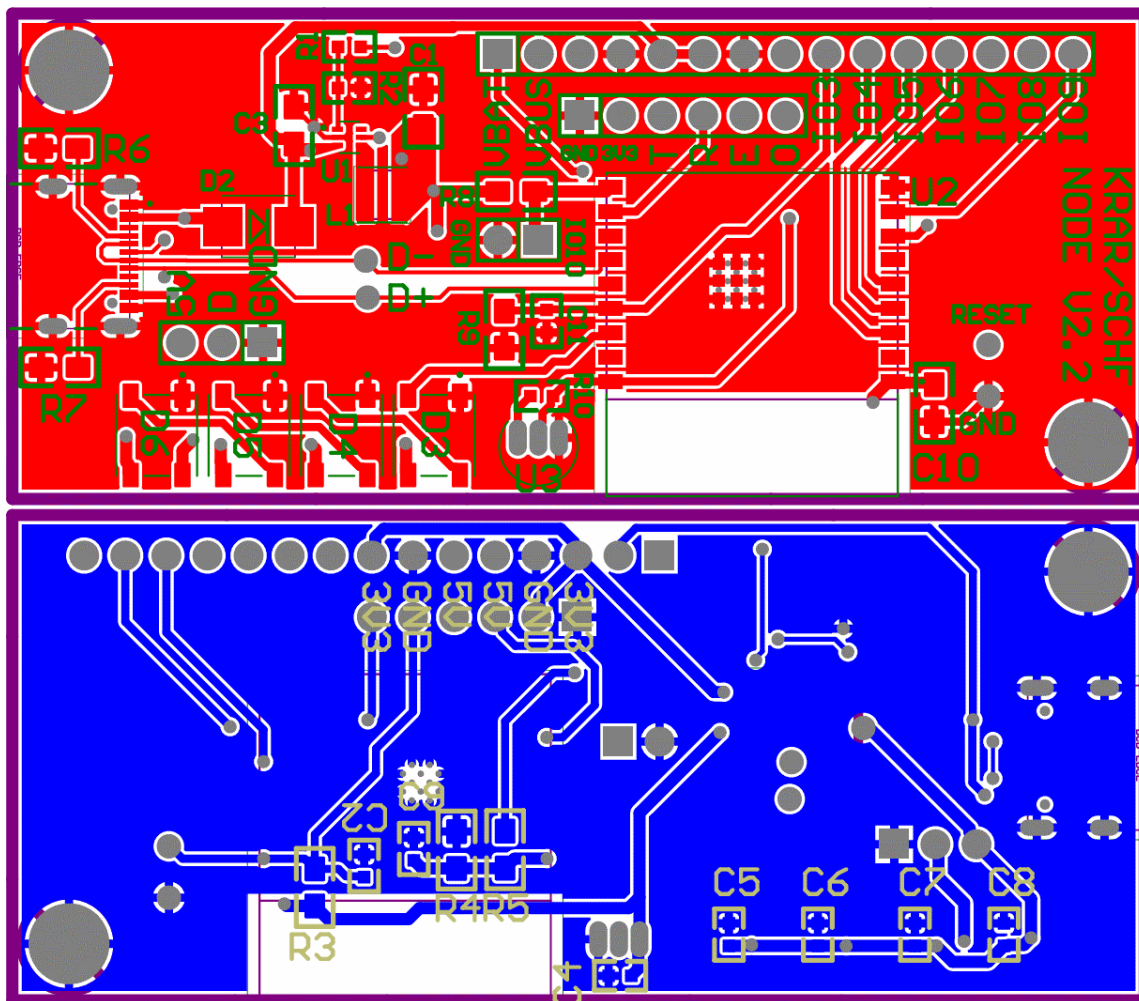
Total	156
--------------	------------

10 Production Files

The production files are the files that are used to produce the hardware. This includes the schematic, the PCB layout and the bill of materials (BOM).

10.1 Basic Module

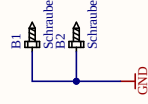
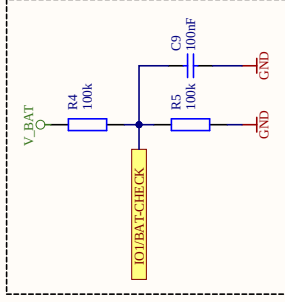
Basic Module V2.2



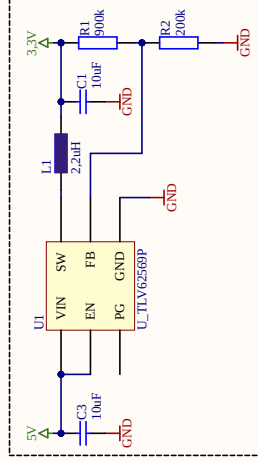
Basic Module V2.2 - BOM

Name	Description	Designator	LibRef	Footprint	Quantity
Schraube	Schraubenloch für M3 Schrauben	B1, B2	B_Schraube	B_Schraube-M3 PAD	2
10uF	Kondensator SMD Universal, Keramik, Folie, usw.	C1, C3	C_SMD_KO	C_SMD_0805	2
100nF	Kondensator SMD Universal, Keramik, Folie, usw.	C2, C4, C5, C6, C7, C8, C9, C11	C_SMD_KO	C_SMD_0603	8
1uF	Tantalkondensator	C10	C_SMD_Tantal	C_SMD_0805	1
STPS340	Diode SMD Schottky	D2	D_SMD_Schottky	D_DO-221	1
WS2812b	Diode SMD ARGB	D3, D4, D5, D6	D_SMD_LED-ARGB	D_SMD_LED-ARGB	4
MODULE_COM	Stecker	J1	J_THT_CON15	SP-15	1
BUTTON	Stecker	J2	J_THT_CON2	SP-2	1
SERIAL	Stecker	J3	J_THT_CON6	SP-6	1
J_USB-C	USB4105-GF-	J4	J_USB-C	GCT_USB4105-GF-A	1
ARGB_OUTPUT	Stecker	J6	J_THT_CON3	Sp-3	1
2,2uH	Spule SMD	L1	L_SMD_Spule	L_SMD_ND_NF301 5T2R2M	1
900k	Widerstand SMD	R1	R_SMD_Widerstand	R_SMD_0603	1
200k	Widerstand SMD	R2	R_SMD_Widerstand	R_SMD_0603	1
>10k	Widerstand SMD	R3	R_SMD_Widerstand	R_SMD_0805	1
100k	Widerstand SMD	R4, R5, R8, R9	R_SMD_Widerstand	R_SMD_0805	4
5k1	Widerstand SMD	R6, R7	R_SMD_Widerstand	R_SMD_0805	2
4k7	Widerstand SMD	R10	R_SMD_Widerstand	R_SMD_0603	1
U_TLV62569P		U1	U_TLV62569P	SOT50P160X60-6N	1
ESP32-C3-WROOM-02-N4	WiFi Modules (802.11) (Engineering Samples) SMD module, ESP32-C3, 4MB SPI flash, PCB antenna, -40 C+85 C	U2	ESP32-C3-WROOM-02-N4	MODULE_ESP32-C3-WROOM-02-H4	1
U_DS18B20	Temperature Sensor Digital, Local -55Å°C~125Å°C 12b 8-uMAX-EP 8-uSOP-EP	U3	U_DS18B20	TO-92_DS18B20	1

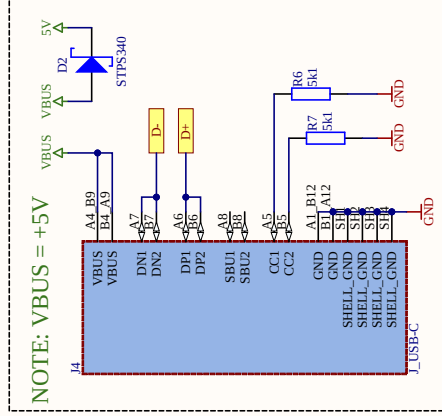
Battery Monitoring



DC-DC Converter

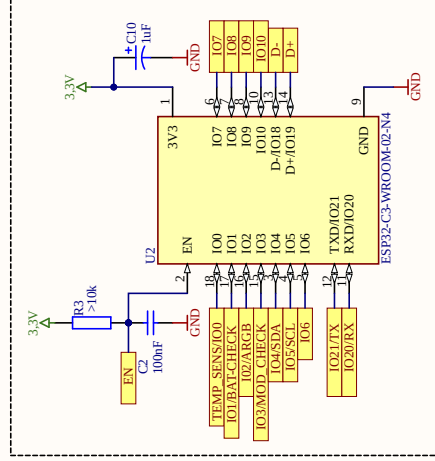


USB-C Connector

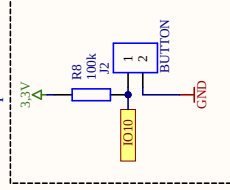


NOTE: VBUS = +5V

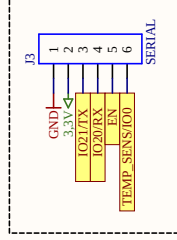
Microcontroller



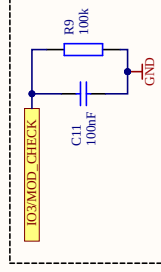
Input



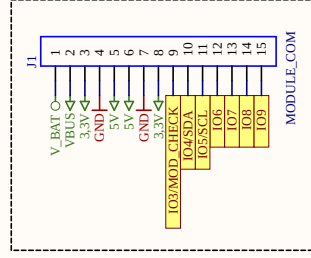
Serial-Connection



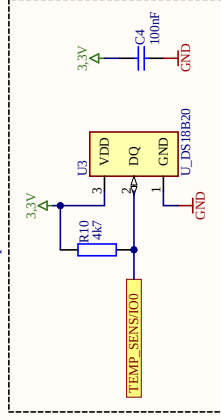
Module-Detection



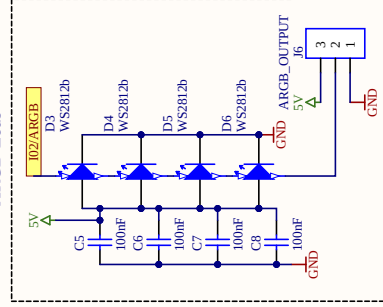
Module-Port



Temperature Sensor



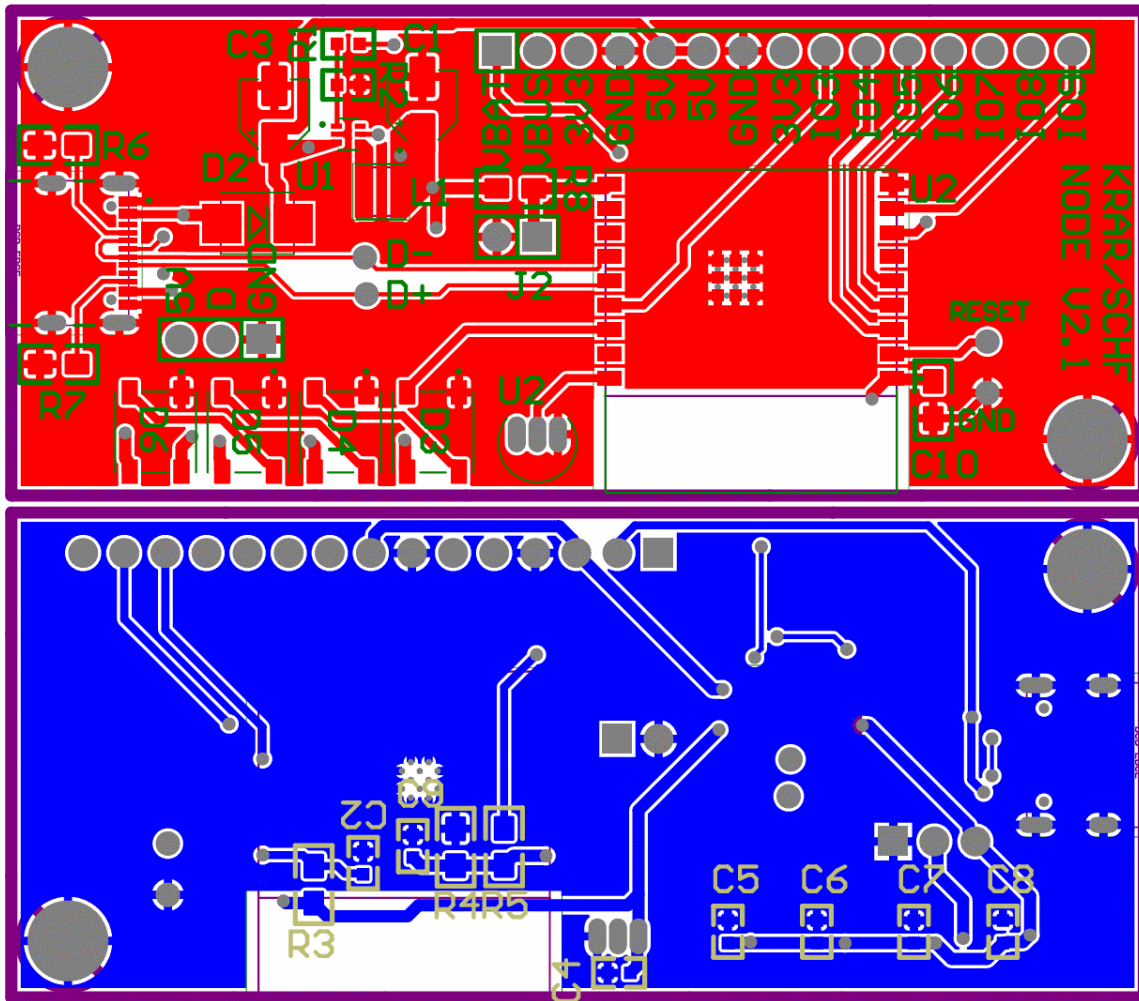
ARGB-Leds



NOTE

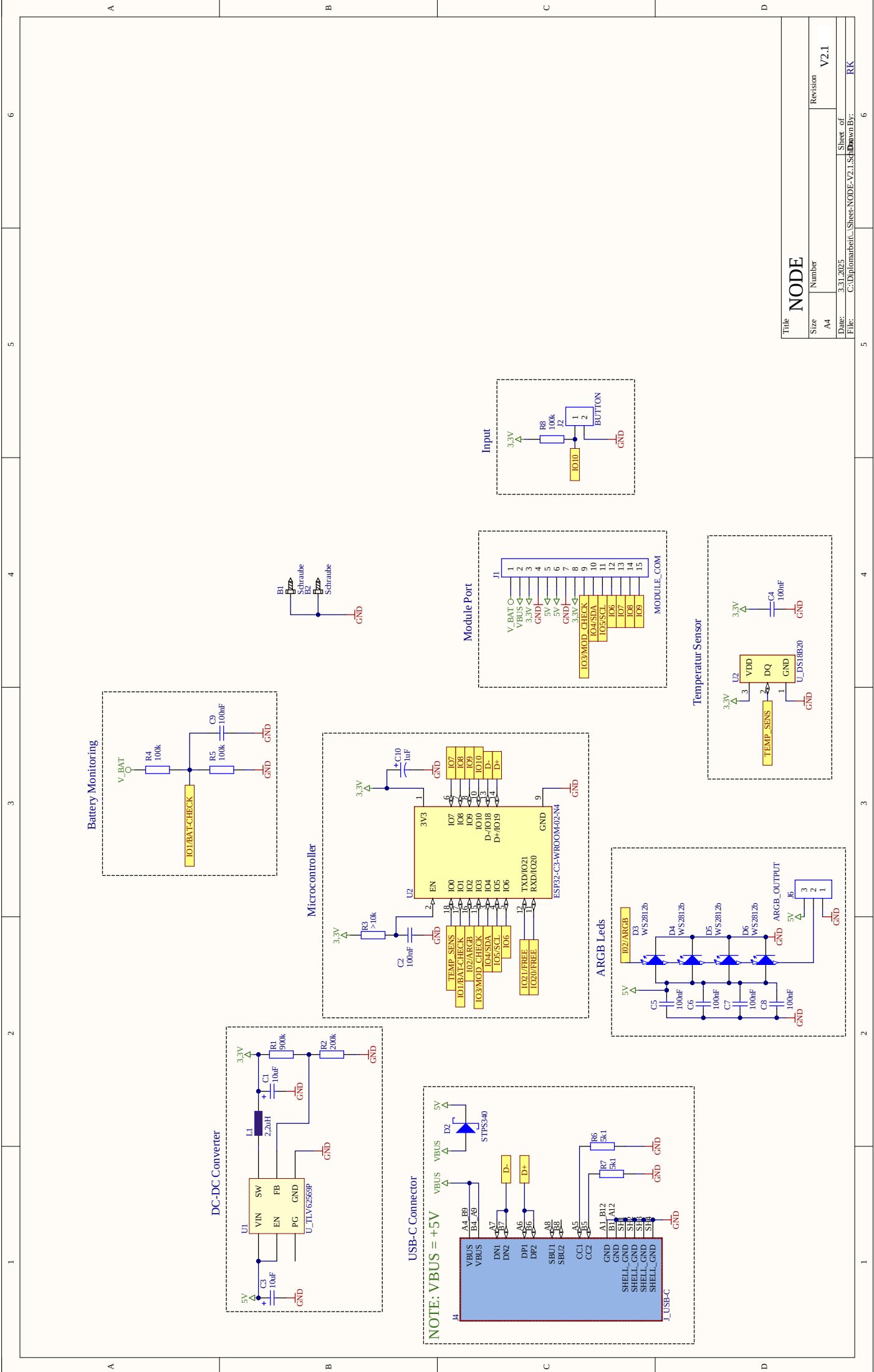
Title	Size	Number	Revision
	A4		V2.2
Date:	File:	Sheet of	Drawn By:
3.24.2025	C:\Diplomarbeit\	Sheet 1. SchDoc	RK

Basic Module V2.1



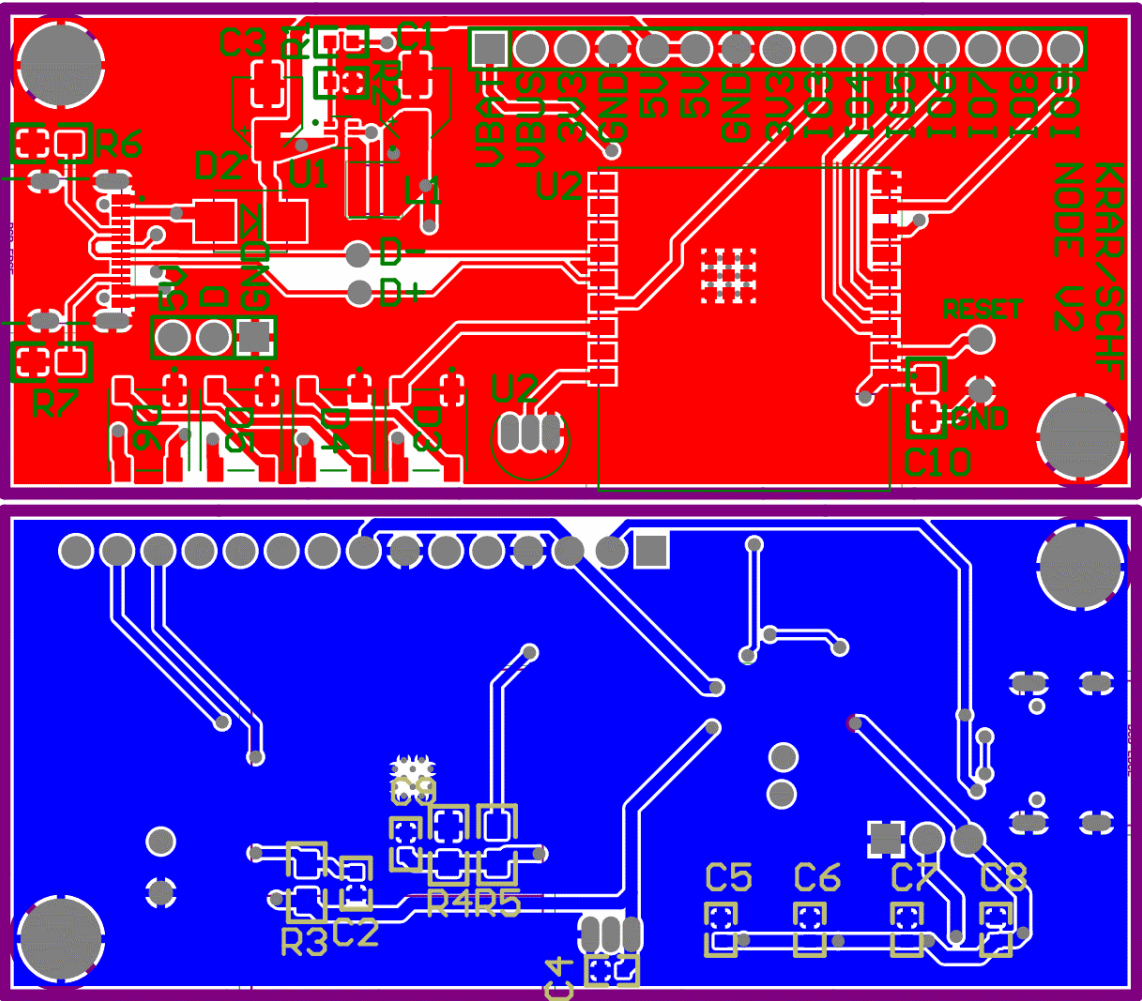
Basic Module V2.1 - BOM

Comment	Description	Designator	Footprint	LibRef	Quantity
Schraube	Schraubenloch für M3 Schrauben	B1, B2	B_Schraube-M3 PAD	B_Schraube	2
10uF	SMD Elektrolytkondensator	C1, C3	C_SMD_B-Typ	C_SMD_ELK	2
100nF	Kondensator SMD Universal, Keramik, Folie, usw.	C2, C4, C5, C6, C7, C8, C9	C_SMD_0603	C_SMD_KO	7
1uF	Tantalkondensator	C10	C_SMD_0805	C_SMD_Tantal	1
STPS340	Diode SMD Schottky	D2	D_DO-221	D_SMD_Schottky	1
WS2812b	Diode SMD ARGB	D3, D4, D5, D6	D_SMD_LED-ARGB	D_SMD_LED-ARGB	4
MODULE_COM	Stecker	J1	SIP-15	J_THT_CON15	1
BUTTON	Stecker	J2	SIP-2	J_THT_CON2	1
J_USB-C	USB4105-GF-	J4	GCT_USB4105-GF-A	J_USB-C	1
ARGB_OUTPUT	Stecker	J6	Sip-3	J_THT_CON3	1
2,2uH	Spule SMD	L1	L_SMD_ND_NR3015T2R2M	L_SMD_Spule	1
900k	Widerstand SMD	R1	R_SMD_0603	R_SMD_Widerstand	1
200k	Widerstand SMD	R2	R_SMD_0603	R_SMD_Widerstand	1
>10k	Widerstand SMD	R3	R_SMD_0805	R_SMD_Widerstand	1
100k	Widerstand SMD	R4, R5, R8	R_SMD_0805	R_SMD_Widerstand	3
5k1	Widerstand SMD	R6, R7	R_SMD_0805	R_SMD_Widerstand	2
U_TLV62569P		U1	SOT50P160X60-6N	U_TLV62569P	1
ESP32-C3-WROOM-02-N4	WiFi Modules (802.11) (Engineering Samples) SMD module, ESP32-C3, 4MB SPI flash, PCB antenna, -40 C +85 C	U2	MODULE_ESP32-C3-WROOM-02-H4	ESP32-C3-WROOM-02-N4	1
U_DS18B20	Temperature Sensor Digital, Local -55Â°C ~ 125Â°C 12 b 8-uMAX-EP 8-uSOP-EP	U2	TO-92_DS18B20	U_DS18B20	1



Title			NODE		
Size	Number	Revision	V2.1		
A4					
Date:			3.31.2025		
File:			C:\Diplomarbeit\Sheet-NODE-V2.1_SchDown By: RK		

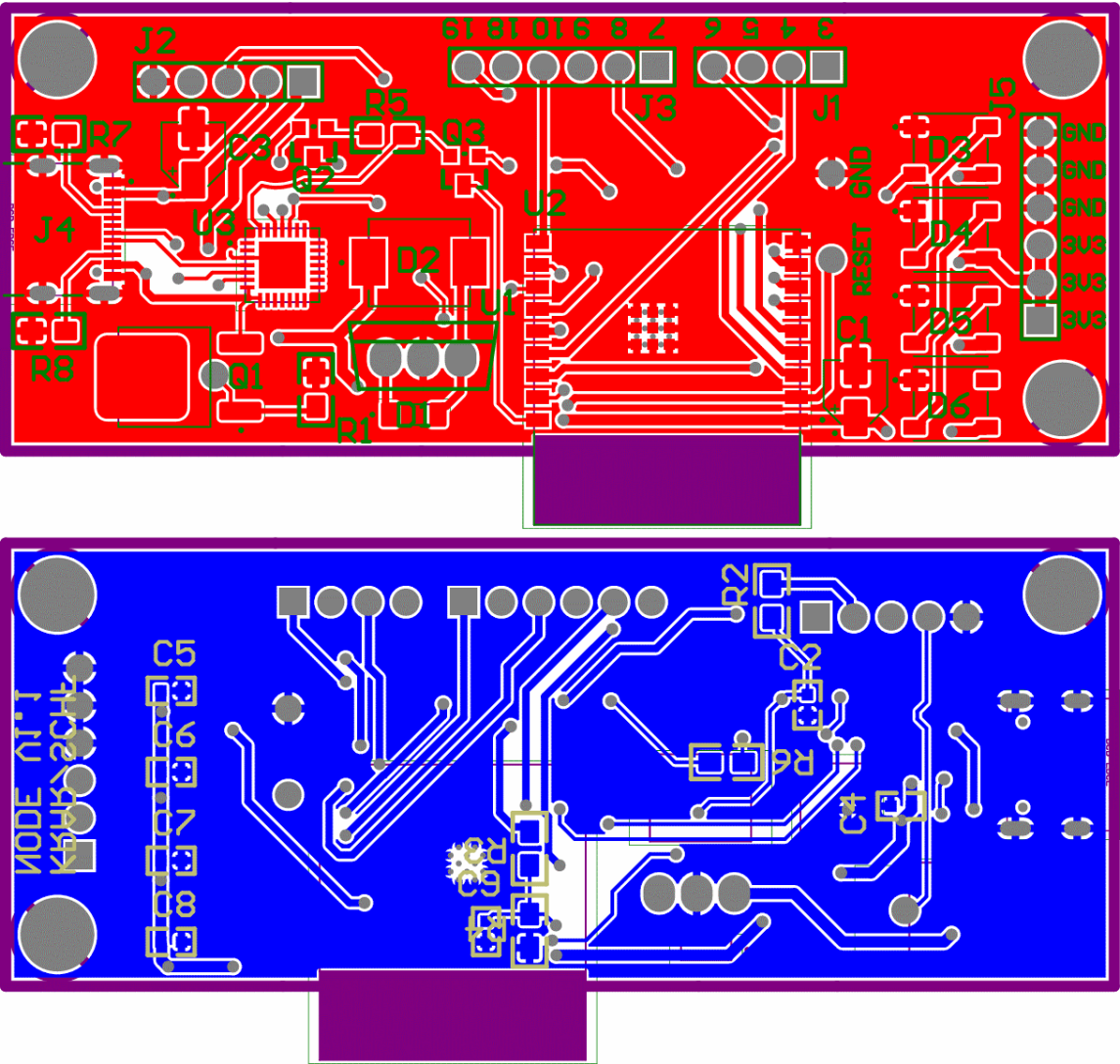
Basic Module V2.0



Basic Module V2.0 - BOM

Comment	Description	Designator	Footprint	LibRef	Quantity
Schraube	Schraubenloch für M3 Schrauben	B1, B2	B_Schraube-M3 PAD	B_Schraube	2
10uF	SMD Elektrolytkondensator	C1, C3	C_SMD_B-Typ	C_SMD_ELK	2
100nF	Kondensator SMD Universal, Keramik, Folie, usw.	C2, C4, C5, C6, C7, C8, C9	C_SMD_0603	C_SMD_KO	7
1uF	Tantalkondensator	C10	C_SMD_0805	C_SMD_Tantal	1
STPS340	Diode SMD Schottky	D2	D_DO-221	D_SMD_Schottky	1
WS2812b	Diode SMD ARGB	D3, D4, D5, D6	D_SMD_LED-ARGB	D_SMD_LED-ARGB	4
MODULE_COM	Stecker	J1	SIP-15	J_THT_CON15	1
J_USB-C	USB4105-GF-	J4	GCT_USB4105-GF-A	J_USB-C	1
ARGB_OUTPUT	Stecker	J6	Sip-3	J_THT_CON3	1
2,2uH	Spule SMD	L1	L_SMD_ND_NR3015T2R2M	L_SMD_Spule	1
900k	Widerstand SMD	R1	R_SMD_0603	R_SMD_Widerstand	1
200k	Widerstand SMD	R2	R_SMD_0603	R_SMD_Widerstand	1
>10k	Widerstand SMD	R3	R_SMD_0805	R_SMD_Widerstand	1
100k	Widerstand SMD	R4, R5	R_SMD_0805	R_SMD_Widerstand	2
5k1	Widerstand SMD	R6, R7	R_SMD_0805	R_SMD_Widerstand	2
U_TLV62569P		U1	SOT50P160X60-6N	U_TLV62569P	1
ESP32-C3-WROOM-02-N4	WiFi Modules (802.11) (Engineering Samples) SMD module, ESP32-C3, 4MB SPI flash, PCB antenna, -40 C +85 C	U2	MODULE_ESP32-C3-WROOM-02-H4	ESP32-C3-WROOM-02-N4	1
U_DS18B20	Temperature Sensor Digital, Local -55Â°C ~ 125Â°C 12 b 8-uMAX-EP 8-uSOP-EP	U2	TO-92_DS18B20	U_DS18B20	1

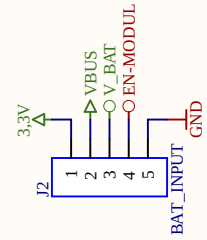
Basic Module V1.1



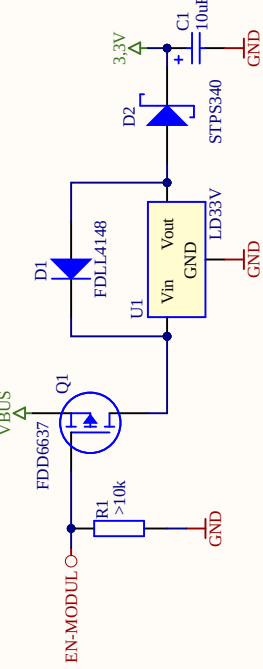
Basic Module V1.1 - BOM

Comment	Description	Designator	Footprint	LibRef	Quantity
Schraube	Schraubenloch für M3 Schrauben	B1, B2, B3	B_Schraube-M3 PAD	B_Schraube	3
10uF	SMD Elektrolytkondensator	C1, C3	C_SMD_B-Typ	C_SMD_ELK	2
100nF	Kondensator SMD Universal, Keramik, Folie, usw.	C2, C4, C5, C6, C7, C8, C9	C_SMD_0603	C_SMD_KO	7
FDLL4148	Diode SMD Silizium	D1	SOD-80-2	D_SMD_Silizium	1
STPS340	Diode SMD Schottky	D2	D_DO-214	D_SMD_Schottky	1
WS2812b	Diode SMD ARGB	D3, D4, D5, D6	D_SMD_LED-ARGB	D_SMD_LED-ARGB	4
IO 3-6	Stecker	J1	Sip-4	J_THT_CON4	1
BAT_INPUT	Stecker	J2	SIP-5	J_THT_CON5	1
IO 7-10&18-19	Stecker	J3	SIP-6	J_THT_CON6	1
J_USB-C	USB4105-GF-	J4	GCT_USB4105-GF-A	J_USB-C	1
POWER	Stecker	J5	SIP-6	J_THT_CON6	1
FDD6637	P-Kanal MOSFET	Q1	FDD6637_TO-252-3	Q_SMD_MOSFET-P-Kanal	1
BC846	NPN Transistor	Q2, Q3	SOT-23-3_BC846	Q_SMD_NPN	2
>10k	Widerstand SMD	R1, R2	R_SMD_0805	R_SMD_Widerstand	2
100k	Widerstand SMD	R3, R4	R_SMD_0805	R_SMD_Widerstand	2
10k	Widerstand SMD	R5, R6	R_SMD_0805	R_SMD_Widerstand	2
5k1	Widerstand SMD	R7, R8	R_SMD_0805	R_SMD_Widerstand	2
LD33V	Linearwandler	U1	TO-220-3V	U_THT_78xx	1
ESP32-C3-WROOM-02-N4	WiFi Modules (802.11) (Engineering Samples) SMD module, ESP32-C3, 4MB SPI flash, PCB antenna, -40 C +85 C	U2	MODULE_ESP32-C3-WROOM-02-H4	ESP32-C3-WROOM-02-N4	1
U_CP2102	CP2102N USB 2.0 to UART (RS232) Bridge Interface Evaluation Board	U3	QFN50P500X500X100-29N	U_CP2102	1

Battery Connector

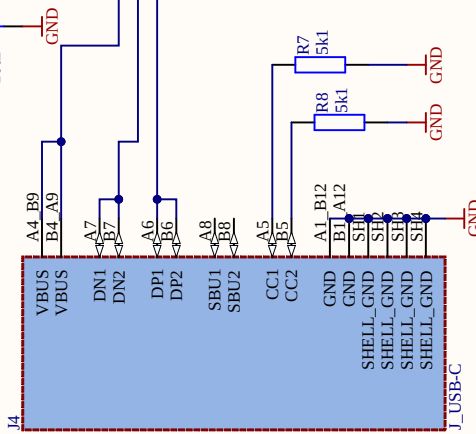


Linear Regulator

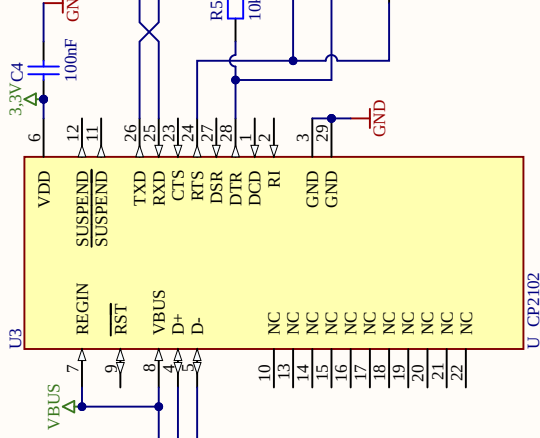


USB-C Connector

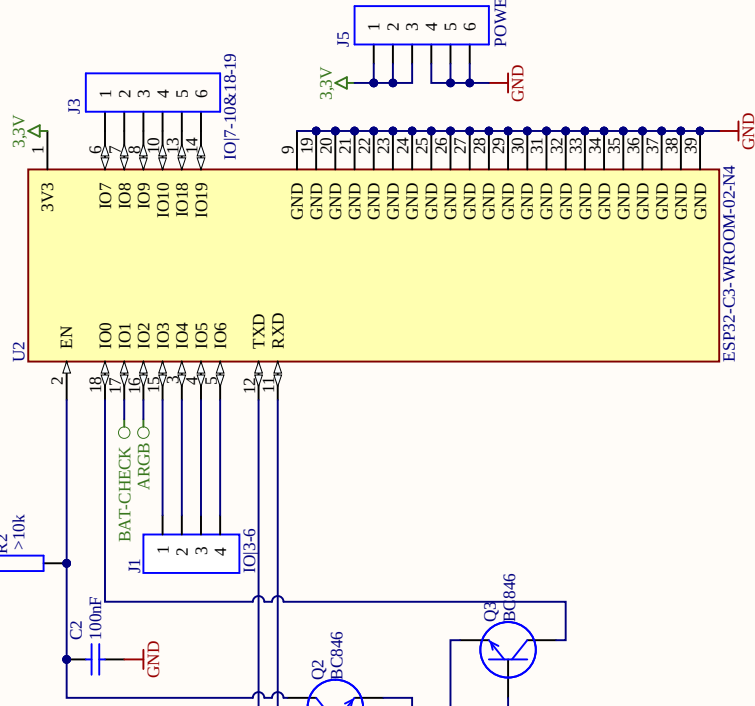
NOTE: VBUS = +5V



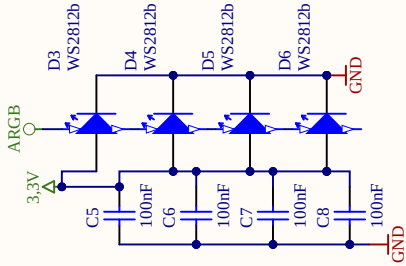
USB To UART Bridge



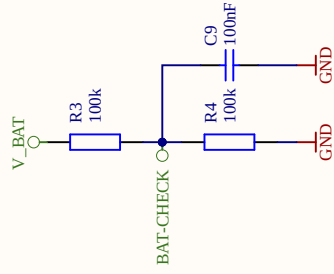
Microcontroller & Module Ports



ARGB-Leds

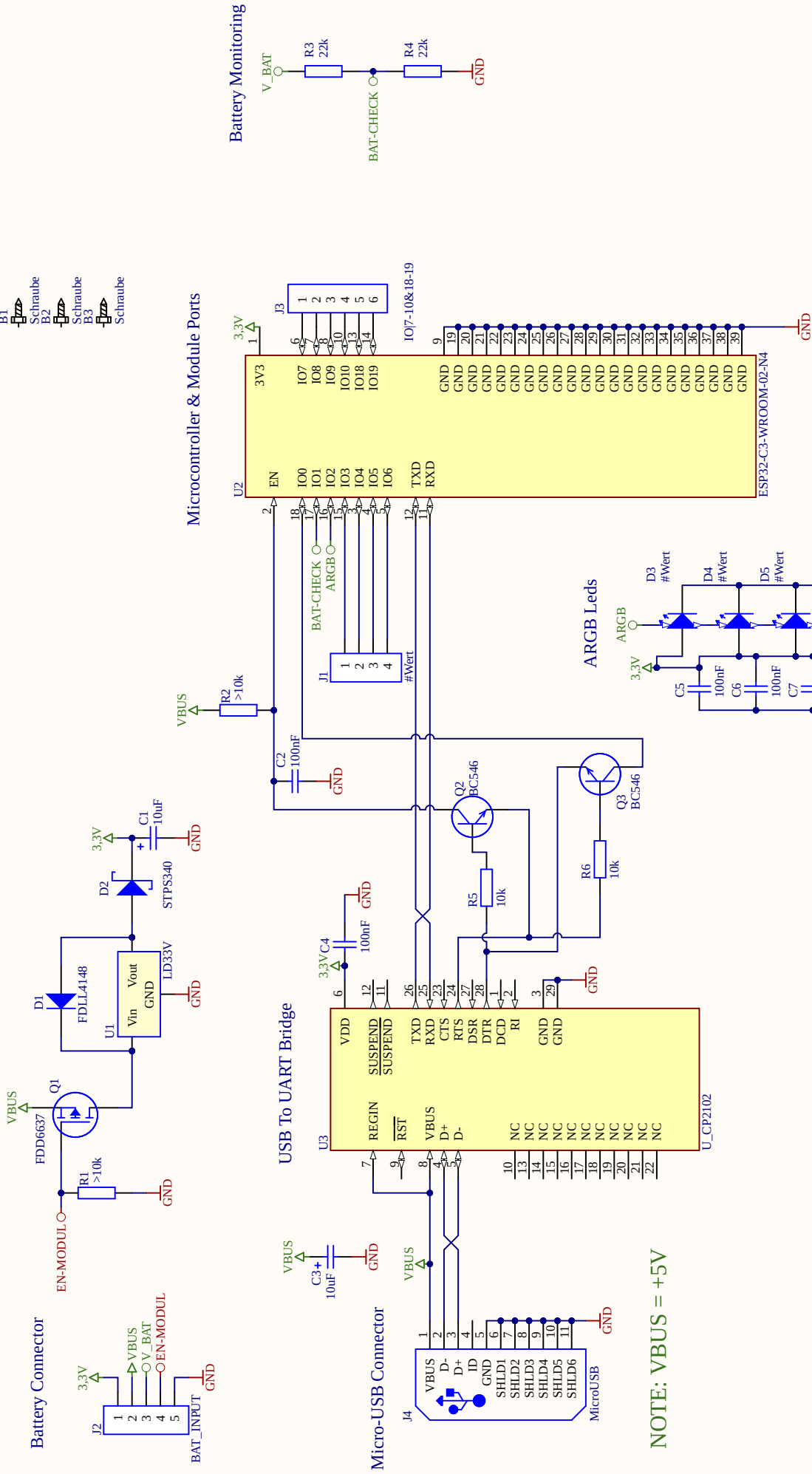


Battery Monitoring



Title		
NODE		
Size	Number	Revision
A4		V1.1
Date:	3.31.2025	Sheet of
File:	C:\Diplomarbeit\...\Sheet-NODE-V1.1.Sch	Drawn By: RK

Linear Regulator

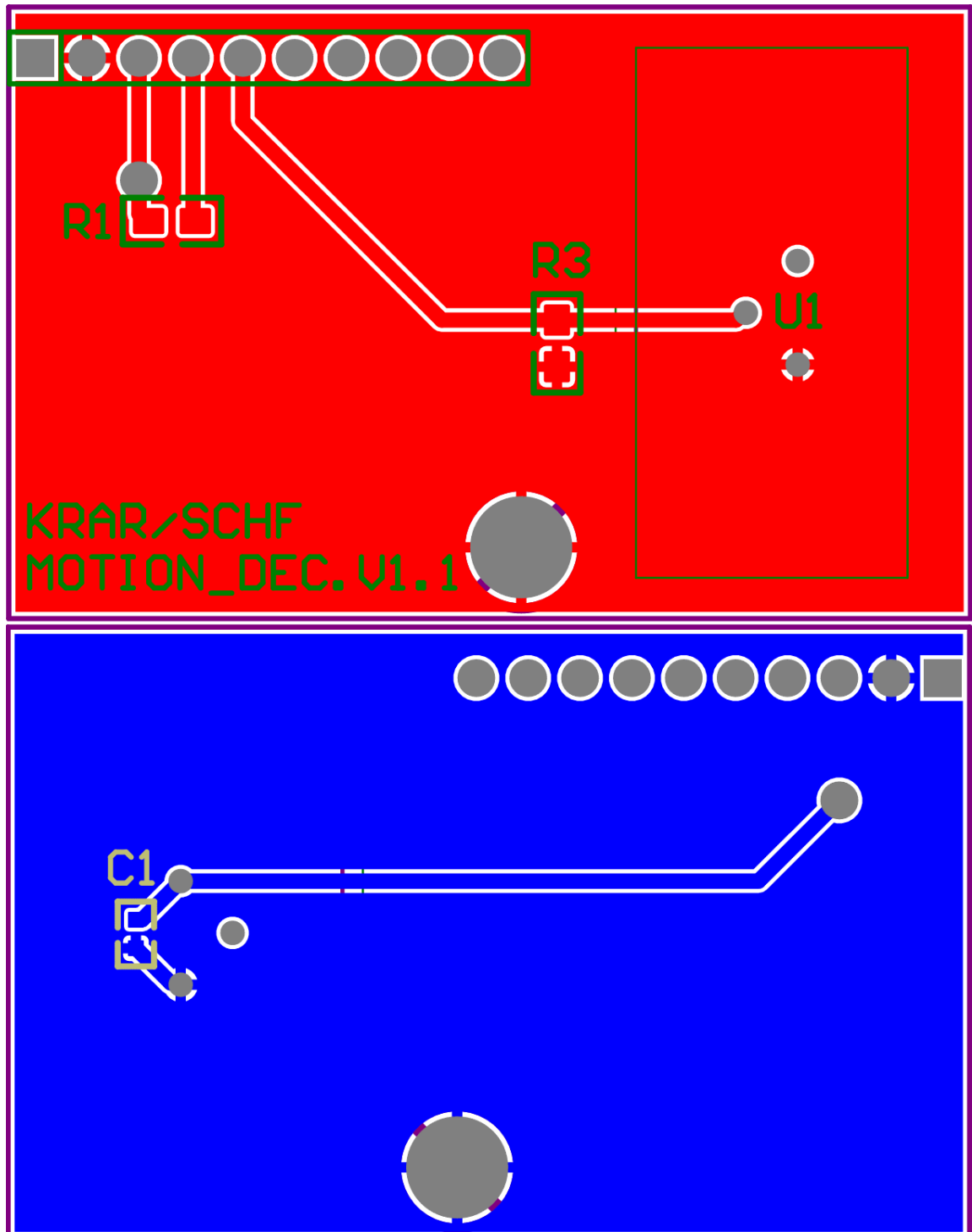


NODE

Title		Revision	
Size	Number	V1.0	
A4			
Date:	3.31.2025	Sheet of	
File:	C:\Diplomarbeit\Sheet-NODE-V1.0.Sch	Drawn By:	
		RK	

10.2 Motion Module

Motion Module V1.1



Motion Module V1.1 - BOM

Comment	Description	Designator	Footprint	LibRef	Quantity
Schraube	Schraubenloch für M3 Schrauben	B1	B_Schraube-M3 PAD	B_Schraube	1
10µF	Kondensator SMD Universal, Keramik, Folie, usw.	C1	C_SMD_0603	C_SMD_KO	1
MODULE_COM	Stecker	J1	SP-10	J_THT_CON10	1
22k	Widerstand SMD	R1	R_SMD_0805	R_SMD_Widerstand	1
>33k	Widerstand SMD	R3	R_SMD_0805	R_SMD_Widerstand	1
EKMB1305111K	Undefined or Miscellaneous	U1	EKMB1305111K	EKMB1305111K	1



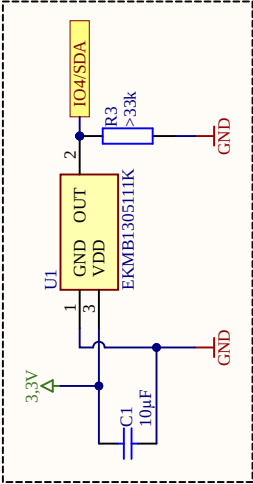
A

B

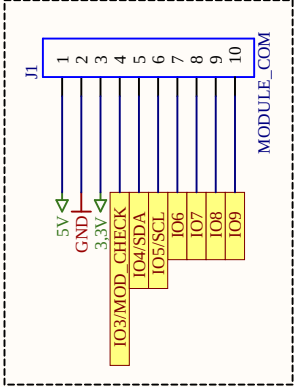
C

D

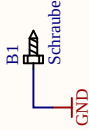
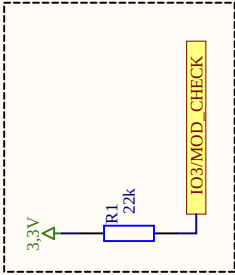
IFR - Sensor



Module-Port



Module-Detection



Title

MOTION

Size

Number

Revision

A4

V1.1

Date:

3.26.2025

Sheet of

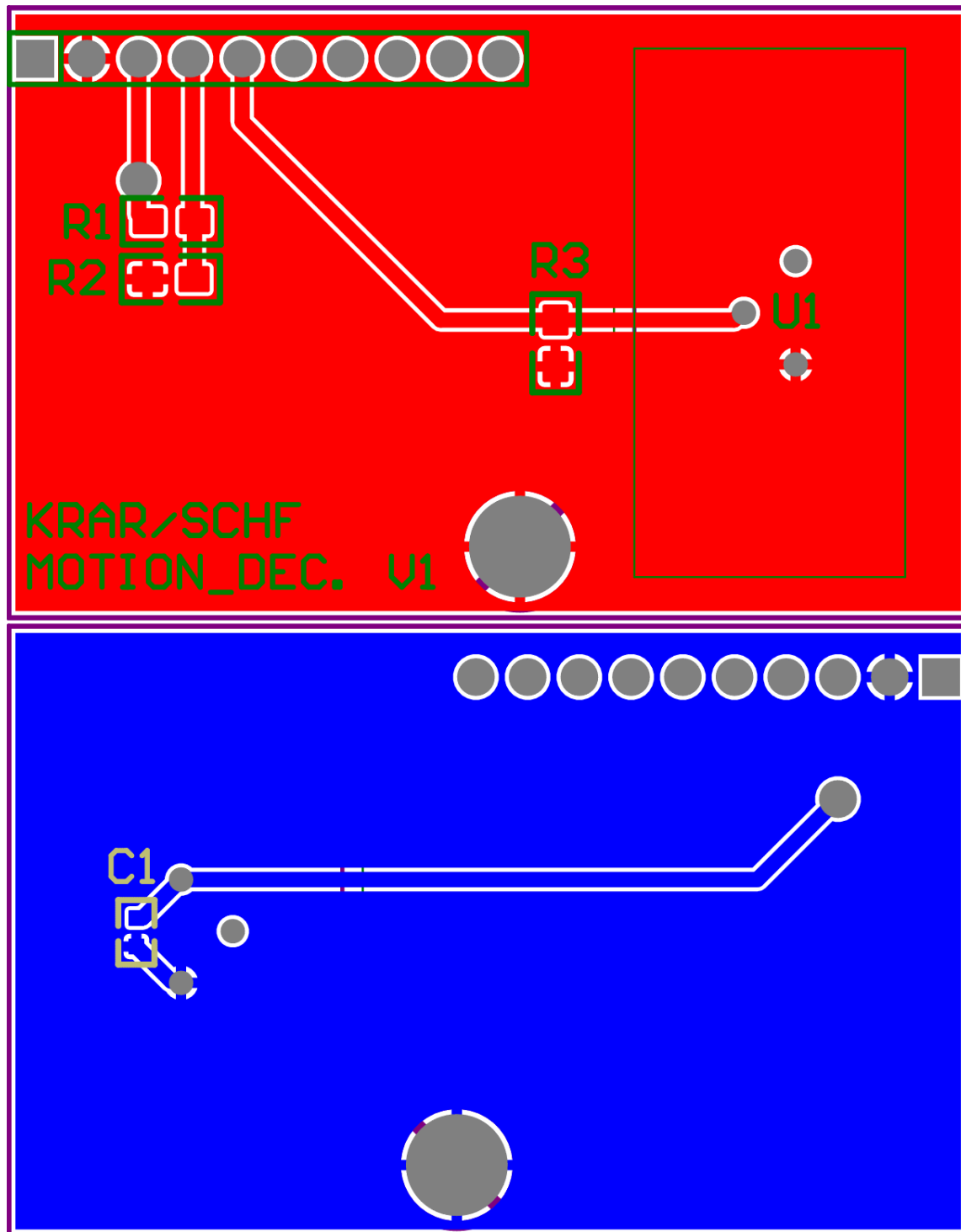
File:

C:\FaRISmarthome\..Sheet1.SchDoc

Drawn By:

RK

Motion Module V1.0

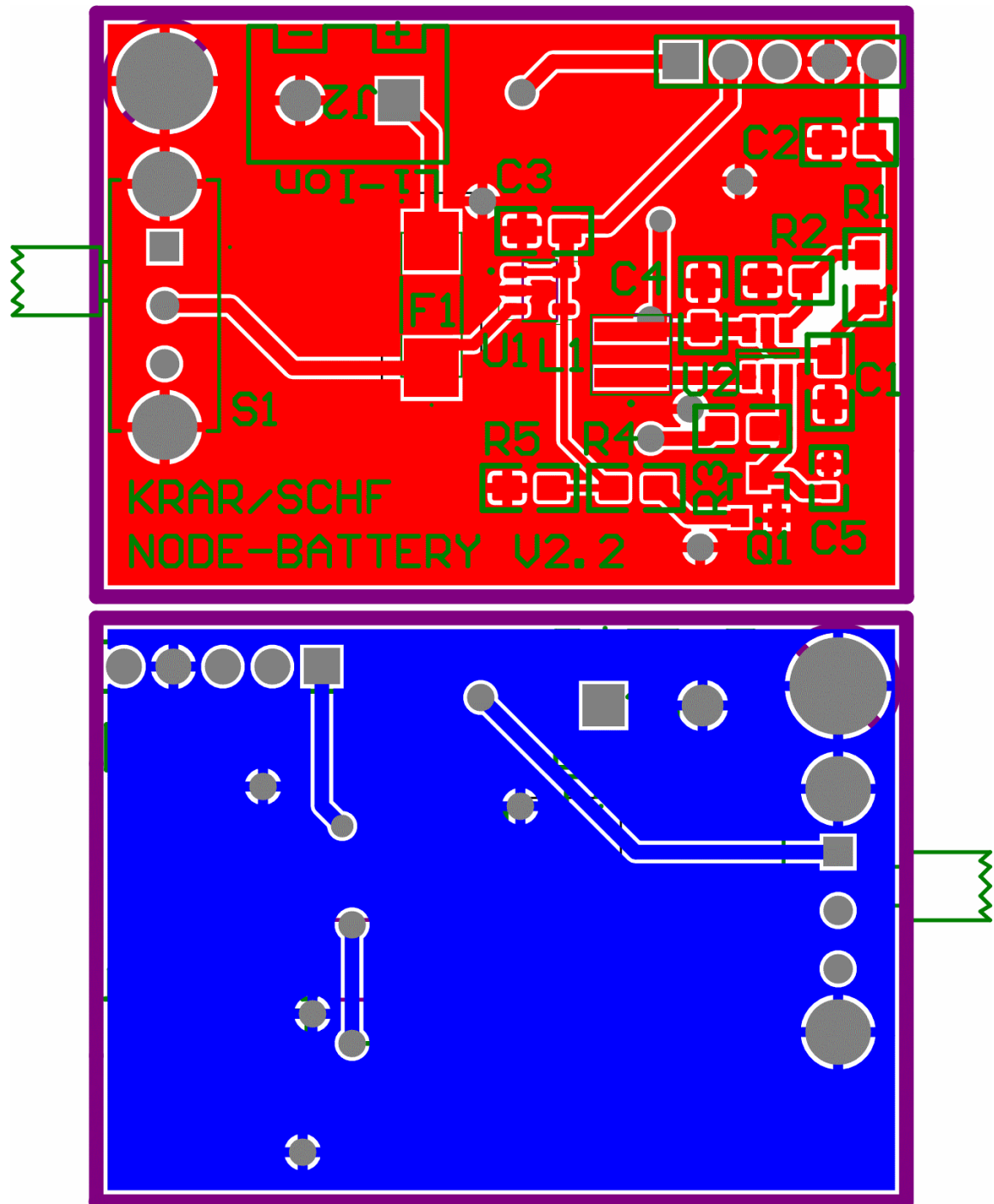


Motion Module V1.0 - BOM

Comment	Description	Designator	Footprint	LibRef	Quantity
Schraube	Schraubenloch für M3 Schrauben	B1	B_Schraube-M3 PAD	B_Schraube	1
10µF	Kondensator SMD Universal, Keramik, Folie, usw.	C1	C_SMD_0603	C_SMD_KO	1
MODULE_COM	Stecker	J1	SIP-10	J_THT_CON10	1
330k	Widerstand SMD	R1	R_SMD_0805	R_SMD_Widerstand	1
680k	Widerstand SMD	R2	R_SMD_0805	R_SMD_Widerstand	1
>33k	Widerstand SMD	R3	R_SMD_0805	R_SMD_Widerstand	1
EKMB1305111K	Undefined or Miscellaneous	U1	EKMB1305111K	EKMB1305111K	1

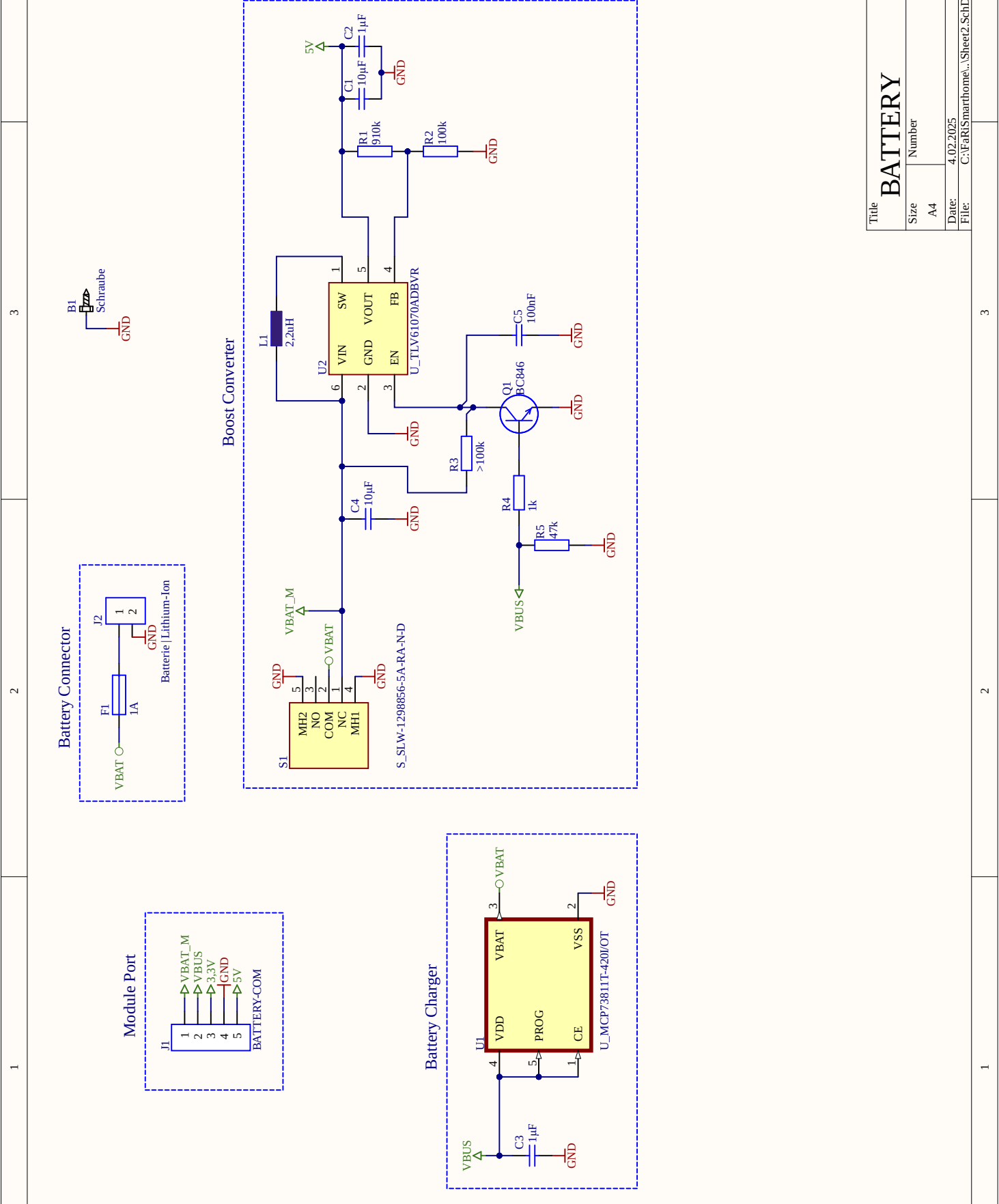
10.3 Battery Module

Battery Module V2.2

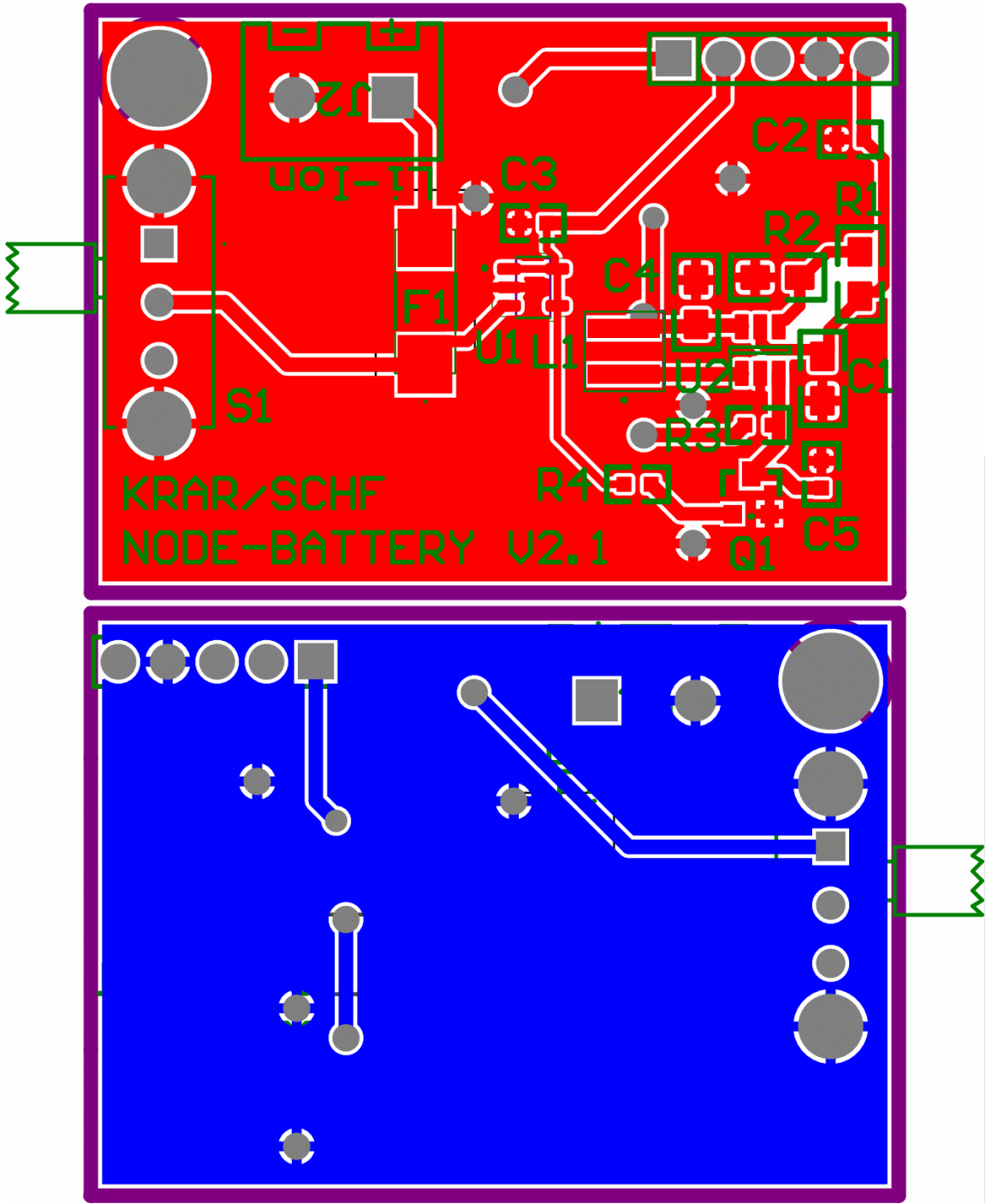


Battery Module V2.2 - BOM

Comment	Description	Designator	Footprint	LibRef	Quantity
Schraube	Schraubenloch für M3 Schrauben	B1	B_Schraube-M3 PAD	B_Schraube	1
10µF	Kondensator SMD Universal, Keramik, Folie, usw.	C1, C4	C_SMD_0805	C_SMD_KO	2
1µF	Kondensator SMD Universal, Keramik, Folie, usw.	C2, C3	C_SMD_0805	C_SMD_KO	2
100nF	Kondensator SMD Universal, Keramik, Folie, usw.	C5	C_SMD_0603	C_SMD_KO	1
1A	Sicherung	F1	F_OMF63_Schurtner	F_THT_Fuse	1
BATTERY-COM	Stecker	J1	SIP-5	J_THT_CON5	1
Batterie Lithium-Ion	Stecker	J2	J_THT_Printklemme 2x	J_THT_CON2	1
2,2uH	Spule SMD	L1	L_SMD_74438357100	L_SMD_Spule	1
BC846	NPN Transistor	Q1	SOT-23-3_BC846	Q_SMD_NPN	1
910k	Widerstand SMD	R1	R_SMD_0805	R_SMD_Widerstand	1
100k	Widerstand SMD	R2	R_SMD_0805	R_SMD_Widerstand	1
>100k	Widerstand SMD	R3	R_SMD_0805	R_SMD_Widerstand	1
1k	Widerstand SMD	R4	R_SMD_0805	R_SMD_Widerstand	1
47k	Widerstand SMD	R5	R_SMD_0805	R_SMD_Widerstand	1
S_SLW-1298856-5A-RAN-D	Switch	S1	SLW12988565ARAND	S_SLW-1298856-5A-RAN-D	1
U_MCP73811T-420I/OT	Charger IC Lithium Ion/ Polymer SOT-23-5	U1	SOT95P280X145-5N	U_MCP73811T-420I/OT	1
U_TLV61070ADBVR	Integrated Circuit, Boost converter: Input 0.5 - 5.5V Output 2.2 - 5.5V	U2	SOT23-6	U_TLV61070ADBVR	1

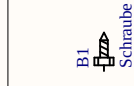


Battery Module V2.1



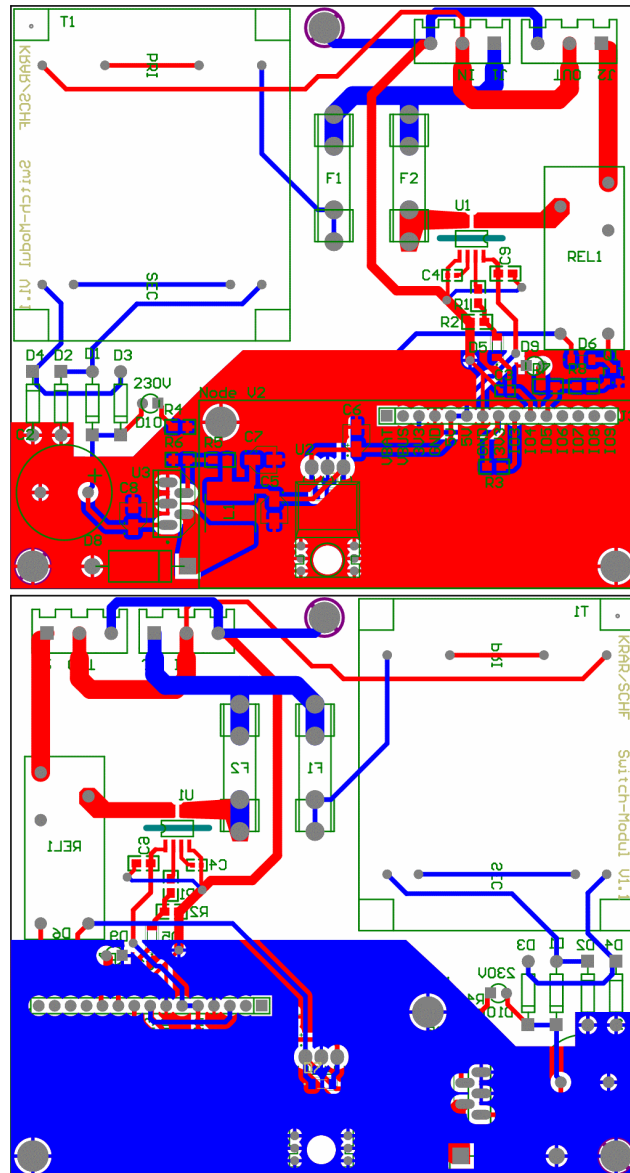
Battery Module V2.1 - BOM

Comment	Description	Designator	Footprint	LibRef	Quantity
Schraube	Schraubenloch für M3 Schrauben	B1	B_Schraube-M3 PAD	B_Schraube	1
10µF	Kondensator SMD Universal, Keramik, Folie, usw.	C1, C4	C_SMD_0805	C_SMD_KO	2
1µF	Kondensator SMD Universal, Keramik, Folie, usw.	C2, C3	C_SMD_0603	C_SMD_KO	2
100nF	Kondensator SMD Universal, Keramik, Folie, usw.	C5	C_SMD_0603	C_SMD_KO	1
1A	Sicherung	F1	F_OMF63_Schurtner	F_THT_Fuse	1
BATTERY-COM	Stecker	J1	SIP-5	J_THT_CON5	1
Batterie Lithium-Ion	Stecker	J2	J_THT_Printklemme 2x	J_THT_CON2	1
2,2µH	Spule SMD	L1	L_SMD_74438357100	L_SMD_Spule	1
BC846	NPN Transistor	Q1	SOT-23-3_BC846	Q_SMD_NPN	1
910k	Widerstand SMD	R1	R_SMD_0805	R_SMD_Widerstand	1
100k	Widerstand SMD	R2	R_SMD_0805	R_SMD_Widerstand	1
>100k	Widerstand SMD	R3	R_SMD_0603	R_SMD_Widerstand	1
1k	Widerstand SMD	R4	R_SMD_0603	R_SMD_Widerstand	1
S_SLW-1298856-5A-RAN-D	Switch	S1	SLW12988565ARAND	S_SLW-1298856-5A-RAN-D	1
U_MCP73811T-420I/OT	Charger IC Lithium Ion/ Polymer SOT-23-5	U1	SOT95P280X145-5N	U_MCP73811T-420I/OT	1
U_TLV61070ADBVR	Integrated Circuit, Boost converter: Input 0.5 - 5.5V Output 2.2 - 5.5V	U2	SOT23-6	U_TLV61070ADBVR	1



10.4 Switch Module

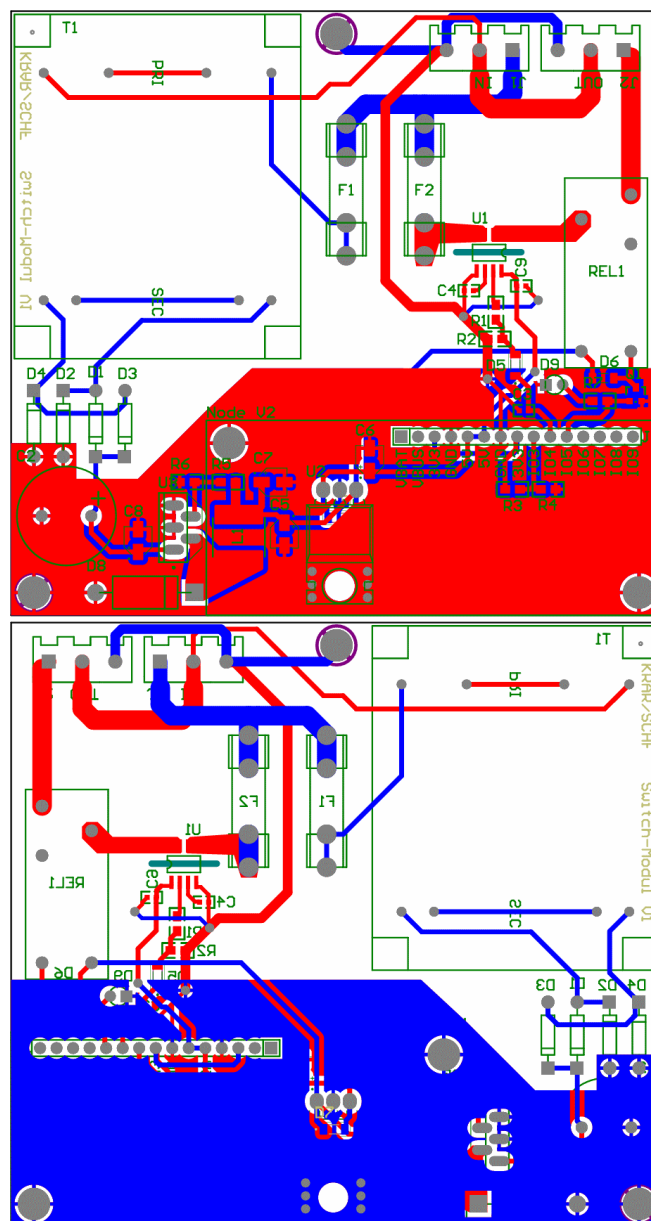
Switch Module V1.1



Switch Module V1.1 - BOM

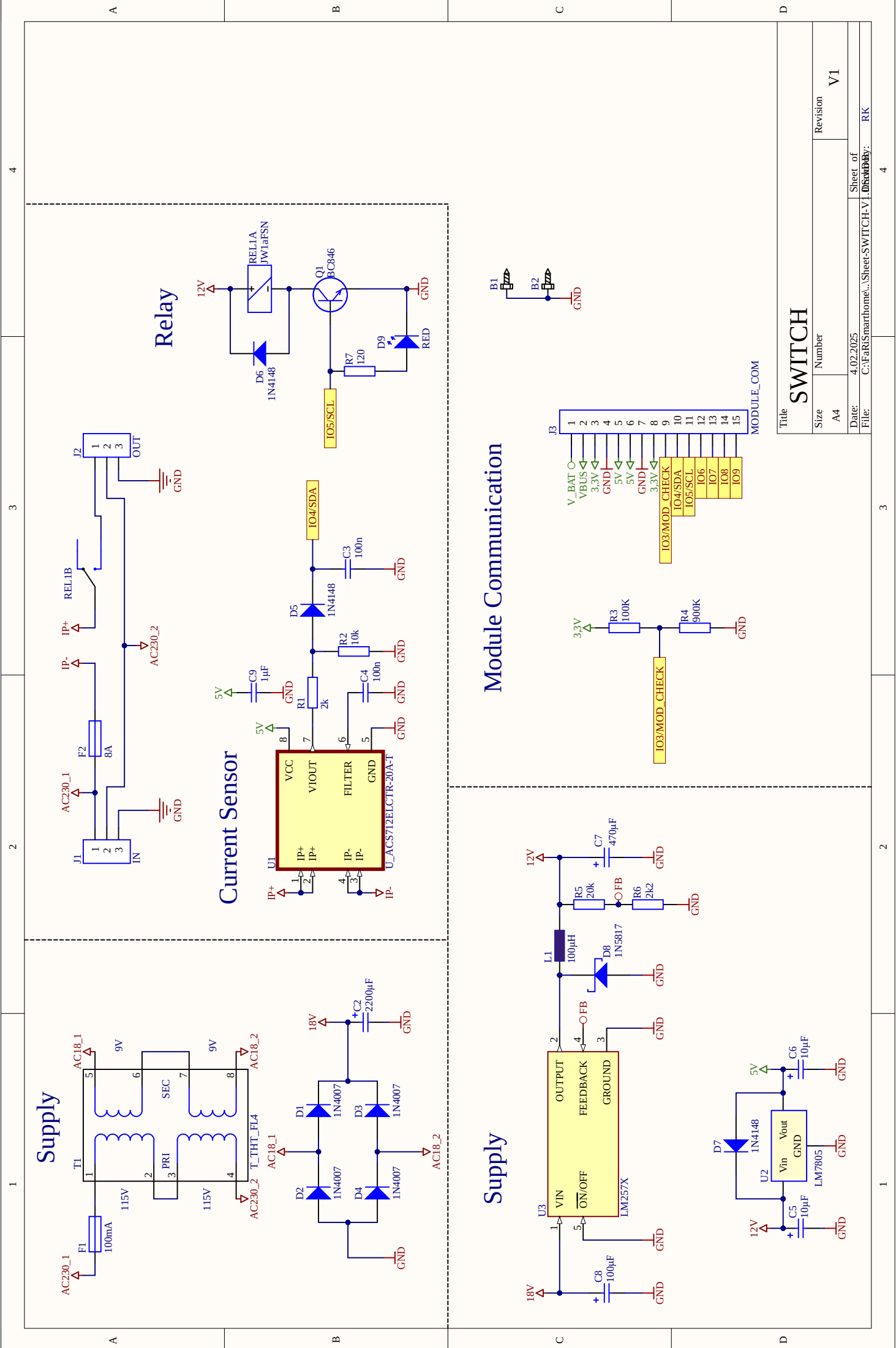
Comment	Description	Designator	Footprint	LibRef	Quantity
Schraube	Schraubenloch für M3 Schrauben	B1, B2	B_Schraube-M3 PAD	B_Schraube	2
2200µF	THT Elektrolytkondensator	C2	C_THT_RB-.3/.6	C_THT_ELK	1
100n	Kondensator SMD Universal, Keramik, Folie, usw.	C3, C4	C_SMD_0603	C_SMD_KO	2
10µF	SMD Elektrolytkondensator	C5, C6	C_SMD_B-Typ	C_SMD_ELK	2
470µF	SMD Elektrolytkondensator	C7	C_SMD_B-Typ	C_SMD_ELK	1
100µF	SMD Elektrolytkondensator	C8	C_SMD_B-Typ	C_SMD_ELK	1
1µF	Kondensator SMD Universal, Keramik, Folie, usw.	C9	C_SMD_0805	C_SMD_KO	1
1N4007	Diode THT Silizium	D1, D2, D3, D4	D_THT_0.4	D_THT_Silizium	4
1N4148	Diode SMD Silizium	D5, D6, D7	SOD-80-2	D_SMD_Silizium	3
1N5817	Diode THT Schottky	D8	D_THT_0.6	D_THT_Schottky	1
RED	Diode THT LED	D9, D10	D_THT_LED-3	D_THT_LED	2
100mA	Sicherung	F1	F_THT_Fuse2pol	F_THT_Fuse	1
8A	Sicherung	F2	F_THT_Fuse2pol	F_THT_Fuse	1
IN	Stecker	J1	J_THT_Printklemme 3x	J_THT_CON3	1
OUT	Stecker	J2	J_THT_Printklemme 3x	J_THT_CON3	1
MODULE_COM	Stecker	J3	SIP-15 Node_V2	J_THT_NODE_V2	1
100µH	Spule SMD	L1	L_SMD_NPIS42LS	L_SMD_Spule	1
BC846	NPN Transistor	Q1	SOT-23-3_BC846	Q_SMD_NPN	1
2k	Widerstand SMD	R1	R_SMD_0805	R_SMD_Widerstand	1
10k	Widerstand SMD	R2, R3	R_SMD_0805	R_SMD_Widerstand	2
2k2	Widerstand SMD	R4, R6	R_SMD_0805	R_SMD_Widerstand	2
20k	Widerstand SMD	R5	R_SMD_0805	R_SMD_Widerstand	1
120	Widerstand SMD	R7	R_SMD_0805	R_SMD_Widerstand	1
1k	Widerstand SMD	R8	R_SMD_0805	R_SMD_Widerstand	1
JW1aFSN	Relais 1 Öffner, 1 Schließer	REL1	Rel_THT_1x 2	Rel_THT_1x2	1
T_THT_FL4	Flachtrafo von Block 4VA 2x Primär 2x Sekundär	T1	FL_4_Trafo	T_THT_FL4	1
U_ACS712ELCTR-20A-T	Current Sensor 20A 1 Channel Hall Effect, Open Loop Bidirectional 8-SOIC (0.154, 3.90mm Width)	U1	SO-8	U_ACS712ELCTR-20A-T	1
LM7805	Linearwandler	U2	TO-220-3H	U_THT_78xx	1
LM257X	Spannungswandler Step Up/Down	U3	TO220-5V	U_THT_LM25xx	1

Switch Module V1.0



Switch Module V1.1 - BOM

Comment	Description	Designator	Footprint	LibRef	Quantity
Schraube	Schraubenloch für M3 Schrauben	B1, B2	B_Schraube-M3 PAD	B_Schraube	2
2200µF	THT Elektrolytkondensator	C2	C_THT_RB-.3/.6	C_THT_ELK	1
100n	Kondensator SMD Universal, Keramik, Folie, usw.	C3, C4	C_SMD_0603	C_SMD_KO	2
10µF	SMD Elektrolytkondensator	C5, C6	C_SMD_B-Typ	C_SMD_ELK	2
470µF	SMD Elektrolytkondensator	C7	C_SMD_B-Typ	C_SMD_ELK	1
100µF	SMD Elektrolytkondensator	C8	C_SMD_B-Typ	C_SMD_ELK	1
1µF	Kondensator SMD Universal, Keramik, Folie, usw.	C9	C_SMD_0603	C_SMD_KO	1
1N4007	Diode THT Silizium	D1, D2, D3, D4	D_THT_0.4	D_THT_Silizium	4
1N4148	Diode SMD Silizium	D5, D6, D7	SOD-80-2	D_SMD_Silizium	3
1N5817	Diode THT Schottky	D8	D_THT_0.6	D_THT_Schottky	1
RED	Diode THT LED	D9	D_THT_LED-3	D_THT_LED	1
100mA	Sicherung	F1	F_THT_Fuse2pol	F_THT_Fuse	1
8A	Sicherung	F2	F_THT_Fuse2pol	F_THT_Fuse	1
IN	Stecker	J1	J_THT_Printklemme 3x	J_THT_CON3	1
OUT	Stecker	J2	J_THT_Printklemme 3x	J_THT_CON3	1
MODULE_COM	Stecker	J3	SIP-15_Node_V2	J_THT_NODE_V2	1
100µH	Spule SMD	L1	L_SMD_NPIS42LS	L_SMD_Spule	1
BC846	NPN Transistor	Q1	SOT-23-3_BC846	Q_SMD_NPN	1
2k	Widerstand SMD	R1	R_SMD_0805	R_SMD_Widerstand	1
10k	Widerstand SMD	R2	R_SMD_0805	R_SMD_Widerstand	1
100K	Widerstand SMD	R3	R_SMD_0805	R_SMD_Widerstand	1
900K	Widerstand SMD	R4	R_SMD_0805	R_SMD_Widerstand	1
20k	Widerstand SMD	R5	R_SMD_0805	R_SMD_Widerstand	1
2k2	Widerstand SMD	R6	R_SMD_0805	R_SMD_Widerstand	1
120	Widerstand SMD	R7	R_SMD_0805	R_SMD_Widerstand	1
JW1aFSN	Relais 1 Öffner, 1 Schließer	REL1	Rel_THT_1x 2	Rel_THT_1x2	1
T_THT_FL4	Flachtrafo von Block 4VA 2x Primär 2x Sekundär	T1	FL_4_Trafo	T_THT_FL4	1
U_ACS712ELCTR-20A-T	Current Sensor 20A 1 Channel Hall Effect, Open Loop Bidirectional 8-SOIC (0.154, 3.90mm Width)	U1	SO-8	U_ACS712ELCTR-20A-T	1
LM7805	Linearwandler	U2	TO-220-3H	U_THT_78xx	1
LM257X	Spanungswandler Step Up/Down	U3	TO220-5V	U_THT_LM25xx	1



Transcript of the first thesis seminar

Topic: Modular Smart Home System
Candidates: Fabian Schätzschock, Richard Krammer

Year: 5BHELS
Supervisor: RZEPA
Location: HTL St. Pölten W120
Date: 23.10.2024
Time: 7:25

Tasks:

Fabian Schätzschock

State

Task	Done
Communication between ESP32 nodes	OK
Networking Concept	OK

To be done

Task	Done at
Communication between Raspberry Pi and ESP32	07.11.2024
Interface between Webserver and Database	21.11.2024
Available Website hosted in Network	21.11.2024

Richard Krammer

State

Task	Done
Node V1 Schematic and PCB	OK
Battery V1 Schematic and PCB	OK
Node V2 Schematic and PCB	OK

To be done

Task	Done at
Switch V1 Schematic	6.11.24
Switch V1 PCB	6.11.24

Transcript of the second thesis seminar

Topic: Modular Smart Home System
Candidates: Fabian Schätzschock, Richard Krammer

Year: 5BHELS
Supervisor: RZEPA
Location: HTL St. Pölten W120
Date: 04.12.2024
Time: 7:25

Tasks:

Fabian Schätzschock

State

Task	Done
Available Website hosted in Network	Not completed
Communication between Raspberry Pi and ESP32	Not completed

To be done

Task	Done at
Communication between Raspberry Pi and ESP32	18.12.2024
Interface between Webserver and Database	18.12.2024
Available Website hosted in Network	18.12.2024

Richard Krammer

State

Task	Done
Switch V1 PCB and Schematic	OK
Motion-Detection V1	OK
Ordering the PCB	OK

To be done

Task	Done at
Assemble all PCBs and testing	28.12.24
Fix all Issues in the schematic	28.12.24

Transcript of the third thesis seminar

Topic: Modular Smart Home System
Candidates: Fabian Schätzschock, Richard Krammer

Year: 5BHELS
Supervisor: RZEPA
Location: HTL St. Pölten W120
Date: 29.01.2025
Time: 16:40

Tasks:

Fabian Schätzschock

State

Task	Done
Communication between Raspberry Pi and ESP32	OK
Interface between Webserver and Database	SCRAPED

To be done

Task	Done at
Completing Bridge Communication	12.02.2025
Implementing Node Source Code	19.02.2025
WebUI Prototype	26.02.2025

Richard Krammer

State

Task	Done
Assemble all PCBs and testing	OK
Fix all Issues in the schematic	OK
New revision of the modules ordered	OK

To be done

Task	Done at
Assemble the new versions and testing	19.02.25

Transcript of the fourth thesis seminar

Topic: Modular Smart Home System
Candidates: Fabian Schätzschock, Richard Krammer

Year: 5BHELS
Supervisor: RZEPA
Location: HTL St. Pölten W120
Date: 05.03.2025
Time: 07:30

Tasks:

Fabian Schätzschock

State

Task	Done
Implementing Node Source Code	OK
WebUI Prototype	OK
Completing Bridge Communication	PARTLY

To be done

Task	Done at
Completing Bridge to Server Communication	16.03.2025
Implementing the Bridge Source Code	16.03.2025
Finishing Documentation	24.03.2025

Richard Krammer

State

Task	Done
Assemble the new versions and testing	OK

To be done

Task	Done at
Design case for modules	26.03.25
Assemble modules	28.03.25

Bibliography

- [1] *ACS712*. English. Feb. 2024. URL: https://www.mouser.at/datasheet/2/1115/ACS712_Datasheet-3178682.pdf (visited on 12/09/2024).
- [2] *Autodesk Fusion | 3D CAD, CAM, CAE, & PCB Cloud-Based Software | Autodesk*. en-US. URL: <https://www.autodesk.com/products/fusion-360/overview> (visited on 03/31/2025).
- [3] *Boot Mode Selection - ESP32-C3 - — esptool.py latest documentation*. English. URL: <https://docs.espressif.com/projects/esptool/en/latest/esp32c3/advanced-topics/boot-mode-selection.html> (visited on 03/30/2025).
- [4] *Bundesgesetz über die Ordnung von Unterricht und Erziehung in den im Schulorganisationsgesetz geregelten Schulen*. 1986. URL: <https://www.ris.bka.gv.at/GeltendeFassung.wxe?Abfrage=Bundesnormen&Gesetzesnummer=10009600>.
- [5] *Data transmission reliability over ESP-NOW protocol in indoor environment*. Espressif Developer Portal. Section: blog. Dec. 19, 2024. URL: <https://developer.espressif.com/blog/reliability-esp-now/> (visited on 12/29/2024).
- [6] *Dockerfile reference*. Docker Documentation. 100. URL: <https://docs.docker.com/reference/dockerfile/> (visited on 03/28/2025).
- [7] *DS18B20*. English. URL: <https://mm.digikey.com/Volume0/opasdata/d220001/medias/docus/2890/UMW%20DS18B20.pdf> (visited on 10/12/2024).
- [8] *ESP-NOW Wireless Communication Protocol | Espressif Systems*. URL: <https://www.espressif.com/en/solutions/low-power-solutions/esp-now> (visited on 04/02/2025).
- [9] *ESP32-C3*. English. 2023. URL: https://www.espressif.com/sites/default/files/documentation/esp32-c3-wroom-02_datasheet_en.pdf (visited on 09/09/2024).
- [10] *ESP32-S3 ETH Development Board, 10/100Mbps RJ45 Ethernet port, Wi-Fi & Bluetooth Support, 240MHz Dual Core Processor, ESP32 Ethernet Development Board | ESP32-S3-ETH*. URL: <https://www.waveshare.com/esp32-s3-eth.htm> (visited on 03/30/2025).
- [11] *ESP32-S3-WROOM-2*. English. 2022. URL: https://cdn-shop.adafruit.com/product-files/5364/esp32-s3-wroom-2_datasheet_en_0.pdf (visited on 12/29/2024).
- [12] *FreeRTOS Documentation - FreeRTOS™*. URL: https://freertos.org/Documentation/02-Kernel/07-Books-and-manual/01-RTOS_book (visited on 03/20/2025).

-
- [13] *JW Relays Datasheet*. English. Dec. 2024. URL: https://industry.panasonic.com/global/en/downloads/?file_cd=300212&tab=catalog&series_cd=1140 (visited on 04/02/2025).
- [14] *lm2575*. English. Aug. 2015. URL: https://www.ti.com/lit/ds/symlink/lm2575.pdf?ts=1732093174469&ref_url=https%253A%252F%252Fwww.ti.com%252Fproduct%252Fde-de%252FLM2575 (visited on 11/20/2024).
- [15] *Low profile transformer FL Range*. Deutsch, English. URL: <https://asset.conrad.com/media10/add/160267/c1/-/gl/000710247DS01/datenblatt-710247-block-fl-49-printtransformator-2-x-115-v-2-x-9-vac-4-va-222-ma.pdf>.
- [16] *MCP73811*. English. 2007. URL: <https://ww1.microchip.com/downloads/en/DeviceDoc/22036b.pdf> (visited on 09/09/2024).
- [17] *MQTT - The Standard for IoT Messaging*. URL: <https://mqtt.org/> (visited on 01/15/2025).
- [18] *Nuxt: The Progressive Web Framework*. Nuxt. URL: <https://nuxt.com/> (visited on 03/28/2025).
- [19] *PCB-Design Software | Innovation für das PCB-Design | Altium*. de. URL: <https://www.altium.com/de> (visited on 03/31/2025).
- [20] *PIR MOTION SENSORS 2019*. English. 2019. URL: https://cdn.sparkfun.com/assets/3/f/8/8/1/4541_fileversion.pdf (visited on 12/10/2024).
- [21] PlatformIO. *PlatformIO: Your Gateway to Embedded Software Development Excellence*. en. URL: <https://platformio.org> (visited on 01/20/2025).
- [22] *Prüfungsordnung BMHS, Bildungsanstalten*. 2012. URL: <https://www.ris.bka.gv.at/GeltendeFassung.wxe?Abfrage=Bundesnormen&Gesetzesnummer=20007845>.
- [23] Phil Salmony. *Fügen Sie USB Typ-C Power Delivery Ihren Entwürfen hinzu!* de. Nov. 2023. URL: <https://resources.altium.com/de/p/add-usb-type-c-power-delivery-your-designs> (visited on 09/04/2024).
- [24] *tlv61070a.pdf*. English. Sept. 2022. URL: https://www.ti.com/lit/ds/symlink/tlv61070a.pdf?ts=1730839090595&ref_url=https%253A%252F%252Fwww.ti.com%252Fproduct%252FTLV61070A.
- [25] *tlv62569*. English. Oct. 2017. URL: https://www.ti.com/lit/ds/symlink/tlv62569.pdf?ts=1743073852523&ref_url=https%253A%252F%252Fwww.ti.com%252Fproduct%252FTLV62569 (visited on 09/12/2024).
- [26] *tps6300x*. English. Oct. 2015. URL: https://www.ti.com/lit/ds/symlink/tps63000.pdf?ts=1725800412426&ref_url=https%253A%252F%252Fwww.mouser.cn%252F (visited on 09/09/2024).

- [27] *Visual Studio Code - Code Editing. Redefined.* URL: <https://code.visualstudio.com/> (visited on 03/20/2025).
- [28] *Welcome to Flask — Flask Documentation (3.1.x).* URL: <https://flask.palletsprojects.com/en/stable/> (visited on 03/28/2025).
- [29] *What is Arduino?* URL: <https://www.arduino.cc/en/Guide/Introduction/> (visited on 04/02/2025).
- [30] *What is Docker?* Docker Documentation. 200. URL: <https://docs.docker.com/get-started/docker-overview/> (visited on 03/28/2025).
- [31] *WS2812B.* English. URL: <https://cdn-shop.adafruit.com/datasheets/WS2812B.pdf> (visited on 04/01/2025).

List of Figures

4.1	Networking Structure	10
4.2	UDP Discovery Process	12
4.3	ESP-NOW Discovery Process	14
5.1	ESP-NOW Object Inheritance	18
6.1	Physical comparison of the ESP32-C3 and ESP32-S3, Unit in mm. . .	22
6.2	ESP32-C3 implemented in the schematic.	23
6.3	DS18B20 implemented in the schematic.	23
6.4	RGB-LEDs implemented in the schematic.	24
6.5	USB-C Connector implemented in the schematic.	25
6.6	Serial-Port implemented in the schematic.	26
6.7	TLV62569 Converter implemented in the schematic.	27
6.8	Module-Connector implemented in the schematic.	27
6.9	Spare header implemented in the schematic.	28
6.10	Battery Monitoring implemented in the schematic.	29
6.11	Module-Detection implemented on the base module.	29
6.12	PCB of the basic node. [Red - Frontside, Blue - Backside]	30
6.13	MCU implemented in the schematic, note the missing pull-up resistor. .	31
6.14	PCB of the basic node V2.1. [Red - Frontside, Blue - Backside] . . .	32
6.15	MCU implemented in the schematic version 2.0 with pin 10 not used. .	33
6.16	PCB of the basic node V2.0. [Red - Frontside, Blue - Backside] . . .	34
6.17	Microcontroller and module connector implemented in the schematic. .	35
6.18	Battery connector implemented in the schematic.	36
6.19	DC-DC Converter implemented in the schematic.	37
6.20	Serial-to-USB Converter implemented in the schematic.	38
6.21	PCB of the basic module V1.1. [Red - Frontside, Blue - Backside] . .	38
6.22	USB-C Connector implemented in the schematic.	39
6.23	Battery implemented in the schematic.	40
6.24	Node Module Selection	41
6.25	Picture of the Waveshare ESP32-S3 ETH Development Board[10]. . .	42
6.26	Pinning of the Waveshare ESP32-S3 ETH Development Board[10]. . .	43
6.27	PIR Sensor implemented in the schematic.	44
6.28	Module-Connector implemented in the schematic.	45
6.29	Module detection implemented in the schematic.	45
6.30	PCB of the motion module V1.1. [Red - Frontside, Blue - Backside] .	46
6.31	Module detection implemented in the schematic.	47
6.32	PCB of the motion module V1.0. [Red - Frontside, Blue - Backside] .	48
6.33	Module-Connector implemented in the schematic.	50

6.34	Battery implemented in the schematic.	50
6.35	Battery-Management-System implemented in the schematic.	51
6.36	Boost-Converter implemented in the schematic.	52
6.37	PCB of the battery module V2.2. [Red - Frontside, Blue - Backside] .	53
6.38	Boost-Converter implemented in version 2.1.	54
6.39	PCB of the battery module V2.1. [Red - Frontside, Blue - Backside] .	55
6.40	Battery implemented in the schematic.	56
6.41	DC-DC-Converter implemented in the schematic.	57
6.42	PCB of the battery module V1.0. [Red - Frontside, Blue - Backside] .	58
6.43	Power-Supply implemented in the schematic.	59
6.44	Power-Supply implemented in the schematic.	60
6.45	Switch implemented in the schematic.	61
6.46	Switch implemented in the schematic.	61
6.47	Current-Sensor implemented in the schematic as shown in the datasheet.	62
6.48	Measurment circuit	63
6.49	Output Voltage versus Sensed Current of the ACS712.	63
6.50	Cutout to prevent leakage current.	64
6.51	PCB of the switch module V1.1 [Blue - Backside, Red - Frontside]. .	65
6.52	Missing base resistor before transistor Q1.	66
6.53	PCB of the switch module V1.0 [Blue - Backside, Red - Frontside]. .	67
6.54	Top view	68
6.55	Bottom view	69
6.56	Partly disassembled	69
6.57	Front view	70
6.58	Back view	71
6.59	Partly disassembled	71
6.60	Top view	72
6.61	Disassembled	72
6.62	Switch fully assembled with input and output cables.	73
6.63	Partly disassembled	73
6.64	Battery and motion module next to each other.	74
6.65	Basic, battery and motion module combined.	74
6.66	Base module combined with switch module.	75
7.1	Rendering process of the web page	78